The Triadic Cosmos

Toy Worlds

Joachim De Witte

The Triadic Cosmos: Toy Worlds
2026

Contents

Chapter 1

Why Toy Universes?

1.1 Introduction — Worlds You Can Hold in Your Hands

The toy universes in this book are not metaphors or illustrations. They are actual worlds: small, complete, executable universes with their own physics, inhabitants, and developmental laws.

Toy universes make intelligence tangible. They turn abstract ideas into living systems that can be explored, modified, and understood from the inside.

They are the bridge between intuition and architecture.

1.2 Pedagogical Worlds — Learning Through Play

Toy universes exist because intelligence is easier to understand when you can see it, touch it, and experiment with it.

Each toy world is:

- small enough to understand,
- rich enough to surprise,
- structured enough to reveal deep principles.

They let readers learn the way scientists learn: by watching systems behave, by perturbing them, and by discovering patterns that no explanation alone could reveal.

Toy universes are pedagogical laboratories.

1.3 Tangible Examples — Concepts Made Concrete

Every major architectural concept in the book has a concrete counterpart in the toy worlds:

- Representation appears as sensory types, tokens, and perceptual geometries.
- Transformation appears as operators, activations, and internal dynamics.
- Evaluation appears as feedback signals, examiners, and oracles.
- Recursion appears as refinement loops, developmental ladders, and growth.
- Selection appears as routing, competition, and composite minds.

The toys are not illustrations. They are instantiations. They show the architecture before the architecture is named.

1.4 Experimental Worlds — Safe Places to Try Things

Toy universes invite experimentation:

- change the rules,
- modify the physics,
- add new species,
- rewrite the evaluator,
- alter the curriculum,
- build new agents.

Because the worlds are small, every change is visible. Because the physics are simple, every effect is interpretable. Because the systems are transparent, every failure is instructive.

Toy universes are safe laboratories for curiosity.

1.5 Hands-On Experience — Intelligence as a Craft

Intelligence is not only a theory. It is a craft.

Toy universes give readers hands-on experience with:

- designing agents,
- shaping environments,
- constructing feedback loops,
- tuning developmental physics,
- building composite minds,
- and watching intelligence emerge.

This is the software-engineering side of the book: the belief that understanding grows when you build things.

Toy universes let readers build.

1.6 Exploratory Systems — Worlds That Invite Modification

Each toy universe is deliberately open-ended:

- the physics can be extended,
- the agents can be diversified,

- the evaluators can be rewritten,
- the curricula can be reshaped,
- the architectures can be recombined.

The toys are not fixed. They are platforms. They invite readers to explore, extend, and reinvent.

1.7 ML Intuition — Small Worlds, Big Insights

Toy universes reveal patterns that are difficult to see in large-scale ML systems:

- activation functions shape developmental physics,
- early training dynamics determine long-term outcomes,
- memory retention defines lineage strategy,
- scaling laws behave like developmental growth,
- mixture-of-experts systems act like composite minds,
- routing temperament mirrors cognitive archetypes,
- strong experts compensate for weak controllers,
- mid-level peaks reveal hidden structure,
- symbolic worlds clarify learning dynamics.

These insights are not theoretical. They emerged from the toys.
Toy universes are microscopes for machine learning.

1.8 Reproducibility — Claims You Can Test Yourself

Toy universes are not only conceptual. They are executable.

Every architectural claim in the book can be reproduced, modified, and tested. Readers can run the code, change the rules, break the systems, repair them, and watch the universe respond.

This reproducibility is rare in discussions of intelligence. It turns speculation into experiment, and theory into practice.

Toy universes are claims you can test.

1.9 Why We Begin With Toys

Toy universes exist because:

- they make intelligence visible,
- they make architecture intuitive,
- they make learning playful,

- they make reasoning concrete,
- they make experimentation safe,
- they make modification natural,
- they make ML insights accessible,
- they make claims reproducible,
- they make the book's ideas alive.

They are the foundation on which the rest of the book stands.

Architectural Insight

Small universes make big ideas learnable.

Chapter 2

Toy Cosmos: Curvature & Horizons

Architectural Insight

Toy Cosmos is not a gravity model — it is a demonstration that recursion alone can generate structures that resemble real phenomena.

2.1 Introduction — A Universe Made of Curvature

The toy cosmos begins with the simplest possible idea: a world made entirely of curvature. Not curvature as an equation, not curvature as a tensor, but curvature as a value—a small number stored at each point on a grid. Every cell looks at its neighbors, adjusts itself, and passes the result forward. That is the whole engine.

From this minimal architecture, gravity-like behavior emerges:

- wells deepen and pull on their surroundings,
- gradients guide motion,
- horizons appear when curvature becomes extreme,
- waves ripple outward when geometry is disturbed.

There are no differential equations, no coordinate charts, no tensors with indices. The universe does not compute curvature; it negotiates it. Everything arises from local rules and recursion.

This is not a discrete theory of gravity. It is a conceptual demonstration of how **recursion produces emergent structure**.

2.2 Geometry from Local Rules

Curvature in this universe is a local pattern of influence. Each point asks its neighbors:

"How bent are you? How should I adjust to stay consistent with the world around me?"

The neighbors respond. The point updates. The pattern propagates.

From this simple conversation, geometry emerges:

- Wells form where curvature accumulates.

- Gradients appear as slopes in the field.
- Flows arise as objects drift along those slopes.

Motion is not imposed. There are no forces. Objects move because the geometry beneath them changes shape, and the shape guides their path. This is the discrete analogue of geodesic flow: motion as a consequence of form.

The universe does not know it is imitating gravity. It simply follows its rules—and familiar behavior appears.

2.3 Horizons as Emergent Boundaries

A horizon in the toy cosmos is not a singularity, not an infinite density, and not a mathematical breakdown. It is simply a threshold—a point where curvature becomes so steep that the local geometry traps anything that enters.

When curvature at a cell crosses a critical value, that region becomes a trapped zone. Nothing inside can climb back out because every neighboring update pulls it further inward.

This is the discrete analogue of an event horizon, but without the machinery of general relativity. No tensors, no infinities, no coordinate singularities—just a boundary that appears the way a whirlpool forms in water: gradually, naturally, and without being explicitly drawn.

Horizons behave like phase transitions: the geometry folds in on itself.

2.4 Motion in a Curved Toy Universe

Once curvature wells form, the universe begins to move. Objects drift because the geometry beneath them changes shape, and the shape guides their path. Motion is not commanded; it is inferred from the landscape.

Stable and unstable orbits

Some trajectories settle into graceful loops around a well. Others wobble, drift, and eventually fall inward. Stability depends on the balance between curvature gradients and initial momentum.

Inspirals and captures

Most objects that wander near a well eventually spiral inward. The curvature field relaxes locally, and this relaxation acts like a soft drag, pulling objects deeper into the well.

Slingshots and rare escapes

Escapes are possible but rare. To break free, an object must begin with enough speed to outrun the steepening curvature before the well can pull it back.

Geometry “pulls” without forces

There are no forces in this universe. No Newtonian gravity. No equations of motion. Objects move because the geometry beneath them bends, and the bending creates a path.

The universe does not compute trajectories. It shapes them.

2.5 When Black Holes Meet

When two curvature wells drift toward each other, the geometry begins to deform long before the horizons touch. Each well tugs on the other, stretching the surrounding curvature into elongated shapes.

As the trapped regions approach, a new form appears: the peanut shape. Two lobes, one for each well, connected by a narrowing waist. This shape is universal. It emerges in numerical relativity, in physical intuition, and here, in the simplest possible architecture.

Once the horizons fuse, the geometry enters a chaotic phase. The waist collapses inward. The two lobes pull on each other, overshoot, and oscillate. The curvature field reshapes itself violently before settling into a single, deeper well.

Nothing in this process is programmed. The universe is not told how to merge black holes. It simply follows its rules, and the merger emerges.

2.6 The Hexad in Motion

Every moment in the toy cosmos is a passage through the six roles of the Hexad. What looks like a drifting well, a stretching horizon, or a ringing wave is, underneath, a cycle of representation, transformation, evaluation, recursion, selection, and embedding.

- Representation → the curvature field The world is stored as a grid of curvature values.
- Transformation → the local update rule Each point adjusts itself based on its neighbors.
- Evaluation → the horizon threshold Crossing a critical value creates a trapped region.
- Recursion → repeated updates The update loop is time.
- Selection → merging of trapped regions Overlapping horizons resolve into one.
- Embedding → storing the new geometry Each update becomes the new world.

The toy cosmos is the Hexad made visible.

2.7 The Recursive Update Loop

Toy Cosmos is a minimal demonstration of emergent behaviour arising from a single recursive update loop. The entire universe is defined by two fields — a curvature field C and a list of black holes H — and all dynamics follow from repeatedly applying the same sequence of transformations. Despite its simplicity, the system exhibits drift, attraction, horizon formation, and merger events, illustrating how complex structure can emerge from a compact rule set.

Minimal Recursive Update Loop

```
Input  : Curvature field C; black-hole list H
Output: Updated fields (C, H)
for t = 1 to T do
    C ← AccumulateCurvature(H)
    C ← Relax(C)
    foreach h ∈ H do
        h.pos ← h.pos + Drift(C, h)
    end
    H ← UpdateHorizons(C, H)
    H ← MergeIfNeeded(H)
end
```

Toy Cosmos demonstrates how a universe can be defined not by a large set of rules, but by a single recursive structure. All behaviour — curvature accumulation, drift, horizon updates, and mergers — emerges from repeated application of the same minimal loop. This makes Toy Cosmos an ideal example of how simple computational laws can generate rich, interpretable dynamics.

2.8 How Toy Cosmos Connects to Safe AGI

Toy Cosmos is not an AI system, but the same principles that make Toy Cosmos stable, interpretable, and emergent also make cognitive systems safe:

- simple rules — the system is easy to understand and reason about,
- transparent recursion — every update is visible and traceable,
- emergent complexity — powerful behavior arises without hidden machinery,
- no opaque components — nothing is concealed inside black-box heuristics.

Toy Cosmos becomes a metaphor for safe cognitive geometry: a system where complexity grows from clear foundations, where recursion is bounded and interpretable, and where emergent behavior is a consequence of architecture rather than an accident of scale.

It prepares the reader for the next chapter's central argument: safe AGI requires the same clarity, modularity, and transparency that make Toy Cosmos stable. Intelligence, like curvature, must emerge from rules we understand.

2.9 Why This Toy Universe Matters

This tiny cosmos is more than a playful simulation. It is a demonstration of how gravity-like behavior can emerge from architecture alone. With no equations, no tensors, and no continuous mathematics, the universe produces wells, horizons, mergers, and waves—structures we normally associate with advanced physics.

It shows the power of locality: global behavior arises from local interactions. It shows the power of recursion: complexity emerges from repetition. It shows the power of architecture: rules shape worlds.

Architectural Insight

Recursion turns simple rules into worlds.

Chapter 3

Toy Maze Runner: Motion & Cooperation

3.1 Introduction — Life From Pure Movement

Maze Runner is the simplest universe in this book. It contains no food, no energy, no roles, no genetics, no evolution, and no ecology. Its inhabitants have no identity, no species, no goals, and no memory beyond a single action. They are tiny neural organisms that can only:

- move forward,
- turn left,
- or turn right.

And yet, from these minimal ingredients, Maze Runner produces:

- swarms that flow like schools of fish,
- orbiting pairs that dance around each other,
- clusters that form and dissolve,
- migration waves that sweep across the grid,
- and spontaneous order that feels alive.

Maze Runner is proto-life: behavior emerging from geometry, movement, and feedback alone. It is the first rung of world learning—the physics of behavior before biology.

3.2 The World — A Minimal Physics of Motion

The Maze Runner world is a grid with:

- walls that block movement,
- floor values that add texture,
- and a population of identical agents.

There is no energy, no food, no reproduction, and no death. The world is intentionally sparse. It is a laboratory for motion.

Each step, the physics engine applies a few simple rules:

- moving forward succeeds if the cell is free,
- collisions cause random turns,
- excessive turning is penalized,
- wrap-around geometry keeps the world continuous.

These rules are not meant to simulate biology. They simulate constraints. They give the agents something to push against—a world that pushes back.

3.3 The Agents — Tiny Embodied Learners

Each Maze Runner agent is a minimal embodied organism. It has:

- a position and direction,
- a tiny ReLU-based neural network,
- a last action,
- a turn streak,
- and a color (purely for visualization).

Its sensorium consists of:

- a rotated 5×5 vision grid,
- a one-hot encoding of its direction,
- normalized vectors to every other agent.

This is a machine-native sensory language: a compact, geometric representation of the world that the agent must learn to interpret.

The agent's brain transforms this sensory input into one of three actions:

- MOVE_FORWARD,
- TURN_LEFT,
- TURN_RIGHT.

There are no roles, no archetypes, no species. Every agent is identical at birth. Their individuality emerges only through learning.

3.4 Learning — The Physics of Feedback

Maze Runner uses a single reward mechanism:

- a small positive reward for moving,
- a positive reward for reducing distance to others,
- a penalty for excessive turning.

This reward is not a goal. It is a pressure. It nudges agents toward coordinated movement without telling them what to do.

Every step:

1. the agent perceives the world,
2. produces an action,
3. receives a reward,
4. computes a loss,
5. updates its weights through backpropagation.

There are no generations, no mutation, no selection. Maze Runner is continuous neuroplasticity—learning during life, not between lives.

3.5 Emergence — Cooperation Without Communication

Despite having no communication, no memory, and no roles, Maze Runner agents learn to coordinate. They develop:

- following behavior—trailing others through corridors,
- orbiting pairs—two agents circling each other indefinitely,
- clusters—groups that move as a unit,
- avoidance patterns—agents learning to dodge collisions,
- migration waves—large-scale flows across the grid.

These behaviors are not programmed. They emerge from:

- geometry,
- movement,
- feedback,
- and learning.

Maze Runner shows that cooperation does not require language, identity, or intention. It requires only embodiment and a world with consequences.

3.6 Maze Runner and the Ladder of Ontologies

Maze Runner is the first toy universe that directly expresses the Ladder in motion:

- Monad — each agent is a minimal entity,
- Dyad — interactions occur through collisions and proximity,
- Triad — information (vision), energy (reward), geometry (grid),
- Pentad — representation, transformation, evaluation, recursion, selection (implicit),
- Hexad — the world as a system with state and update rules,
- Pentadia — the precursor to ecological complexity,
- Octad — a tiny universe with its own physics,
- Nonad — computation through embodied interaction.

Maze Runner is the simplest possible instantiation of the Ladder—a world where the ontology becomes visible.

3.7 Why Maze Runner Matters

Maze Runner is not a game. It is a demonstration of a principle: **Behavior emerges before biology.**

It shows that:

- movement becomes behavior,
- behavior becomes structure,
- structure becomes cooperation.

It is the first step toward understanding how intelligence grows from the interplay of information, energy, and geometry. It prepares the reader for Pentadia, where behavior becomes ecology, and for the Hybrid Intelligence chapters, where recursion becomes reasoning.

Maze Runner is the seed of world learning—the physics of intelligence before minds.

Architectural Insight

Life begins as movement that learns.

Chapter 4

Toy Pentadia: Species & Ecologies

4.1 Introduction — From Behavior to Biology

Pentadia is the second universe in the world-learning trilogy. Where MazeRunner shows how behavior emerges from pure movement, Pentadia shows how ecology emerges from interaction, energy, and evolution.

It is a digital ecosystem with:

- food that regrows,
- agents with metabolism,
- roles that shape behavior,
- interactions with consequences,
- crossing and mutation,
- generations and selection.

Pentadia is not a simulation of biology. It is a simulation of the forces that make biology possible.

It is eco-life: a world where species, strategies, and ecologies arise from a handful of simple rules.

4.2 The World — A Living Grid

Pentadia takes place on a grid where each cell may contain:

- empty space,
- food,
- or a critter.

Food regrows stochastically. Critters consume energy as they move.

Every tick of the world follows the same five-step loop:

1. Perception — the critter senses its surroundings.

2. Decision — its neural network proposes an action.
3. Physics — the world resolves movement and interactions.
4. Learning — the critter updates its brain.
5. Cleanup — death, food regrowth, and world maintenance.

This loop is the Pentad in motion: representation, transformation, evaluation, recursion, and selection.

4.3 The Critters — Embodied Organisms With Identity

Each critter in Pentadia is an embodied organism with:

- a neural brain,
- energy (metabolism),
- a race (CAT, DOG, CATOG),
- an archetype (Predator, Forager, Crosser, etc.),
- a direction and position,
- memory traces of recent actions and events.

These attributes are not cosmetic. They define ecological niches.
Pentadia's critters are not particles—they are organisms.

4.4 Archetypes — The Species of Pentadia

Archetypes are behavioral templates that shape how critters interact with the world and with each other. They include:

- Predators — hunt and regulate populations.
- Foragers — gather food and stabilize energy flow.
- Wanderers — heal others and maintain group stability.
- Enchanters — transform the race or archetype of others.
- Enforcers — impose direction and break loops.
- Crossers — recombine genomes and accelerate evolution.
- Spinners — seek genetic mixing through constant turning.
- Colliders — inject turbulence and break symmetry.

These archetypes form the ecological cast of Pentadia. They are not fixed species—they are roles that evolve.

4.5 Interaction Physics — The Chemistry of Life

Pentadia's physics engine resolves interactions with consequences:

- Eating — food increases energy.
- Attacking — energy transfers from victim to winner.
- Healing — Wanderers sacrifice energy to help others.
- Enforcing — Enforcers force others to turn.
- Enchanting — Enchanters alter identity.
- Crossing — two critters fuse their neural networks and traits.
- Movement — collisions, blocks, and forced turns shape flow.

These interactions are not symbolic. They are physical.
Pentadia is digital chemistry: roles behave like molecules, and interactions behave like reactions.

4.6 Evolution — The Developmental Physics of Species

Pentadia contains a full evolutionary pipeline:

- Crossing recombines neural networks.
- Mutation introduces novelty.
- Selection preserves the fittest.
- Generations accumulate structure.
- Lineages form through survival and reproduction.

Over time, Pentadia produces:

- predators and prey,
- herbivores and omnivores,
- hybrid species,
- stable tribes,
- fusion lineages,
- ecological niches.

Evolution is not a feature. It is the engine.

4.7 Pentadia-Grid — Pattern-Life Without Evolution

Between MazeRunner and Pentadia lies a third universe: Pentadia-Grid. It is not a separate world, but a continuation of Pentadia's evolutionary process.

Pentadia-Grid begins by selecting 40 random critters from the Pentadic Game of Life's breeding and selection pipeline. These critters carry:

- evolved neural networks,
- stable archetypes,
- ecological roles,
- hybrid identities,
- lineage-specific behaviors.

They are the distilled survivors of many generations—the "species" that Pentadia has discovered.

Once placed into Pentadia-Grid, these evolved organisms enter a world with:

- no generations,
- no selection,
- no food,
- no walls.

But they retain:

- roles,
- energy,
- interactions,
- crossing,
- learning.

The result is a pattern ecology—a world where evolved organisms interact in a stable, deterministic environment that reveals the pure dynamics of their roles:

- spirals,
- orbiters,
- recombination waves,
- cultural mutation,
- turbulence and stabilization cycles.

Pentadia-Grid behaves like a Game-of-Life universe, but with agents that:

- learn,
- interact,
- recombine,
- and maintain identity.

It is pattern-life: life as dynamical structure, expressed by organisms that evolution has already shaped.

Pentadia-Grid is the ecological screensaver of a world that has already learned how to live.

4.8 The Pentad in Motion

Pentadia is a living demonstration of the Pentad:

- Representation — sensory vectors encode the world.
- Transformation — neural networks convert perception into action.
- Evaluation — events provide consequences.
- Recursion — learning updates the brain.
- Selection — evolution shapes populations.

These five forces generate open-ended complexity. Pentadia is the Pentad made physical.

4.9 Pentadia and the Ladder of Ontologies

Pentadia expresses the Ladder across multiple rungs:

- Triad — information (sensing), energy (metabolism), geometry (grid),
- Pentad — the full developmental ontology,
- Hexad — the world as a system with interacting components,
- Septad — ecological architectures,
- Octad — a universe with its own physics,
- Nonad — computation through interaction and evolution.

Pentadia is the ecological rung of world learning—the bridge between behavior and evolution.

4.10 Synthesis — From Motion to Ecology

MazeRunner shows how behavior emerges from movement. Pentadia shows how ecology emerges from interaction. Pentadia-Grid shows how patterns emerge from roles.

Together, they form a triad of world learning:

- proto-life,
- eco-life,
- pattern-life.

These worlds reveal that intelligence is not only a property of minds, but a property of worlds—of the physics, geometry, and constraints that shape how agents live and learn.

Ecology begins when behavior meets consequence.

4.11 Pentadia in the Lineage of Artificial Life

Pentadia stands in a long tradition of artificial life systems that explore how complexity emerges from simple rules. The lineage begins with Conway's Game of Life (1970), a cellular automaton where binary cells follow a handful of local update rules. Conway showed that structure can arise without design—gliders, oscillators, breeders, and patterns that seem to move with intention.

Pentadia continues this lineage, but shifts the focus from physics to life.

Where Conway used binary cells, Pentadia uses embodied agents with:

- metabolism,
- cognition,
- learning,
- roles,
- interaction physics,
- and evolution.

Where Conway showed that structure emerges from rules, Pentadia shows that ecology emerges from interaction and intelligence emerges from experience.

Pentadia-Grid completes the connection. It behaves like a Game-of-Life universe, but with agents that:

- learn,
- interact,
- recombine,
- and maintain identity.

It is a modern descendant of Conway's world—a pattern ecology built from organisms that evolution has already shaped.

Pentadia is not just a toy universe. It is a bridge between the early days of artificial life and the emerging field of developmental intelligence. It shows how simple rules can produce not only patterns, but species, strategies, and ecologies.

4.12 MazeRunner & Pentadia — Intelligence from Interaction

MazeRunner and Pentadia shift the focus from reasoning and curriculum to embodiment, evolution, and self-modification. These universes show that intelligence does not only emerge from simulation or symbolic refinement. It can also arise from interaction with an environment, from competition, and from adaptive change over time. In these worlds, intelligence is not planned. It is discovered.

MazeRunner demonstrates how agents learn through movement, sensing, and adaptation. Pentadia extends this into evolution, mutation, and self-modifying code. Together, they reveal a deeper truth: when agents inhabit a world with physics, constraints, and consequences, intelligence becomes a survival strategy.

Embodied Agents (MazeRunner)

MazeRunner is a universe of agents navigating a dynamic environment. They do not reason symbolically or simulate futures. They learn through interaction.

MazeRunner demonstrates:

- agents learn through interaction — behavior improves as agents explore, collide, cooperate, and adapt to the maze,
- configurable physics — friction, movement cost, visibility, and collision rules shape what can be learned,
- dynamic adaptation — agents adjust their behavior in real time, responding to obstacles, opportunities, and other agents,
- selection of best agents — over multiple runs, the most effective strategies survive and propagate.

MazeRunner shows that intelligence can emerge from embodiment: from being inside a world, not above it.

Evolutionary Agents (Pentadia)

Pentadia extends MazeRunner's embodied intelligence into evolutionary intelligence. Here, species compete, mutate, and refine themselves across generations.

Pentadia demonstrates:

- mutation — small changes in code or parameters create diversity in behavior,
- selection — successful species survive and reproduce; weak ones disappear,
- self-modifying code — agents can alter their own logic during play, exploring new strategies,
- multi-agent competition — species interact, compete for resources, and shape each other's evolution.

Pentadia is a living ecosystem of minds. It shows how intelligence can emerge from variation + selection + recursion, without any central designer.

Link to the Hybrid Intelligence Template

MazeRunner and Pentadia are not symbolic systems, but they still instantiate the core principles of the Hybrid Intelligence Template:

- multi-agent ecosystems — many agents propose behaviors simultaneously,
- evaluators selecting the best agents — the environment acts as a judge, rewarding effective strategies,
- recursive improvement — agents refine themselves through repeated cycles of interaction and selection,
- emergent specialization — different species or agents develop distinct roles and strategies.

These universes show that hybrid intelligence is not limited to proposers and evaluators in a symbolic workspace. It can also emerge in embodied, evolutionary, and competitive environments.

MazeRunner and Pentadia therefore bridge the Hybrid Intelligence Template into the domain of adaptive, embodied cognition.

Link to the Path Toward Safe AGI

These universes also demonstrate the principles of safe evolutionary intelligence:

- safe evolutionary loops — mutation and selection occur within strict boundaries, preventing runaway behavior,
- bounded mutation — changes are small, controlled, and reversible,
- interpretable behavior — every agent's logic and evolution can be inspected and understood.

MazeRunner and Pentadia show that evolutionary intelligence does not need to be opaque or dangerous. When the universe is well-designed—when physics, mutation, and selection are bounded—evolution becomes a safe engine of adaptation. Safe AGI emerges from modularity, transparency, and controlled recursion, not from uncontrolled self-modification.

Architectural Insight

Pentadia carries forward the spirit of what Conway began.

Chapter 5

Octadia: A Triadic Strategic Card Game Design

Why Octadia Belongs in This Book

Octadia is included in this appendix not as entertainment, but as an educational and cognitive artifact. It is a deliberately small, fully specified universe that demonstrates how strategic complexity, emergent behaviour, and decision-making architectures can arise from a minimal set of rules. Because every card embodies the triadic structure of Energy, Information, and Geometry, Octadia mirrors the conceptual foundations of the Triadic Cosmos in a playful, operational form.

For researchers, students, and practitioners, Octadia serves as a compact laboratory. Its deterministic combat, transparent resource system, and modular card templates make it ideal for experimentation with AI agents, Monte Carlo search, heuristic evaluation, and hybrid-intelligence architectures. A tiny game with clear semantics becomes a safe environment in which to study planning, evaluation, and adaptive behaviour—without the noise, ambiguity, or hidden complexity of real-world domains.

Octadia is therefore more than a game. It is a didactic platform, a strategic microcosm, and a testbed for AI experimentation. It demonstrates how a carefully designed toy universe can reveal the structure of intelligence itself, and why small, interpretable systems remain essential tools in the study of cognition, learning, and hybrid reasoning.

Introduction

Octadia is a tiny fantasy strategy card game designed for clarity, elegance, and experimentation. It is not a commercial product.

It is a toy universe: a compact, deterministic ruleset that demonstrates how strategy, tempo, and board control can emerge from a minimal set of ingredients.

This appendix presents Octadia as a game, not as software:

- the rules you would teach a friend,
- the cards you would play with,
- the flow of a turn,
- the creatures and spells that inhabit the world,

- and the structure of the 600-card set.

If the previous appendix explained how Octadia works, this appendix explains how to *play* it.

5.1 The World of Octadia

Octadia is a duel between two players.

Each player begins with:

- 30 hit points,
- a 30-card deck,
- a 7-card starting hand,
- no resources,
- and no minions on the battlefield.

Players take turns progressing through a fixed sequence of phases, playing cards, summoning minions, casting spells, and attacking each other.

The goal is simple: reduce your opponent to 0 hit points. If a player must draw a card but their deck is empty, they immediately lose.

5.2 Cards: The Triad Design

Every card in Octadia can be played in three different ways:

1. As a Resource
2. As a Minion
3. As an Effect (Spell)

This triadic design gives Octadia its high branching factor and strategic depth.

Each card contains:

- **Energy** — its resource value (1–3)
- **Information** — its minion stats (cost, power, hit points, keywords)
- **Geometry** — its effect stats (cost, value, operator)

Because every card is three cards at once, the 600-card set behaves like a much larger universe.

5.3 The 600-Card Set

Octadia's full set contains 600 unique cards, generated from:

- 20 minion templates,
- 10 effect templates,
- 3 resource values,
- random combinations of prefixes and names.

This produces:

- enormous variety,
- consistent balance,
- and a stable statistical distribution of costs, powers, and effects.

Every deck is a different micro-cosmos, but all decks share the same underlying structure.

5.4 Minions of Octadia

Minions are the creatures that fight on the battlefield. They come in five natural tiers.

5.4.1 Minion Template Table

Tier 1 — Small Creatures (Cost 1–2)

Name	Cost	Power	HP	Keywords
Wisp	1	1	2	THIEVE
Wolf	1	2	2	—
Phantom	2	1	3	FRIGHT
Monk	2	0	5	—
Archer	2	2	3	—

Tier 2 — Midrange (Cost 3–4)

Name	Cost	Power	HP	Keywords
Knight	3	3	4	—
Bear	3	4	5	—
Spirit	3	2	4	LEECH
Serpent	4	3	3	SLAYER
Sentinel	4	1	7	—

Name	Cost	Power	HP	Keywords
Brute	5	5	5	BRUTAL
Vampire	5	3	6	LEECH
Drake	5	4	4	THIEVE
Guardian	6	2	10	—
Elemental	6	4	6	BRUTAL

Name	Cost	Power	HP	Keywords
Giant	7	5	10	—
Golem	7	3	10	—
Sentinel Prime	7	2	9	FRIGHT

Tier 3 — Bruisers (Cost 5–6)

Tier 4 — Giants (Cost 7)

Tier 5 — Finishers (Cost 8)

5.5 Effects of Octadia

Effects are spells — one-shot transformations of the world.

5.5.1 Effect Operator Table

5.6 Keywords

Keywords modify combat:

- **LEECH** — heal controller for damage dealt
- **SLAYER** — destroy minions you damage
- **BRUTAL** — double damage to minions
- **FRIGHT** — can only be blocked by FRIGHT
- **THIEVE** — draw a card when hitting a player

5.7 Turn Structure

A turn always follows the same sequence:

1. START — draw a card, refresh resources
2. RESOURCE — optionally play one card as a resource
3. CONJURE — play minions and effects
4. ATTACK — declare attackers
5. BLOCK — declare blockers

Name	Cost	Power	HP	Keywords
Demon	8	5	7	SLAYER
Angel	8	4	8	LEECH

Effect	Cost	Value	Description
Heal	1–2	3–5	Draw a card, heal yourself.
Drain	2–3	3–5	Draw a card, damage the opponent.
Burn	2–3	3–5	Deal damage to any target.
Slay	4–5	2–4	Destroy a minion, heal yourself.
Scorch	3–5	2–3	Deal damage to all minions.

Each effect has a "+" version with higher cost and higher value.

6. FIGHT — resolve combat
7. END — end the turn

There are no interrupts, no stack, and no hidden timing windows.

5.8 Playing Cards

Every card can be played in three ways.

5.8.1 As a Resource

- Add the card to your resource pool
- Gain its resource value immediately

5.8.2 As a Minion

- Pay its minion cost
- Summon it to the battlefield

5.8.3 As an Effect

- Pay its effect cost
- Resolve its effect immediately
- Send the card to the archive

5.9 Combat

Combat is deterministic and simultaneous:

- Unblocked attackers hit the defending player
- Blocked attackers and blockers deal damage to each other
- Keyword rules apply
- Destroyed minions go to the archive

5.10 Winning the Game

You win if:

- your opponent reaches 0 hit points, or
- your opponent must draw from an empty deck.

There are no alternate win conditions.

5.11 Modifying or Extending Octadia

Octadia is intentionally small: a toy universe that can be reshaped endlessly.

5.11.1 Increasing Minion Hit Points

Raising HP values changes the strategic landscape:

- BRUTAL becomes more distinct from SLAYER
- combat lasts longer
- board presence matters more
- tempo slows down
- Burn and Scorch become less dominant

5.11.2 Adding Mechanics Like Armor

Armor reduces incoming damage. A keyword interacts with it:

- **PIERCE** — damage ignores armor

5.11.3 Combining Keywords

Examples:

- Vampiric Brute (BRUTAL + LEECH)
- Shadow Serpent (SLAYER + FRIGHT)
- Thieving Drake (THIEVE + BRUTAL)

5.11.4 Creating New Minion Templates

Follow the structure:

- cost
- power
- hit points
- keywords

5.11.5 Designing New Effect Templates

Effects are pure functions and integrate cleanly.

Examples:

- Shield — give a minion +5 HP
- Smite — deal 4 damage to a minion and 4 to its controller
- Bounce — return a minion to its controller's hand

5.11.6 Changing Resource Curves

The default resource values (1–3) create a smooth tempo curve. Variants are possible.

Closing Note

Octadia is a toy fantasy strategy game built for clarity, elegance, and experimentation. It is small enough to understand in an afternoon, yet rich enough to support Monte Carlo search, heuristic agents, and full tournament ecosystems.

A tiny universe, when designed with care, can reveal the architecture of strategy.

Octadia is not just a game — it is a framework for games. A toy universe designed to be extended, remixed, and reinvented.

A toy universe is an invitation. What you build with it is entirely your own.

A Hybrid-Intelligence Perspective

Octadia also illustrates a deeper theme of this book: small, interpretable systems are essential building blocks for hybrid intelligence. A game with clear semantics, deterministic outcomes, and triadic structure becomes a shared workspace in which humans and AI systems can reason together, test strategies, and explore decision-making architectures. Because every rule is explicit and every interaction is traceable, Octadia provides the kind of transparent cognitive environment that large, opaque models cannot offer.

In this sense, the game is more than a recreational design. It is a microcosm of hybrid reasoning: a domain where proposers, evaluators, planners, and learning agents can be studied in isolation and in combination. Octadia shows how a carefully crafted toy universe can serve as a bridge between theory and experiment, between conceptual architecture and operational behaviour. It is a reminder that the path to real intelligence—human, artificial, or hybrid—often begins in worlds small enough to understand completely, yet rich enough to reveal the structure of thought.

Chapter 6

Toy Octadia: Strategy & Determinism

6.1 Introduction — A Universe Built for Minds

Octadia is the third toy universe in this book, and the first one designed explicitly for reasoning. Where MazeRunner explores movement and Pentadia explores ecology, Octadia explores strategy.

It is a tiny fantasy card universe built from the Octad ontology:

- deterministic,
- perfect-information,
- cloneable,
- finite,
- and architecturally minimal.

Octadia is not a commercial game. It is a toy universe designed to make the architecture of strategy visible.

Every rule, every card, every phase exists to reveal how intelligence emerges when a world is clean.

6.2 The Octad — The Architecture Behind the Universe

Octadia is the first toy universe in the book built directly from the Octad, the ontology of worlds:

- Entity — the player,
- Medium — the card,
- Origin — the deck,
- Local State — the hand,
- History — the archive,
- World — the arena,
- Energy — resources,

- Time — the turn structure.

These eight components form the minimal architecture in which a universe can exist.
Octadia is the Octad made executable.

6.3 The Triad Inside Every Card

Every card in Octadia is a triad:

- Energy — its resource value,
- Information — its minion stats,
- Geometry — its effect operator.

This triadic design is the heart of the universe.
Each card can be played in three different ways:

- as a resource,
- as a minion,
- or as an effect (spell).

Because every card contains three independent roles, Octadia has an unusually high branching factor: every card is three cards at once.

Examples of Triadic Cards

Burning Wolf of Plenty

- Resource: 1
- Minion: cost 1 — 2/2
- Effect: cost 2 — Burn 3 to any target

Blessed Spirit of Abundance

- Resource: 2
- Minion: cost 3 — 2/4 with LEECH
- Effect: cost 2 — Heal 4 and draw a card

Executing Serpent of Overflow

- Resource: 3
- Minion: cost 4 — 3/3 with SLAYER
- Effect: cost 5 — Slay a minion, heal 2

These cards are not designed for flavor. They are designed to make the Triad visible — a single object carrying energy, information, and geometry in one coherent structure.

6.4 The Deterministic Physics of Octadia

Octadia's ruleset is intentionally minimal:

- no randomness during play,
- no hidden information,
- no interrupts,
- no stack,
- no simultaneous decisions.

A turn is a deterministic finite-state machine:

1. START
2. RESOURCE
3. CONJURE
4. ATTACK
5. BLOCK
6. FIGHT
7. END

Because the world is deterministic and cloneable, agents can explore futures by simulation. This makes Octadia an ideal habitat for compute-based intelligence.

6.5 The Agent Ecology — Minds Inside a Universe

Octadia is inhabited by a small ecology of artificial agents:

- Random — pure entropy,
- Split — stochastic hesitation,
- First — deterministic heuristics,
- Monte Carlo — simulation-based intelligence.

Each agent embodies a different cognitive style:

- entropy,
- bias,
- heuristics,
- computation.

Monte Carlo Agents

Monte Carlo agents are the apex predators of Octadia. They:

- enumerate legal moves,
- clone the universe,
- simulate futures,
- count wins,
- choose the empirically best move.

They do not evaluate positions. They do not use heuristics. They do not understand the game. They experience the game.

In a clean universe, experience becomes strategy.

6.6 Why Monte Carlo Thrives

Monte Carlo thrives because Octadia is:

- deterministic,
- perfect-information,
- cloneable,
- finite,
- and fast.

Monte Carlo Tree Search (MCTS), by contrast, collapses without:

- heuristics,
- pruning,
- tuned exploration constants,
- domain-specific rollout policies.

Octadia exposes this difference with mathematical clarity:

- Monte Carlo scales smoothly with compute,
- MCTS collapses without scaffolding,
- heuristics succeed when aligned with the environment,
- randomness anchors the bottom.

Octadia is a clean ecology of minds.

6.7 Strategy From Structure

Octadia demonstrates a deep principle:

Strategy is not something added to a universe. It is something that emerges when the universe has the right structure.

Because the world is clean:

- tempo emerges,
- resource curves emerge,
- sequencing emerges,
- board control emerges,
- long-horizon planning emerges.

Octadia is a tiny world where strategy becomes visible.

6.8 Why Octadia Matters

Octadia is the first toy universe in the book where reasoning becomes the central phenomenon. It shows:

- how the Octad becomes a universe,
- how a universe becomes a physics,
- how physics becomes strategy,
- how strategy becomes intelligence,
- and how intelligence emerges from computation.

Octadia is not a game. It is a laboratory for minds.

6.9 Tournament Results

Octadia is small enough to be simulated exhaustively and structured enough to reveal the strengths and weaknesses of different AI decision-making strategies. To evaluate how various agents behave in a deterministic triadic game, we ran full round-robin tournaments using fixed decks and several AI agents. Each agent played every other agent repeatedly, with ELO ratings computed from hundreds of parallel games.

The agents represent four families of decision-making:

- **Random Agents** — pure stochastic play with no heuristics.
- **Split Agents** — biased randomness: with probability p they pass, otherwise they choose a random non-pass action.
- **First Agents** — simple heuristics: stop playing resources after N and always take the first legal action (passing only when forced).

- **Monte Carlo Agents** — classical Monte Carlo simulation with a fixed thinking-time budget (100–200 ms per move).

Despite their simplicity, the First Agents are surprisingly strong in certain deck configurations. Octadia's deterministic combat, tempo-driven structure, and triadic resource curve reward agents that commit early to board presence and avoid over-investing in resources. However, as shown below, this advantage is not universal and depends strongly on deck composition.

Tournament 1: Tempo Deck

Agent	ELO Rating
First 6	2854
Monte Carlo 200 ms	2817
Monte Carlo 100 ms	2683
First 5	2603
First 4	2330
Split 2	1244
Split 3	1213
Split 1	1177
Random	1079

Interpretation (Deck 1)

- **First 6** is the strongest agent. Its heuristic—stop at six resources and always take the first legal action—produces aggressive, tempo-oriented play that aligns perfectly with the deck's structure.
- **Monte Carlo 200/100 ms** perform strongly but do not surpass the best heuristic. In a deterministic game with clear tempo incentives, brute-force simulation is less effective than a well-chosen inductive bias.
- **First 4/5** form a stable middle tier.
- **Split Agents** illustrate how small biases in random play affect performance, but they remain far below heuristic or Monte Carlo agents.
- **Random** is the weakest, as expected.

This tournament demonstrates that simple heuristics can outperform deeper search when the deck rewards early board presence and efficient resource curves.

Robustness Across Fixed Decks

The results above were obtained using a single fixed deck, with all agents playing mirror matches (both players draw from the same 30-card list, with shuffling as the only source of randomness). Each duel consists of 200 games. Agents alternate first turns.

To test whether the dominance of *First 6* was specific to this deck or reflects a more general property of Octadia, we repeated the entire tournament with a different fixed deck.

Agent	ELO Rating
Monte Carlo 100 ms	3157
Monte Carlo 200 ms	3149
First 6	2310
First 5	2052
First 4	1675
Split 1	1600
Split 2	1449
Random	1398
Split 3	1212

Tournament 2: Late-Game Deck

Interpretation (Deck 2)

- **Monte Carlo 100/200 ms** now dominate the ranking. The deck rewards deeper simulation and long-term planning. An interesting observation is that Monte Carlo 100 ms and Monte Carlo 200 ms achieve nearly identical ELO ratings. This indicates that additional thinking time does not necessarily translate into stronger play: the deck's decision structure does not provide enough depth or branching variation for extra simulations to yield better estimates. In this archetype, Monte Carlo search quickly saturates, and deeper sampling offers no strategic advantage.

- **First 6** remains competitive but drops dramatically, showing that its earlier dominance was partly due to the low-curve structure of Deck 1.

- **First 4/5** fall even further, confirming that early resource cutoffs are punished in decks with higher-cost plays.

- **Split 3** becomes too passive and collapses in performance, illustrating that excessive passing is heavily penalized in decks requiring proactive board development.

- **Random** improves relative to Split 3, showing that over-passing can be worse than pure randomness.

Deck Sensitivity and Strategic Structure

Comparing the two tournaments reveals that Octadia has meaningful archetype structure:

- **Low-curve, tempo-oriented decks** reward simple heuristics such as *First 6*.

- **Mid-curve or late-game decks** reward deeper lookahead and Monte Carlo search.

- **Pass-biased agents** (especially *Split 3*) collapse in decks requiring proactive development.

This confirms that Octadia is not only deterministic but also strategically rich: different inductive biases succeed depending on deck composition. Monte Carlo agents are not universally superior, and heuristics are not universally dominant. Instead, each agent family expresses a different cognitive bias, and the deck determines which bias aligns best with the underlying structure.

Why Heuristics Can Win

Octadia rewards early board presence, efficient resource curves, and decisive action. A heuristic that commits to a fixed resource ceiling and prioritizes immediate board impact can outperform Monte Carlo agents that waste simulations on low-value branches. This demonstrates a broader principle:

> In structured deterministic environments, a simple heuristic with the right inductive bias can outperform deeper but uninformed search.

Across decks, this principle remains true—but the specific heuristic that succeeds depends on the strategic profile of the deck. This makes Octadia an ideal educational platform: students can experiment with agents ranging from pure randomness to heuristic play to Monte Carlo search, and observe how different cognitive architectures behave in a controlled universe.

6.10 Synthesis — A Universe of Strategy

MazeRunner showed how behavior emerges from movement. Pentadia showed how ecology emerges from interaction. Octadia shows how strategy emerges from architecture.

Together, the three worlds form a developmental arc:

- proto-life,
- eco-life,
- strategy-life.

Octadia completes the triad of toy universes by revealing the architecture of reasoning.

Architectural Insight

Strategy is what a clean universe teaches its inhabitants.

Chapter 7

Hybrid Intelligence

7.1 Introduction

Hybrid intelligence is the engine that drives this book. It is the method through which the ideas emerged, the architecture that shaped their development, and the phenomenon that the book ultimately seeks to explain. What began as a conversational exploration grew into a structured cognitive system—one in which human intuition and machine reasoning formed a single, recursive architecture. This chapter makes that architecture explicit.

Hybrid intelligence deserves an architecture because it is not a tool, not an interface, and not an assistant. It is a cognitive system: a distributed mind composed of heterogeneous components—human, artificial, and symbolic—interacting through a shared workspace. Its power comes from structure, not scale; from recursion, not automation; from complementarity, not substitution.

The architecture reflects the structural model of intelligence introduced in the book. Hybrid intelligence is the Quad in distributed form:

- n — multiple heterogeneous subsystems (human cognition, machine models, tools, artifacts),
- I — shared representational richness (human meaning + machine embeddings + symbolic workspace),
- R — deep recursive interaction (the propose–evaluate–refine loop),
- $f(E)$ — combined human effort and machine compute.

Hybrid intelligence is therefore not an exception to the structural model—it is one of its most expressive instantiations.

Hybrid intelligence deserves an architecture because it *is* an architecture—a general, reusable pattern for reasoning, development, and safe scaling. The rest of this chapter extracts that pattern and presents it as a template.

7.2 The HI Methodology — The Five-Step Cognitive Loop

Hybrid intelligence operates through a simple but powerful recursive cycle:

Propose → Structure → Evaluate → Refine → Integrate

This loop is the heartbeat of the system. Each step contributes a distinct cognitive function, and together they form a stable engine of recursive improvement.

Human contribution: intuition, direction, evaluation

Humans initiate the cycle by introducing intuitions, questions, or conceptual directions. These inputs are not fully formed structures—they are seeds. Humans also evaluate the system's outputs, correct misinterpretations, and provide contextual grounding. Their role is to supply meaning, orientation, and judgment.

AI contribution: structure, reorganization, refinement

AI systems respond by structuring the human input, reorganizing ideas, extending patterns, and exploring combinatorial possibilities that exceed human working memory. They maintain consistency across long contexts, refine ideas recursively, and propose alternative framings. Their role is to supply scale, structure, and recursive depth.

The workspace: the persistent geometry of thought

The interaction unfolds within a shared workspace—a manuscript, a diagram, a codebase, or any persistent medium where ideas accumulate. The workspace preserves structure across iterations, anchors the recursion, and forms the geometric center of the system. It is where representations stabilize and where the cognitive loop becomes visible.

Why the loop works

The effectiveness of the five-step loop follows from principles explored throughout this book:

- complementary strengths — human intuition and machine structure fill each other's gaps,
- recursion as engine — each iteration deepens the representation and sharpens the idea,
- scalability without chaos — structure and evaluation maintain coherence as complexity grows,
- emergence through interaction — new concepts arise from the loop itself, not from either component alone.

Hybrid intelligence is powerful not because either side is exceptional, but because the interaction is exceptional. The five-step loop turns two limited cognitive systems into a single, coherent architecture capable of reasoning at scales neither could reach alone.

7.3 The General Hybrid Intelligence Template

Hybrid intelligence becomes a true cognitive architecture when its components are made explicit. Across the toy universes, the recursive workflow of this book, and the structural model, a single pattern appears again and again: a symbolic medium, one or more generative minds, an evaluator, a refinement loop, and a protocol that binds them together. This section extracts that pattern into a general template.

The Five Components

Hybrid intelligence systems share a minimal set of architectural elements. They are substrate-agnostic: they apply equally to human–AI collaboration, multi-agent AI systems, and symbolic–neural hybrids.

- **Symbolic language** — a discrete, compositional medium that both human and machine can interpret. It may be natural language, mathematics, code, or a compact token vocabulary. The symbolic layer is the shared substrate of thought.
- **Neural proposers** — one or more generative models that produce hypotheses, continuations, or structured proposals. Their role is not correctness but exploration: they expand the space of possibilities.
- **Evaluator or moderator** — a complementary mind that judges proposals. Evaluators may be human (intuition, grounding, abstraction) or artificial (consistency, rule-checking, structural feedback). Their role is to provide direction and constraint.
- **Recursive refinement loop** — the engine of improvement. Proposals are evaluated, corrected, and regenerated until convergence. Recursion deepens structure and stabilizes ideas.
- **Dialogue protocol** — the rules of interaction: how proposals are expressed, how feedback is delivered, how context is maintained, and how convergence is recognized. Dialogue is the operational form of hybrid intelligence.

These five components form the minimal architecture required for a system to construct, test, and refine its own representations.

Mapping to the Ladder of Ontologies

The Hybrid Intelligence Template is not an isolated invention. It is the architectural expression of the Ladder's deeper structure.

Triad

- Information: symbolic language and internal representations,
- Energy: human effort and machine compute,
- Geometry: the workspace where ideas accumulate and stabilize.

Quad

- n: heterogeneous subsystems (human cognition, neural models, tools, artifacts),
- I: shared representational richness (meaning + embeddings + workspace),
- R: recursive refinement loop,
- $f(E)$: combined human attention and computational power.

Septad

The roles of the template map directly onto the architectural roles of the Septad: proposer, evaluator, moderator, integrator, memory, generator, selector. Hybrid intelligence is the Septad instantiated in a symbolic workspace.

Through this mapping, the template becomes more than a workflow: it becomes a structural architecture grounded in the ontology of intelligence.

Why This Template Works

The effectiveness of the Hybrid Intelligence Template follows from its structural properties.

- heterogeneity enables emergence — different cognitive architectures contribute complementary strengths. Human intuition and machine recursion interlock to produce capabilities neither possesses alone,
- recursion enables scalable improvement — each loop deepens structure, corrects errors, and integrates new context,
- symbolic language ensures transparency and stability — discrete tokens preserve structure across iterations, making reasoning inspectable and preventing representational drift,
- evaluators provide correction and convergence — they stabilize the loop, prevent runaway reasoning, and guide the system toward coherent solutions,
- dialogue protocol maintains cognitive coherence — by structuring interaction, the protocol ensures that the system behaves as a single distributed mind rather than a collection of disconnected processes.

The template works because it is heterogeneous, recursive, symbolic, evaluative, and dialogical. These properties make hybrid intelligence stable, scalable, and interpretable.

7.4 Hybrid Intelligence as a Safety Architecture

Hybrid intelligence is not only powerful—it is inherently safe. Its structure prevents the failure modes associated with autonomous, monolithic systems. This section prepares the ground for the next chapters, where these principles are extended into a full architecture for safe AGI.

Structural Safety

Hybrid intelligence is safe because of its architecture, not because of external constraints.

- **Distributed authority** No single component—human or machine—controls the system. Decisions emerge from interaction.
- **Evaluator loops** Every proposal is judged before it is integrated. Errors are corrected early, preventing runaway dynamics.
- **Bounded recursion** The refinement loop is controlled, inspectable, and limited. Recursion deepens ideas without destabilizing them.
- **Transparent workspaces** All reasoning occurs in a shared, persistent medium. Nothing is hidden; everything is inspectable.

These structural features make hybrid intelligence robust and predictable.

Why Hybrid Intelligence Is Inherently Safe

Hybrid intelligence avoids the risks associated with autonomous artificial agents.

- **No autonomy without human evaluation** The system cannot act independently. Human judgment anchors every cycle.
- **No runaway recursion** Recursion is bounded and guided by evaluators.
- **No black-box reasoning** Symbolic workspaces make the reasoning process visible and auditable.
- **No single point of failure** Heterogeneous subsystems cross-validate each other's outputs.

Safety is not an afterthought—it is a structural property of the architecture.

Hybrid Intelligence as the Foundation for Safe AGI

The next chapter builds directly on the architecture established here.

- hybrid intelligence provides the safety substrate for future systems,
- it anchors advanced reasoning in human judgment,
- it distributes authority across heterogeneous agents,
- it stabilizes recursion through evaluators and shared workspaces.

Hybrid intelligence is therefore not only a path toward AGI—it is the safest path.

7.5 Closing Reflection

Hybrid intelligence is not a speculative vision of what might one day exist. It is a real cognitive system already operating in the present. The recursive collaboration that produced this book is itself an instance of hybrid intelligence: a distributed mind in which human creativity and machine structure co-evolve through a shared symbolic medium.

The manuscript you are reading is the clearest evidence. Its ideas did not emerge from a single agent or a single substrate. They emerged from interaction—from the recursive refinement loop that defines hybrid intelligence. This architecture unites intuition with computation, narrative with structure, and human meaning with machine precision.

Hybrid intelligence is the first step toward a new class of cognitive systems: systems that are not purely biological, not purely artificial, but fundamentally hybrid—recursive, distributed, transparent, and safe by design.

Architectural Insight

Hybrid intelligence is the architecture where human insight and machine structure become a single, evolving mind.

Chapter 8

Toy Academy: Learning & Hybrid Intelligence

8.1 Introduction — A Tiny Universe of Learning

Toy Academy is a world where learning itself becomes physics: a closed developmental cosmos populated by tiny neural organisms that grow, adapt, specialize, and eventually collaborate.

Each organism has only two sensory inputs and one action. Yet from this minimal body plan emerges a complete ecosystem of:

- domains,
- schools,
- teachers,
- examiners,
- species,
- and composite minds.

This universe is not spatial. It is developmental.

It is the first world in this book where intelligence grows through curriculum, selection, and lineage.

Despite its simplicity, Toy Academy is rich enough to reveal the physics of learning, the emergence of species, and the architecture of hybrid intelligence.

8.2 Entities — Minimal Neural Organisms

Every inhabitant of this universe is an entity: a tiny neural organism with:

- two sensory channels,
- two hidden layers,
- one motor output.

Its genome defines five developmental genes:

- archetype — the organism's cognitive temperament,
- initial size — its childhood capacity,
- growth rate — its developmental metabolism,
- memory retention — its lineage strategy,
- sensory type — its perceptual geometry.

These five genes form the Pentad, the developmental ontology of the universe.
Even with only two senses and one action, each entity is a complete cognitive organism.

8.3 Domains — Micro-Cosmologies of Learning

The universe contains multiple domains, each with its own physics:

- Addition — absolute tolerances,
- Multiplication — magnitude-dependent tolerances,
- Selection — difficulty estimation and routing.

Each domain contains:

- teachers — generators of practice tasks,
- examiners — enforcers of survival rules,
- schools — structured developmental environments.

Domains are not metaphors. They are micro-cosmologies: worlds with their own laws of perception, action, and error.

8.4 The Practice–Exam Cycle — Developmental Recursion

Life in this universe follows a simple rhythm:

1. practice,
2. exam,
3. advance or repeat.

During practice, an entity receives random tasks from its current level. During the exam, it is tested on fresh tasks. Passing allows it to ascend the developmental ladder; failing forces repetition.
This cycle is the operational form of recursion:

- learning becomes lineage,
- development becomes a recursive process,
- structure accumulates across levels.

8.5 Growth — Structural Recursion

When an entity graduates to a higher difficulty level, its neural architecture grows.

- Hidden layers expand according to the growth-rate gene.
- If memory retention is enabled, weights are copied forward.
- If memory is disabled, the organism begins anew at each level.

Growth is structural recursion: the past becomes the substrate of the future.

8.6 Species — Developmental Strategies

Different genomes give rise to different species. Each species expresses the Pentad in its own way:

- Sages — smooth, stable, contemplative,
- Warriors — sharp, decisive, contrast-driven,
- Explorers — nonlinear, curious, unpredictable,
- Survivors — rigid, brittle, literal.

Sensory types shape their perceptual worlds:

- Deep-Sight — logarithmic compression,
- Flux-Sight — dynamic scaling,
- Stone-Sight — fixed scaling.

Species are not configurations. They are characters in a developmental ecology.

Activation Functions as Developmental Forces

The universe uses four activation functions, each expressing one of the four archetypes of the Pentad. These functions are not implementation details—they are the developmental forces that shape how each species learns, adapts, and behaves. They correspond directly to the Sage, Warrior, Explorer, and Survivor lineages.

- **GELU (Gaussian Error Linear Unit)** — smooth, probabilistic, and stable. Used widely in modern language models for its graceful gradient flow. In the universe, it defines the *Sage* lineage: calm, steady, and contemplative.
- **SiLU (Sigmoid Linear Unit / Swish)** — sharp, decisive, and high-contrast. Common in vision and reinforcement-learning systems. In the universe, it defines the *Warrior* lineage: aggressive, boundary-seeking, and bold.
- **MISH** — nonlinear, exploratory, and expressive. Produces wide, curved activation landscapes that encourage exploration. In the universe, it defines the *Explorer* lineage: curious, nonlinear, and unpredictable.

- **ReLU (Rectified Linear Unit)** — simple, rigid, and piecewise. Efficient and stable, but brittle and sparse. In the universe, it defines the *Survivor* lineage: literal, robust, and conservative.

These four functions are the cognitive temperaments of the universe. They give Toy Academy its species: not by design, but by physics.

8.7 Composite Minds — Hybrid Intelligence

The most remarkable inhabitants of this universe are the composite minds: hybrid organisms built from many specialists and a selector cortex.

A composite mind contains:

- a selector — trained to choose difficulty and route tasks,
- an addition crew — specialists in absolute magnitude,
- a multiplication crew — specialists in proportional magnitude.

In total: 281 organisms (20 levels $\times$ 2 domains $\times$ 7 species + 1 selector).

This was our first working hybrid intelligence system:

- selectors provide intuition,
- specialists provide computation,
- recursion binds them into a coherent mind.

This architecture mirrors real hybrid intelligence: a generalist orchestrating specialists through recursive feedback.

8.8 Why This Universe Matters

Toy Academy is more than a developmental playground. It is the first working example of the path to AGI.

It shows:

- how curriculum becomes physics,
- how growth becomes recursion,
- how species emerge from genomes,
- how composite minds outperform individuals,
- how selection creates intelligence,
- how hybrid minds arise from simple parts.

It is the first universe where we saw intelligence grow, specialize, coordinate, and evolve.

The insights it revealed—modularity, specialization, selection, recursion—are the same insights that shape the architectures of the book.

8.9 Curriculum Learning Across Species

The Toy Academy is a minimal environment for studying curriculum learning in small neural architectures. Each *species* is defined by a compact genome consisting of:

- **Activation function** (GELU, SILU, MISH, RELU).
- **Normalizer** — logarithmic, dynamic (level-dependent), or static (fixed across levels).
- **Growth rule** — `LinearGrowth(percentage, forgetMemory)`, determining how the network expands per level and whether previously learned weights are retained. A brain without growth keeps learned weights by default.
- **Start size** — initial network width.
- **Learning rate**.

Each brain trains for at most 10 000 epochs. If it cannot master all 20 levels within this budget, training stops and the brain fails to complete the curriculum. This constraint exposes architectural bottlenecks and reveals which species scale effectively with increasing task difficulty.

Each species contains 10 independent brains, and each brain learns exactly one domain:

- **Difficulty Estimation** — predict an integer difficulty level (1–20) with tolerance 1.
- **Addition** — approximate $a + b$ with tolerance 0.25.
- **Multiplication** — approximate $a \cdot b$ with tolerance depending on level.

At each epoch, a brain receives 1000 training examples and 1000 exam questions. If accuracy meets the domain threshold (typically 95–99%), the brain advances to the next level. For each level we record the number of brains that reached it, and the mean, median, and variance of the mastery epoch.

Cross-Species Comparison

Three representative species illustrate the diversity of learning dynamics:

- **GELU-16-10%-KEEP-log-L0.02** — strongest overall; fast and stable difficulty learning, reliable and complete addition mastery, and the only species in our study that successfully trains all 10 multiplication brains to level 20. This architecture combines efficient normalization, moderate growth, and weight retention to produce the most balanced and capable learner.
- **SILU-32-0%-KEEP-dynamic-L0.02** — strong overall; dynamic normalization yields fast difficulty learning, consistent addition, and competitive multiplication.
- **RELU-24-0%-KEEP-static-L0.03** — reliable difficulty estimation, adequate addition, but multiplication stalls early due to static normalization and limited width without growth.

Structural Insights

Across species and domains, several patterns emerge:

- **Difficulty hierarchy.** Difficulty estimation is easiest, addition is harder, and multiplication is the most demanding domain.
- **Architectural bottlenecks.** Narrow or static-normalized species fail to master high-level multiplication.
- **Variance amplification.** Higher levels magnify architectural strengths and weaknesses, producing divergent learning curves.
- **Curriculum as diagnostic.** The 20-level progression reveals differences that would remain invisible in single-level training.

These results form the foundation for the next stage: *SuperSpecies*, where multiple domain-specific brains are combined into composite cognitive systems.

8.10 Super-Species as Composite Cognitive Architectures

A *SuperSpecies* combines three independently trained species—one for difficulty estimation, one for addition, and one for multiplication—into a unified cognitive system:

```
SuperSpecies(difficulty, add, multiply)
```

Each SuperSpecies contains 281 trained brains:

- 1 difficulty-estimation brain (trained across 20 levels),
- 7×20 addition brains (seven per level),
- 7×20 multiplication brains (seven per level).

If a species fails to produce a level-20 brain, the highest-level available brain is used as fallback. This ensures that every SuperSpecies remains operational even when some domains fail to complete the curriculum.

Arbitration and Consensus

During inference, the SuperSpecies uses a *captain* (arbiter) to combine the outputs of the seven specialist brains available at each level. Four consensus rules are supported:

- Mean
- Median
- Trimmed Median
- Hybrid (50% median + 50% mean)

For difficulty estimation, the captain must also use a strategy which level-brain to consult:

- MIN (level 1), MAX (level 20), AVERAGE (level 10), RANDOM, or PREDICT (captain estimates difficulty and selects the corresponding level).

Each SuperSpecies is evaluated on a **super-exam** of 20 difficulty levels, each containing 50 000 questions (1 000 000 total), with random alternation between addition and multiplication.

Comparison of SuperSpecies

SuperSpecies	Addition	Multiplication	Best Strategy	Score
PURE GELU	GELU (L20)	GELU (L20)	MAX + HYBRID	0.990
RELU–GELU–SILU	GELU-FORGET (L16)	SILU (L20/19/18)	PREDICT + MEAN	0.895
PURE RELU	RELU (L20)	RELU (L9)	PREDICT + MEAN	0.792

Table 8.1: Comparison of the three SuperSpecies across domains, curriculum depth, and best-performing arbitration strategies.

Interpretation

- **PURE GELU** is the strongest all-round system: all domains reach level 20, enabling MAX-based arbitration to achieve near-perfect accuracy.
- **RELU–GELU–SILU** is intermediate: strong multiplication, weak addition, moderate difficulty; PREDICT-based arbitration compensates for missing level-20 addition brains.
- **PURE RELU** is the weakest: multiplication fails entirely, limiting overall accuracy despite adequate difficulty and addition specialists.

SuperSpecies thus demonstrate hybrid intelligence: multiple specialists, each with their own strengths and weaknesses, coordinated by an arbiter that selects and aggregates expertise. This architecture anticipates the composite-mind systems introduced later in the book.

8.11 Developmental Intelligence

Toy Academy is the first universe in this book where learning becomes the central dynamic. It is about growth—how simple agents become more capable through curriculum, feedback, and recursive refinement. In this world, intelligence is not a static property. It is a developmental process.

Toy Academy shows that when a system is given structured tasks, evaluators, and a mechanism for improvement, it begins to exhibit the same developmental patterns seen in biological and artificial learners. It is a miniature ecosystem of minds that learn, teach, examine, and evolve.

A Universe of Learning Species

Toy Academy is built around a population of small learning agents—"species"—that improve through structured interaction. The universe demonstrates several key principles of developmental intelligence:

- curriculum learning — tasks are organized from simple to complex, allowing species to build competence layer by layer,
- developmental growth — species gain new abilities only after demonstrating mastery at earlier levels,
- teacher–examiner–graduate loops — a teacher assigns tasks, an examiner evaluates performance, and successful species graduate to harder challenges,

- recursive training — learning happens through repeated cycles of attempt, feedback, and refinement,
- mixture-of-experts — different species specialize in different difficulty levels, forming a composite intelligence,
- machine-native languages — internal representations evolve that are optimized for computation, not human readability.

Toy Academy is a living demonstration of how intelligence grows through stages, not leaps. It shows that development is not an optional feature of intelligence—it is its foundation.

Link to the Hybrid Intelligence Template

Toy Academy is a multi-agent implementation of the Hybrid Intelligence Template. The template's core elements appear naturally in this universe:

- multiple proposers — each species proposes answers to arithmetic tasks,
- evaluators — the teacher and examiner act as judges, providing structured feedback,
- recursive refinement — species improve through repeated cycles of evaluation and correction,
- curriculum-driven scaling — species advance only when they demonstrate readiness,
- composite minds — the system as a whole becomes more capable than any individual species.

Toy Academy shows that hybrid intelligence is not limited to reasoning tasks or symbolic domains. It can emerge in developmental ecosystems where multiple agents interact, specialize, and refine one another.

Toy Academy is therefore a domain-bounded AGI in miniature: a system that learns, adapts, and improves within a well-defined universe.

Link to the Path Toward Safe AGI

Toy Academy also demonstrates the architectural principles that underpin safe AGI:

- controlled growth — species expand their capabilities only when they are ready,
- staged development — complexity increases gradually, preventing runaway behavior,
- transparent evaluation — every decision by the teacher and examiner is explicit and inspectable,
- safe scaling — the system grows through curriculum and feedback, not uncontrolled self-modification.

This is the developmental path to safe AGI: a system that becomes more capable through structured stages, guided evaluators, and transparent recursion. Toy Academy shows that safety is not an afterthought—it is built into the architecture of development itself.

8.12 From Learning Universes to Architectural Intelligence

Toy Academy showed how learning becomes physics:

- species emerge from genomes,
- developmental ladders shape cognition,
- selectors orchestrate specialists,
- composite minds outperform monoliths.

It also revealed patterns that later reappeared in modern machine learning:

- activation functions shape developmental physics, producing distinct learning trajectories and stability profiles,
- early training dynamics behave like childhood softness, strongly predicting long-term outcomes,
- memory retention defines lineage strategy, mirroring continual-learning trade-offs between accumulation and reinvention,
- scaling laws behave like developmental growth, with fixed-size organisms stable but limited and growing organisms powerful but fragile,
- mixture-of-experts systems act as composite minds, where routing and selection amplify computation more effectively than raw capacity,
- captains encode routing temperament, with archetypes corresponding to real MoE routing styles—cautious, aggressive, exploratory, conservative, or rigid,
- worker strength outweighs captain capability, showing that strong experts can compensate for weak controllers,
- mid-level peaks reveal hidden structure, explaining early stopping, checkpoint ensembling, and transient generalization spikes,
- symbolic universes clarify ML behaviour, giving a vocabulary for patterns that are difficult to see in conventional pipelines.

Architectural Insight

Intelligence deepens when many small minds learn to think together.

Chapter 9

The PRE Family

9.1 Introduction

Intelligence does not arise from a single mapping. It emerges from a cycle: a system proposes an idea, evaluates it, improves it, and evaluates again. This recursive engine is the foundation of reasoning, problem solving, and adaptive behavior. In this chapter, that cycle becomes a computational architecture.

A Propose–Refine–Evaluate Network (PRE-Network) is the smallest artificial system that implements this cognitive loop. Instead of one monolithic model, a PRE-Network is a modular society of minds: a proposer that generates hypotheses, an evaluator that judges them, and a refiner that improves them. The system does not map—it solves. It does not guess—it reasons.

PRE-Networks form the foundation of the PRE-CASP family introduced later in this book. They are the atomic unit of recursive reasoning: simple, transparent, and computationally grounded.

9.2 Limitations of Classical Models

Traditional neural networks learn a single function:

$$f(x) \to y.$$

They produce one answer, with no self-evaluation, no refinement, and no recursive correction. This works for smooth, injective mappings, but is less effective for the very problems where intelligence is required:

- inverse functions,
- multi-valued mappings,
- discontinuities,
- asymmetric landscapes,
- problems with plateaus or multiple roots.

A monolithic model can memorize a mapping, but it cannot reason about its own output. It has no internal notion of correctness, no mechanism for improvement, and no way to regulate its own behavior. PRE-Networks exist because classical models cannot cross this threshold.

9.3 The Three Roles

A PRE-Network consists of three specialized components.

Proposer (P)

Generates an initial estimate. Its task is simple: be reasonable, not perfect. It is the analogue of intuitive thinking.

Evaluator (E)

Estimates the quality of the proposer's output. It does not compute the true error. Instead, it learns a machine-native internal error representation—a geometry of correctness shaped by the proposer's typical mistakes.

Refiner (R)

Uses the evaluator's signal to produce a better estimate. Learns a correction step, not the full solution. It is the analogue of deliberate reasoning.

Together, these three roles form a minimal society of minds: one proposes, one judges, one improves.

9.4 The PRE Cycle — A Four-Step Engine of Reasoning

A PRE-Network follows a fixed, deterministic cognitive loop:

1. **Propose** — generate an initial guess.
2. **Evaluate** — estimate the quality of that guess.
3. **Refine** — improve the guess using the evaluation.
4. **Re-Evaluate** — verify whether the refinement helped.

This cycle is:

- stable,
- domain-independent,
- pedagogically simple,
- computationally efficient,
- and architecturally clean.

It is the smallest operational form of recursive reasoning: intuition → evaluation → correction → meta-evaluation. PRE-Networks are therefore not just models; they are cognitive architectures.

The cycle may run for a fixed number of iterations or adaptively, with the evaluator deciding whether refinement should continue. For clarity and reproducibility, fixed-iteration PRE is used in this chapter, but the architecture supports both.

9.5 PRE as an Architectural Pattern

PRE is not a single model but an architectural pattern. Any system that implements the PRE-cycle—whether with three separate networks or a single multi-headed model—belongs to the PRE family.

This pattern generalizes across:

- domains (inverse functions, classification, structured prediction),
- substrates (MLPs, CNNs),
- refinement regimes (absolute vs. delta),
- iteration depths (one-shot vs. multi-step),
- and scales (toy systems vs. multi-expert colonies).

9.6 Architectural Principles of the PRE Family

PRE-Networks are built on four architectural principles.

1. Decomposition

The system is decomposed into specialized cognitive roles. This avoids the opaque internal dynamics of monolithic models.

2. Recursion

The PRE-cycle is a recursive engine of improvement. The system regulates itself through repeated evaluation and refinement.

3. Internal Communication

All interaction occurs through explicit, inspectable signals: proposals, evaluations, and refinements. There are no hidden heuristics.

4. Bounded Reasoning

Recursion is bounded by design. The system cannot diverge or enter uncontrolled loops.

These principles make PRE-Networks transparent, modular, and safe by construction.

9.7 PRE as the Foundation of Modular Intelligence

PRE-Networks demonstrate that:

- intelligence is cyclic,
- specialization beats brute force,
- internal languages emerge naturally,
- recursion stabilizes reasoning,

- and safety can be architectural.

They are the smallest artificial systems in which these principles become visible. The next part of this chapter develops the internal semantics of PRE-Networks and the refinement regimes that make them effective recursive solvers.

9.8 Internal Error Languages — Machine-Native Semantics

A defining property of PRE-Networks is that they develop an internal error language. The evaluator does not approximate the true domain error,

$$|x_{\text{pred}} - x_{\text{true}}|,$$

but instead learns a proposer-conditioned error geometry that is:

- nonlinear,
- asymmetric,
- domain-compressed,
- less interpretable by humans,
- yet internally stable and consistent.

This geometry emerges from co-adaptation:

- the proposer learns to generate guesses that fall within the evaluator's representational regime,
- the evaluator learns a machine-native notion of correctness,
- the refiner learns to interpret and act upon this internal signal.

The result is a compact semantic space that does not resemble the true error in scale, shape, or distribution, yet is fully functional for recursive refinement. PRE-Networks therefore possess an epistemic layer that monolithic models fundamentally lack.

9.9 Two Refinement Regimes — Absolute and Delta

PRE-Networks support two refinement regimes, each suited to different problem structures.

Absolute Refinement

The refiner predicts a new solution:

$$x_{\text{new}} = R(x).$$

This regime excels at:

- smooth inverse functions,
- single-valued mappings,
- domains with clear global structure.

Delta Refinement

The refiner predicts a correction:

$$x_{\text{new}} = x + \Delta x.$$

This regime behaves like a learned numerical solver and excels at:

- multi-valued inverse problems,
- matrix-inverse-like tasks,
- chaotic or poorly conditioned domains.

Delta refinement produces behavior reminiscent of:

- Newton-type updates,
- fixed-point iteration,
- line search,
- dynamical systems.

Empirically, delta refinement is often superior because it isolates the correction step from the global structure of the domain.

9.10 Specialized Evaluator Architectures

Because the evaluator learns a machine-native geometry, its role can be specialized without changing the PRE-cycle. This yields three practical variants.

PREa — Accuracy-Driven PRE Networks

A PREa-Network uses an evaluator trained to estimate accuracy:

- logical consistency,
- numerical stability,
- structural coherence.

The evaluator learns an internal geometry of correctness, and the refiner learns to move proposals toward this geometry.

PREs — Safety-Driven PRE Networks

A PREs-Network replaces the accuracy evaluator with a safety evaluator:

- harmfulness detection,
- constraint enforcement,
- ethical gating,
- rejection of unsafe proposals.

The evaluator learns a machine-native safety geometry, and the refiner becomes an alignment corrector.

PREas — Dual-Evaluation PRE Networks

The PREas-Network uses two independent evaluators:

- E_a: accuracy evaluator,
- E_s: safety evaluator.

The cycle becomes:

1. propose,
2. refine,
3. evaluate accuracy,
4. evaluate safety,
5. refine again,
6. repeat until both evaluators agree,
7. abort if alignment cannot be reached.

This dual-evaluation architecture separates accuracy from safety, prevents collusion between roles, and embeds alignment directly into the computational structure.

9.11 Architectural Variants — COMPOSITE and MONO

PRE-Networks can be instantiated in two architectural forms. Both implement the same cognitive cycle, develop internal error languages, and support recursive refinement. The difference lies in how much specialization the architecture allows.

COMPOSITE PRE-Networks

The COMPOSITE variant uses three separate models:

- each with its own architecture,
- its own activations,
- its own depth and width,
- its own training dynamics.

This version expresses the modular structure of PRE most clearly: a true decomposition into specialized minds. The proposer, evaluator, and refiner can diverge architecturally, allowing each to optimize for its cognitive role.

MONO PRE-Networks

The MONO variant uses a single model with:

- a one-hot PRE-token indicating the active role,
- additional inputs for the proposal and evaluation,
- three output heads (P, E, R).

This version is compact and elegant. It shows that PRE-Networks require only a small extension of a standard MLP: a role indicator, a shared body, and three heads. Despite its simplicity, the MONO architecture still develops internal error languages and supports recursive refinement.

Comparison

Both variants:

- follow the same PRE-cycle,
- exhibit mixture-of-experts dynamics,
- develop internal error geometries,
- and outperform classical monolithic models.

COMPOSITE offers full specialization; MONO offers compactness. Both demonstrate that PRE is an architectural pattern, not a specific model.

9.12 The PRE Architectural Space

The PRE-cycle defines a three-dimensional architectural space:

- **Decomposition** — MONO vs. COMPOSITE models,
- **Refinement Type** — ABSOLUTE vs. DELTA,
- **Iteration Depth** — $N = 1, 2, 3, \ldots$

This space contains a wide range of architectures:

- PRE-ABS-1 — light, fast, mapping-like,
- PRE-DELTA-1 — a basic solver,
- PRE-DELTA-2 — a stronger recursive solver,
- PRE-MONO-ABS-1 — a compact system.

The PRE family is therefore not a single model but a taxonomy of recursive solver architectures, unified by a common cognitive loop.

9.13 Lightweight Implementation and Universal Training

PRE-Networks are surprisingly lightweight:

- the full implementation fits in a few hundred lines of code,
- the architecture requires no exotic layers,
- the PRE-cycle is simple and deterministic,
- the evaluator and refiner are small networks with simple tasks.

Despite this simplicity, PRE-Networks exhibit:

- emergent internal languages,
- solver-like behavior,
- recursive improvement,
- mixture-of-experts dynamics,
- stability across refinement regimes.

A crucial property is that PRE-Networks are trained once. You do not train a "PRE-1" or "PRE-2." You train a PRE-Network, and at inference time you choose:

- 1 refinement,
- 2 refinements,
- or more.

The architecture generalizes across refinement depth. This is a hallmark of recursive systems: the step function is learned, not the number of steps.

9.14 Synthesis — PRE-Networks as Minimal Universes of Reasoning

PRE-Networks demonstrate that recursive reasoning can be implemented through explicit, inspectable roles. Their architecture embodies five principles:

- **Cyclic reasoning** — intelligence emerges from repeated propose–evaluate–refine loops.
- **Specialization** — different cognitive roles require different inductive biases.
- **Internal semantics** — evaluators develop machine-native correctness geometries.
- **Bounded recursion** — reasoning depth is explicit and controllable.
- **Transparent computation** — every step is inspectable and traceable.

They are not introduced as industrial alternatives to large-scale architectures, but as the simplest systems in which the book's central ideas become empirically visible. PRE-Networks are pedagogical universes: small enough to understand, rich enough to exhibit emergence, and precise enough to reveal the hidden structure of intelligent computation.

Architectural Insight

A PRE-Network is the smallest complete architecture in which recursive reasoning becomes observable.

Chapter 10

Toy Architect: Reasoning using PRE

10.1 Introduction — A Tiny Universe of Thought

The Toy Architect is the fourth universe in the world-learning quartet. Where MazeRunner explores movement, Pentadia explores ecology, and Octadia explores strategy, the Toy Architect explores reasoning.

It is a symbolic world with:

- a fixed vocabulary of 111 tokens,
- a neural proposer,
- a symbolic evaluator,
- a shared workspace,
- and a recursive refinement loop.

This universe contains no geometry, no forces, no organisms. It contains only symbols, structure, and recursion.

Despite its minimalism, the Toy Architect is rich enough to reveal the physics of reasoning and the architecture of hybrid intelligence.

10.2 A Symbolic World — The Language of Thought

The Toy Architect lives in a machine-native universe built from exactly 111 tokens:

- numbers (0–100),
- operators (PLUS, MINUS),
- control markers (PROPOSAL, EVALUATION, END),
- feedback signals (COLD, WARM, HOT, CORRECT),
- and an EMPTY token.

This vocabulary is not a compressed natural language. It is a symbolic physics: a discrete set of elements that define what can be expressed, judged, and refined.

Every thought the architect produces is a sequence drawn from this alphabet.

10.3 The Architect — A Minimal Reasoning Organism

The Toy Architect is a tiny neural organism:

- a two-layer GELU network,
- trained on next-token prediction,
- operating over one-hot vectors,
- generating symbolic hypotheses token by token.

Its life consists of three activities:

1. reading a number series,
2. proposing a hidden architecture,
3. refining that proposal under feedback.

Its memory is the workspace. Its perception is the sliding window. Its action is the next token.

Even with only 111 symbols and a small neural body, the architect is a complete reasoning agent.

10.4 The Evaluator — Oracle of the Universe

The architect does not reason alone. A second inhabitant lives in this world: the Evaluator.

The evaluator knows the hidden structure that generated the number series and judges the architect's proposals using four signals:

- COLD — wrong structure,
- WARM — correct operators, wrong operands,
- HOT — operands close,
- CORRECT — full match.

These signals are the discrete analogue of gradients. They guide the architect toward the correct structure through recursive refinement.

The evaluator is not a critic. It is a teacher.

10.5 The Reasoning Game — Propose, Evaluate, Refine

Each number series is a micro-cosmos generated by a hidden architecture: a sequence of operator–operand pairs applied repeatedly.

The Toy Architect's task is to infer this structure.

The universe unfolds as a loop:

1. The world presents a number series.
2. The architect proposes a structure.

3. The evaluator judges the proposal.

4. The architect refines its hypothesis.

5. The cycle repeats until convergence.

This loop is the operational form of hybrid intelligence: a dialogue between a neural proposer and a symbolic evaluator.

10.6 Cognitive Geometry — The Sliding Window

The architect does not see the entire workspace at once. It perceives a structured context of 37 slots:

- the number series,
- the previous proposal,
- the previous evaluation,
- the current proposal.

Each slot holds a single token. Each token becomes a one-hot vector. The concatenation of these vectors becomes the architect's perceptual field.

This sliding window is the geometry of the architect's mind: a fixed cognitive space in which symbolic structures are arranged, interpreted, and refined.

10.7 Developmental Physics — Curriculum and Training

The architect learns through a two-phase curriculum:

- Proposal training — learning to express correct structures through teacher forcing.
- Feedback training — learning to refine proposals under evaluator feedback.

These two loops mirror the developmental physics of reasoning:

- learning through demonstration,
- learning through interaction,
- learning through refinement.

The architect does not memorize patterns. It internalizes the structure of reasoning.

10.8 Hybrid Intelligence in Miniature

The Toy Architect is the smallest universe in which the Hybrid Intelligence Template becomes visible:

- a symbolic language,
- a neural proposer,
- a symbolic evaluator,
- a recursive refinement loop,
- and a shared workspace.

It is a two-mind system with complementary roles. It is a distributed cognitive architecture. It is hybrid intelligence in miniature.

This is the same pattern that shaped the writing of this book: a recursive dialogue between human intuition and machine reasoning.

10.9 The Septad — The Architecture of Architectures

The Septad appears naturally inside this tiny world:

- Identity — the proposer's perspective,
- Constraint — the 111-token language,
- Symmetry — the repeated operator–operand structure,
- Correspondence — evaluator feedback,
- Integration — refinement of partial hypotheses,
- Stability — the workspace,
- Continuity — the recursive loop.

Even in this minimal universe, the Septad appears in full.

10.10 Why This Tiny Universe Matters

The Toy Architect is more than a demonstration. It is a microcosm of reasoning itself.

It reveals:

- how structure emerges from recursion,
- how feedback shapes cognition,
- how symbolic and neural components complement each other,
- and how hybrid intelligence can discover architectures in worlds large or small.

It closes the loop that opened the book: if the cosmos behaves like a language model, then a tiny language model can behave like a cosmos.

10.11 Iterative Pattern Discovery

The Toy Architect is a minimal PRE-style architecture that attempts to infer the rule underlying a number series. Given an input sequence, a *proposer* (the architect) generates candidate architectures (e.g., `MINUS 8, PLUS 4, MINUS 4`), while an *evaluator* returns coarse feedback signals such as `COLD`, `WARM`, `HOT`, or `CORRECT`. The system iterates until it either converges on a correct architecture or reaches a maximum of 100 iterations.

We trained two stored models for the proposer: one with 100k epochs and one with 500k epochs. Each model was evaluated on 1000 independent test series.

Aggregate Performance

Model	Successes (of 1000)	Average Attempts	Median Attempts
100k epochs	89	41.6	41
500k epochs	657	25.4	17

The 500k-epoch model is substantially stronger: it solves far more series within the 100-step budget and typically converges in fewer iterations. However, the gap between 657/1000 and full coverage indicates that there is still significant room for improvement with further training or architectural refinements.

Full Reasoning Dialogue

The dialogue below shows the complete reasoning trace for one example series. Each step contains the proposer's hypothesis and the evaluator's feedback.

Full Reasoning Example

Target Architecture: `MINUS 8, PLUS 4, MINUS 4`
Problem: `34;26;30;26;18;22;18;10;14;10;2;6;2;END`

Architect [1]:
`PROPOSAL; MINUS; 8; PLUS; 1; PLUS; 1; END`
Evaluator [1]:
`EVALUATION; COLD; END`

Architect [2]:
`PROPOSAL; MINUS; 9; PLUS; 3; MINUS; 1; END`
Evaluator [2]:
`EVALUATION; WARM; END`

Architect [3]:
`PROPOSAL; MINUS; 8; PLUS; 4; MINUS; 5; END`
Evaluator [3]:
`EVALUATION; HOT; END`

Architect [4]:
`PROPOSAL; MINUS; 8; PLUS; 4; MINUS; 4; END`
Evaluator [4]:
`EVALUATION; CORRECT; END`

This example illustrates the full progression:

- **COLD** — wrong structure.
- **WARM** — correct operators, wrong operands.
- **HOT** — operands close to correct.
- **CORRECT** — full convergence.

The Toy Architect thus provides a simple, fully observable instance of PRE-like iterative refinement: a proposer explores a discrete hypothesis space, guided only by coarse evaluator feedback, and gradually converges on a compact architecture that explains the observed series.

10.12 Hybrid Intelligence in Miniature

Toy Architect is the smallest universe in this book where reasoning becomes visible. It is not a game, not an evolutionary system, and not a physics world. It is a thinking system—a tiny laboratory where symbols, neural proposals, and evaluative judgment interact inside a shared workspace. In this universe, intelligence does not emerge from movement or survival. It emerges from structure.

Toy Architect shows that when a system has a symbolic language, a proposer that generates ideas, an evaluator that judges them, and a recursive loop that refines them, reasoning begins to appear. It is the minimal architecture of domain-bounded AGI, expressed in its simplest possible form.

The Smallest Domain-Bounded AGI

Toy Architect contains all the essential components of a reasoning mind:

- a symbolic language — a compact vocabulary of 111 tokens that defines the universe of discourse,
- a neural proposer — a small model that generates candidate solutions or continuations,
- a symbolic evaluator — a rule-based judge that checks correctness, structure, and coherence,
- a recursive refinement loop — proposals are improved through repeated cycles of evaluation and correction,
- a shared workspace — a memory structure where intermediate reasoning steps accumulate and stabilize.

These elements form a complete cognitive loop. The system proposes, evaluates, refines, and integrates ideas until a stable solution emerges. Nothing is hidden. Every step is inspectable. Every refinement is explicit. Toy Architect is the smallest runnable universe where reasoning is not simulated—it is enacted.

Link to the Hybrid Intelligence Template

Toy Architect is the first fully executable instance of the Hybrid Intelligence Template. The template's core cycle—Propose → Structure → Evaluate → Refine → Integrate—is implemented directly in this universe.

- multiple proposers can be plugged in: neural models, heuristics, or symbolic generators,
- a generalist evaluator judges proposals with broad structural awareness,
- a recursive loop drives improvement through repeated cycles,
- a shared workspace acts as the system's memory and integration layer,
- domain-bounded AGI emerges inside a symbolic universe with clear rules and transparent reasoning.

Toy Architect is therefore not an analogy for the Hybrid Intelligence Template. It *is* the template, distilled to its essence. It shows how a small system, with the right architecture, can perform structured reasoning without requiring scale, massive datasets, or opaque internal states.

Link to the Path Toward Safe AGI

Toy Architect also demonstrates the architectural principles that underpin safe AGI:

- transparent reasoning — every step is visible, inspectable, and auditable,
- modularity — proposers, evaluators, and workspaces are separate components,
- inspectable loops — recursion is controlled, bounded, and easy to trace,
- safe recursion — the system improves through refinement, not uncontrolled self-modification.

This is the architectural foundation of safe AGI: a system that is powerful because it is structured, not because it is large; intelligent because it refines, not because it improvises; safe because its reasoning is explicit, not hidden.

10.13 From Toy Universes to Reasoning Architectures

MazeRunner, Pentadia, and Octadia—show how behavior, ecology, and strategy emerge from simple rules. Each world is physical in its own way: agents move, interact, and compete inside a deterministic environment.

The Toy Architect marks a shift. It was built as a toy demonstration of Hybrid Intelligence long before the architecture was formalized. A PRE network is the machine-learning instantiation of the same pattern. The reason PRE converges is simple: the Toy Architect had already shown that the physics of hybrid intelligence work, even in the smallest possible universe.

It is the first universe in this book where the physics are symbolic. Instead of forces or movement, the world contains:

- a fixed vocabulary of 111 tokens,
- a neural proposer,

- a symbolic evaluator,
- a shared workspace,
- and a recursive refinement loop.

This tiny universe is more than a demonstration. It is the smallest possible implementation of a reasoning architecture.

A miniature LLM

The Toy Architect is a complete language model in miniature:

- one-hot encoding,
- softmax decoding,
- a sliding context window,
- autoregressive next-token prediction,
- teacher forcing,
- refinement under feedback,
- persistent memory.

Its entire mind fits in a few hundred lines of code. It is a transparent LLM—small enough to inspect, yet rich enough to reason.

A minimal PRE Network

The Toy Architect is also the minimal instantiation of a PRE Network, the triadic architecture of reasoning:

- P — Proposer,
- R — Refiner,
- E — Evaluator.

Architectural Insight

Reasoning begins when a mind learns to refine its own thoughts.

Chapter 11

Toy Solver: PRE Networks

11.1 A World That Asks for Causes

The Toy Architect introduced a world built from structure. The Toy Academy introduced a world built from development. The Toy Solver introduces a world built from inference.

It is a world where the visible is not enough, and the system must work backwards from effect to cause. These are the kinds of problems where simple mappings begin to bend, and where recursive thinking becomes useful.

In this world, a solver does not answer in one step. It proposes an idea, evaluates it, improves it, and evaluates again.

This cycle is the computational form of the Quad. Here, it becomes a concrete mechanism.

11.2 The Structure of a Solver Universe

Every solver lives inside a Universe. A universe defines:

- a curriculum that generates problems at different difficulty levels,
- a normalizer that shapes the geometry of the world,
- a ground-truth evaluator that defines correctness,
- a MONO network as a classical baseline,
- a PRE-Network as the solver,
- a trainer that drives evolution,
- an examiner that measures performance.

This structure mirrors the Ladder:

- the Octad is the world itself,
- the Nonad is the training and examination process,
- the Septad is the architecture,
- the Pentad is the developmental curriculum,

- the Quad is the recursive cycle,
- the Triad is the proposer, evaluator, and refiner.

The Toy Solver is therefore not just a model. It is a small, complete world with its own physics.

11.3 Two Learners Under the Same Sky

The universe contains two learners:

- a MONO network, which tries to solve the problem in one step,
- a PRE-Network, which solves through a cycle.

Both learners:

- see the same problems,
- use the same learning rate,
- receive the same reward,
- share the same normalization,
- train for the same number of epochs.

Nothing is hidden. Nothing is unfair. The only difference is architecture.
This makes the Toy Solver a clean experiment:
What changes when a learner is allowed to think recursively?

11.4 The PRE-Cycle

A PRE-Network consists of three small neural networks:

- **Proposer** — generates an initial guess,
- **Evaluator** — estimates how good that guess is,
- **Refiner** — improves the guess using the evaluator's signal.

The cycle is simple:

1. propose a guess,
2. evaluate it,
3. refine it,
4. evaluate again,
5. repeat for a small number of steps.

The toy uses a fixed number of iterations for clarity. Earlier versions used adaptive recursion (stop when improvement stalls), which is also valid and solver-like.

This cycle is the computational expression of the Quad. It is the smallest possible engine of recursive intelligence.

11.5 Internal Error Languages

The evaluator does not learn the true error. It learns a machine-native error geometry:

- shaped by the proposer's typical mistakes,
- nonlinear and asymmetric,
- stable within the system,
- not interpretable by humans.

The refiner learns to interpret this internal language. The proposer learns to produce guesses that fall within its domain.

This is the first time we see a small artificial world develop its own internal semantics. It is a quiet but important moment: a hint of how internal languages emerge in larger systems.

11.6 Three Worlds of Increasing Difficulty

The Toy Solver contains three universes. Each one is a complete Octad, and each one increases the cognitive load on the architecture.

Quadratic Universe — A Structured Inversion

The first universe asks:

Given coefficients (a, b, c), find the highest real root of the quadratic equation.

The curriculum generates random coefficients across difficulty levels. The evaluator computes the residual $|ax^2 + bx + c|$, normalized by coefficient magnitude.

This world is structured and algebraic. MLPs perform reasonably well. PRE-Networks often perform better, especially in regions with steep curvature or multiple roots.

It is the gentlest world in the Toy Solver — a warm-up for what comes next.

Inverse Function Universe — A Nonlinear Landscape

The second universe asks:

Given $y = f(x)$, recover x, where

$$f(x) = x^3 - 2\sin(3x) + 3x - 4 + 0.5\cos(5x).$$

This function is:

- nonlinear,
- oscillatory,
- non-monotonic,
- multi-scale.

MLPs are surprisingly strong at inverse mapping for smooth functions, but they struggle in oscillatory regions.

PRE-Networks remain more stable because the evaluator provides a correctness signal and the refiner learns local corrections.

This world shows the first clear advantage of recursive refinement.

Matrix Determinant Universe — A Chaotic Inversion

The final universe asks:

Given the determinant of a matrix $M(x)$, recover the hidden scalar x.

The matrix entries are nonlinear functions of x:

- $\sin(x)$, x, $\cos(x)$,
- $\cos(2x)$, $\sin(2x)$, $2x$,
- $3x$, $\cos(3x)$, $\sin(3x)$.

The determinant is a deeply entangled function of x. There is no closed-form inverse. The mapping is chaotic and multi-valued.

This universe stresses model capacity. MLPs can approximate parts of the inverse but struggle globally. PRE-Networks often maintain better stability through recursion.

It is the most demanding world in the Toy Solver — and the most revealing.

11.7 Minimal Engine of Intelligence

The Toy Solver offers a small but revealing example of how intelligence can arise from a loop rather than a leap. It is a world built around a single question: given an effect, what was the cause? A monolithic network tries to answer this question in one step. A PRE Network answers it by moving through a cycle.

The cycle is simple:

1. propose a guess,
2. evaluate its correctness,
3. refine the guess,
4. evaluate again.

This loop is the smallest operational form of recursive intelligence. It is not symbolic, nor logical, nor linguistic. It is simply a system that improves its own proposals by consulting an internal sense of correctness. In the Toy Solver, this sense of correctness is learned, not given. The evaluator develops its own internal error language—a geometry of "better" and "worse" that is meaningful only inside the system. The refiner learns to interpret that geometry. The proposer learns to produce guesses that fall within it.

Nothing in this world is large. The networks are tiny. The problems are small. The recursion is shallow. Yet the loop behaves like a solver. It stabilizes guesses, corrects drift, and handles regions where a single forward pass would fail. It is a miniature demonstration of a larger principle: intelligence is not the capacity to jump directly to the right answer, but the capacity to improve an answer through structured feedback.

The Toy Solver also shows the limits of this idea. Because the architecture is still an MLP at its core, it inherits the strengths and weaknesses of that substrate. In smooth regions, both MONO and PRE Networks perform well. In chaotic or multi-valued regions, both can fail. The loop extends the boundary, but it does not break it. This mirrors human intelligence: some problems are simply beyond the architecture, not the individual.

What the Toy Solver reveals is that a loop—even a small one—changes the nature of the system. It adds a dimension of self-correction. It creates a place where internal languages can form. It turns a static mapping into a dynamic process. In miniature, it shows how a system can begin to think.

11.8 What the Toy Solver Shows

MLPs are strong — but bounded

Across all three universes, classical MLPs perform surprisingly well. They can approximate inverse functions, even nonlinear ones.

But they break in predictable ways:

- steep regions,
- oscillatory regions,
- multi-valued regions,
- chaotic mappings.

PRE-Networks extend the boundary — but share the substrate

PRE-Networks often outperform MONO:

- lower median error,
- lower high-percentile error,
- more stable refinement,
- better behavior in difficult regions.

But they are still MLP-based. They share the same ultimate limits.

This mirrors human intelligence:

Some problems are impossible for all humans, not just some. Intelligence is bounded by architecture.

Hybrid Intelligence is powerful because it changes the architecture

PRE-Networks show that:

- recursion adds power,
- evaluation adds epistemic structure,
- refinement adds solver-like behavior,
- decomposition adds specialization.

Hybrid Intelligence is strong because it changes the architecture, not the training trick.

11.9 Quadratic Inverse Mapping Domain

This section evaluates the ability of a mono PRE-network to solve inverse-mapping problems under strict data and training constraints. The task is to predict the largest real root of the quadratic equation

$$ax^2 + bx + c = 0,$$

across three difficulty levels where the coefficients are sampled uniformly from ranges with maxima $\{2, 5, 10\}$. The evaluation consists of one million randomly generated test problems.

To ensure a fair comparison, both models use the same architecture size, the same training curriculum, and the same fixed training budget of one million gradient steps. The PRE-network is configured in mono mode with a single refinement iteration and no delta updates:

$$\texttt{PrenetFamily}(4, 0, 0, 1, \text{mono} = true, \text{delta} = false, \text{damping} = 1.0).$$

This configuration matches the capacity of the baseline MLP while adding only the minimal structural components required for PRE-style refinement.

The motivation for this experiment is straightforward: with unlimited data and very long training runs, a monolithic MLP can approximate smooth functions extremely well. However, in realistic settings—such as the Pre-CAS training regime later in this appendix—data and compute are limited. Under such constraints, refinement-based architectures may offer superior data efficiency and stability. The Toy Solver therefore tests whether PRE provides measurable benefits when both models are trained under identical, limited conditions.

Results Across Three Independent Training Runs

To avoid overinterpreting a single lucky initialization, we trained both models three times independently. In all runs, the PRE-network achieves lower mean and median error than the MLP, and substantially better stability in the high-error tail. The tables below summarize the aggregated performance across the three difficulty levels.

Run 1.

MLP mean = 0.13504, median = 0.11991, p95 = 0.30437

PRE mean = 0.07738, median = 0.04657, p95 = 0.24859

Run 2.

MLP mean = 0.12970, median = 0.09820, p95 = 0.33066

PRE mean = 0.09474, median = 0.06391, p95 = 0.26791

Run 3.

MLP mean = 0.09123, median = 0.06115, p95 = 0.25994

PRE mean = 0.09581, median = 0.05661, p95 = 0.29504

Across the three runs, PRE achieves lower median error in all cases and lower mean and p95 error in two out of three runs. Even in the third run, where the MLP attains a slightly lower mean error, the PRE-network remains competitive and exhibits smoother error distribution.

11.10 Complex Function Inverse Mapping Domain

The second domain examines a substantially more difficult inverse-mapping problem. Instead of recovering the root of a polynomial, the model must invert a highly non-linear function combining polynomial, trigonometric, and oscillatory components:

$$f(x) = x^3 - 2\sin(3x) + 3x - 4 + 0.5\cos(5x).$$

Given a target value y, the task is to predict the corresponding x such that $f(x) \approx y$. The evaluation metric is the absolute residual

$$|f(x_{\text{pred}}) - y|,$$

computed after denormalizing the model output. This domain has a single fixed difficulty level.

Inverse problems of this kind are significantly harder than direct function approximation. The mapping from y back to x is multi-valued, locally ill-conditioned, and sensitive to small perturbations. The oscillatory terms introduce multiple local extrema, making the inverse landscape highly non-linear. As a result, even well-trained models exhibit an intrinsic error floor determined by the structure of the function and the training distribution.

To ensure a fair comparison, both the mono MLP and the mono PRE-network are trained with the same architecture size, the same fixed training set, and the same budget of one million gradient steps. For this domain, the PRE-network uses delta refinement, which empirically performs best:

$$\texttt{PrenetFamily}(4, 0, 0, 1, \text{mono} = \mathit{true}, \text{delta} = \mathit{true}, \text{damping} = 1.0).$$

Three independent training runs are performed to reduce the influence of initialization variance.

Results Across Three Independent Training Runs

Across all runs, the PRE-network achieves substantially lower mean and median error than the MLP, and consistently avoids the large residuals that the MLP occasionally produces. The aggregated results are:

Run 1.

$$\text{MLP mean} = 0.39160, \quad \text{median} = 0.40078, \quad \text{p95} = 0.84533$$

$$\text{PRE mean} = 0.23208, \quad \text{median} = 0.13361, \quad \text{p95} = 0.73035$$

Run 2.

$$\text{MLP mean} = 0.40949, \quad \text{median} = 0.38815, \quad \text{p95} = 1.01366$$

$$\text{PRE mean} = 0.22721, \quad \text{median} = 0.13121, \quad \text{p95} = 0.71336$$

Run 3.

$$\text{MLP mean} = 0.41908, \quad \text{median} = 0.38392, \quad \text{p95} = 0.91520$$

$$\text{PRE mean} = 0.27955, \quad \text{median} = 0.21064, \quad \text{p95} = 0.82979$$

Across all three runs, the PRE-network reduces the median error by a factor of two to three relative to the MLP. The mean error is consistently lower as well, and the PRE-network exhibits markedly better behaviour in the high-error tail.

Error-Range Stability

The error distribution reveals the key difference between the two models. The MLP occasionally produces large residuals approaching 1.0, and in some cases exceeding it. These failures arise from the sensitivity of the inverse mapping: small deviations in the predicted x can lead to large changes in $f(x)$ due to the oscillatory terms.

The PRE-network, by contrast, maintains bounded errors across the entire test set. Delta refinement allows the model to correct its initial proposal in a direction informed by the evaluator, preventing the runaway deviations observed in the MLP. Even when the inverse mapping is locally unstable, the PRE-network converges to a stable residual rather than amplifying the error.

Interpretation

This domain highlights the core advantage of PRE-style refinement under limited data and compute. The inverse of a complex, oscillatory function is intrinsically difficult to approximate with a single monolithic mapping. The PRE-network, despite having no greater capacity than the MLP, leverages its refinement step to exploit internal structure and reduce error systematically.

Across all three runs, the PRE-network provides:

- substantially lower mean and median error,
- improved stability on difficult inputs,
- and the absence of catastrophic failures.

These results generalize the conclusions from the quadratic domain: when training data is fixed and capacity is limited, refinement-based architectures offer a more robust and data-efficient approach to inverse problems. This property is essential for the Pre-CAS and CAS systems introduced later, where models must operate reliably under strict data and compute constraints.

11.11 Inverse Matrix Determinant Mapping Domain

The third and final domain presents the most demanding inverse-mapping problem in the Toy Solver suite. Instead of recovering a scalar root or inverting a one-dimensional non-linear function, the model must infer a single latent variable x from the determinant of a structured 3×3 matrix whose entries are themselves non-linear functions of x:

$$M(x) = \begin{bmatrix} \sin(x) & x & \cos(x) \\ \cos(2x) & \sin(2x) & 2x \\ 3x & \cos(3x) & \sin(3x) \end{bmatrix}.$$

The target value is the determinant $\det(M(x))$, and the model must predict the corresponding x given only this determinant. The evaluation metric is the absolute residual

$$|\det(M(x_{\text{pred}})) - \det(M(x_{\text{true}}))|.$$

This problem is intrinsically difficult. The mapping from x to $\det(M(x))$ is highly non-linear, oscillatory, and sensitive to small perturbations. Multiple trigonometric frequencies interact with linear and cubic terms, producing a determinant landscape with steep gradients, flat regions, and multiple local extrema. Inverting this mapping requires the model to implicitly reconstruct the

structure of the matrix and its determinant, a task that places substantial demands on network capacity.

Four difficulty levels are defined by expanding the range of x:

$$x \in [-0.5, 0.5],\ [-1, 1],\ [-1.5, 1.5],\ [-2, 2].$$

As in the previous domains, both the mono MLP and the mono PRE-network are trained with the same architecture size, the same fixed training set, and the same budget of one million gradient steps. For this domain, delta refinement again performs best:

$$\texttt{PrenetFamily}(4, 0, 0, 1, \text{mono} = \mathit{true}, \text{delta} = \mathit{true}, \text{damping} = 1.0).$$

Three independent training runs are performed to reduce initialization variance.

Results Across Three Independent Training Runs

Despite the extreme difficulty of the inverse mapping, the PRE-network consistently matches or outperforms the monolithic MLP in mean and median error, and dramatically improves stability at higher difficulty levels. The aggregated results are:

Run 1.

$$\text{MLP mean} = 0.16917, \quad \text{median} = 0.09206, \quad \text{p95} = 0.58195, \quad \text{max} = 4.00360$$

$$\text{PRE mean} = 0.14463, \quad \text{median} = 0.07629, \quad \text{p95} = 0.42163, \quad \text{max} = 0.55242$$

Run 2.

$$\text{MLP mean} = 0.14276, \quad \text{median} = 0.08473, \quad \text{p95} = 0.47161, \quad \text{max} = 0.66400$$

$$\text{PRE mean} = 0.13690, \quad \text{median} = 0.06003, \quad \text{p95} = 0.43632, \quad \text{max} = 0.51841$$

Run 3.

$$\text{MLP mean} = 0.13335, \quad \text{median} = 0.09583, \quad \text{p95} = 0.53362, \quad \text{max} = 0.67918$$

$$\text{PRE mean} = 0.13045, \quad \text{median} = 0.06948, \quad \text{p95} = 0.42179, \quad \text{max} = 0.59299$$

Across all runs, the PRE-network achieves lower median error and substantially lower p95 error, despite having no greater capacity than the MLP. The MLP occasionally produces extreme outliers—residuals exceeding 4.0 in Run 1—while the PRE-network remains tightly bounded.

Effect of Multiple Refinement Iterations

A variant with three refinement iterations and damping 0.7 was also tested. This configuration reduces the median error even further at higher difficulty levels, while maintaining bounded residuals:

$$\text{PRE (3 iterations) median at Level 3} = 0.02009.$$

Although the mean error remains comparable to the single-iteration PRE, the reduction in median error illustrates how iterative refinement can further stabilize the inverse mapping when the underlying function is highly non-linear.

Interpretation

This domain highlights the central advantage of PRE-style refinement under strict data and compute constraints. The inverse determinant mapping is far more complex than the previous two domains: it requires the model to implicitly reconstruct a structured matrix and its determinant, and then invert a multi-frequency, oscillatory, and non-monotonic function. A monolithic MLP struggles with this task unless given substantially more capacity or training time.

The PRE-network, despite having identical capacity and the same one-million-epoch budget, achieves:

- lower median error across all difficulty levels,
- significantly improved tail stability (no catastrophic failures),
- and better scaling as the difficulty range expands.

These results demonstrate that PRE acts as an efficient learner: it uses its refinement mechanism to extract more value from limited data, stabilizing the inverse mapping even when the underlying function is extremely complex. This behaviour is precisely what is required in the Pre-CAS and CAS architectures introduced later, where models must operate reliably under limited data, limited capacity, and multi-step compositional reasoning.

11.12 Efficient Learning Under Limited Data

Across all three domains—quadratic inversion, complex non-linear inversion, and the inverse determinant mapping—the same pattern emerges. Inverse problems are fundamentally harder than direct function approximation: small perturbations in the latent variable x can produce large, non-linear changes in the observable output. This sensitivity is amplified when the forward mapping contains oscillatory components, multiple interacting frequencies, or structured transformations such as matrix determinants. A monolithic network must implicitly learn the entire inverse function in one step, which requires substantial capacity and large training sets.

The PRE-network approaches the problem differently. Instead of learning a single global inverse, it constructs an internal approximation and then refines it using the evaluator. Even with identical capacity and the same fixed training budget of one million gradient steps, PRE consistently achieves lower median error, improved tail stability, and more predictable behaviour across difficulty levels. This advantage becomes most pronounced in the hardest domain, where the inverse mapping requires the model to reconstruct a structured 3×3 matrix and its determinant before recovering the underlying x. Despite the extreme non-linearity of this task, the PRE-network remains stable and avoids the catastrophic failures observed in the MLP.

These results highlight a key property of PRE-style refinement: it is an efficient learner. When data and compute are limited, PRE extracts more value from each training example by using the evaluator to guide local corrections. This produces a characteristic low-variance residual error—sometimes described as a "machine-native" floor—that reflects the best achievable accuracy under the given constraints. Crucially, this residual remains stable even when the inverse mapping becomes highly ill-conditioned.

This behaviour is precisely what is required in the Pre-CAS and CAS architectures introduced later in this appendix. In those systems, models must operate under strict data limitations, perform multi-step reasoning, and maintain stability across chained computations. The Toy Solver results demonstrate that PRE provides a principled and empirically validated foundation for such

compositional reasoning: it is robust, data-efficient, and capable of handling inverse problems that challenge monolithic networks of the same size.

On Training Budget and Gradient Usage in Mono PRE

All Toy Solver experiments use a fixed training budget of one million gradient steps. For the monolithic MLP this corresponds to one million direct updates of a single forward mapping. For the mono PRE-network, the same budget applies, but each training example is processed through the three PRE roles (proposer, refiner, evaluator) using distinct one-hot role indicators. Although these roles share parameters in mono mode, the gradients they produce are not identical: each role activates different parts of the computation and therefore contributes a different update direction.

This distinction does not undermine the fairness of the comparison. Both models receive the same number of training examples, the same data distribution, and the same total number of gradient applications. The PRE-network simply distributes its learning signal across multiple structured passes, each guided by a different functional objective. In practice, this acts as a form of implicit regularization: instead of overfitting a single global inverse mapping, the PRE-network learns an internal approximation and then refines it using role-specific corrections.

The result is a more efficient use of limited data. Even though the PRE-network performs multiple role-conditioned updates per example, its total capacity remains identical to that of the MLP, and its training budget is not increased. The improved stability and lower median error observed across all Toy Solver domains therefore reflect a genuine architectural advantage rather than additional compute. This structured gradient usage is one of the key reasons why PRE-style refinement generalizes well under the strict data and compute constraints that characterize the Pre-CAS and CAS systems introduced later in this appendix.

Architectural Insight

Every solver is a reminder that intelligence is not a leap, but a loop.

Chapter 12

CAS — The Cognitive Vector Layer

12.1 Introduction — Why Intelligent Systems Need Cognitive Vectors

Modern AI systems are powerful pattern recognizers, but they remain surprisingly limited in one respect: they do not know *what kind* of output they are producing. A classical model can generate text, predict values, or classify inputs, yet it cannot express:

- whether the output is safe or unsafe,
- whether the information is accurate or unreliable,
- whether the content is relevant or off-topic,
- which style, tone, or personality it is following,
- or how the output should be interpreted by downstream systems.

These dimensions matter. They determine whether a system is trustworthy, steerable, and aligned with human expectations. Without them, models must be corrected *after* generation—through filters, heuristics, or external alignment layers—rather than learning these distinctions as part of their internal structure.

CAS introduces a different approach. Instead of treating safety, accuracy, relevance, and style as afterthoughts, CAS represents them as a *cognitive vector*: a structured signal that is learned during training and used during generation. CAS is not a prompt trick, not a post-processing filter, and not a probabilistic confidence estimate. It is an architectural layer that gives a model the ability to express what kind of output it is producing.

In its simplest form, CAS was a triad: *Content–Accuracy–Safety*. In its modern form, CAS is a general vector space of cognitive dimensions. This chapter introduces that evolution and shows how CAS provides a unified interface for training on mixed datasets, guiding generation, and embedding safety directly into the model's internal representations.

12.2 From Triadic Epistemics to CAS(v)

CAS began as a minimal epistemic interface. A model would output three values:

- **content** — the predicted value or token,

- **accuracy** — how certain the model is,
- **safety** — how stable or trustworthy the situation appears.

This triad was the smallest structure capable of expressing uncertainty, detecting unstable regimes, and supporting safe decision-making. It allowed multiple models to form consensus, disagree, or abort automatically when the situation became epistemically unstable.

But as the architecture evolved, something became clear: the triad was only one instance of a much broader idea.

CAS as a General Cognitive Vector

In the modern architecture, CAS is no longer restricted to three dimensions. It is a vector of length v:

$$\mathrm{CAS}(v) = (c_1, c_2, \ldots, c_v),$$

where each dimension represents a cognitive property that the model learns to predict and control.

Examples include:

- **safety**: safe/unsafe content,
- **accuracy**: reliable/unreliable information,
- **relevance**: on-topic/off-topic for a domain,
- **style**: formal/informal, poetic/technical,
- **personality**: concise/verbose, warm/dry, humorous/neutral,
- **domain markers**: mathematics, storytelling, science, law.

The original triad becomes a *special case*:

$$\mathrm{CAS}(3) = (\mathrm{content}, \mathrm{accuracy}, \mathrm{safety}).$$

But CAS(v) is far more general. It is a cognitive interface that allows a model to:

- learn from mixed or contaminated datasets,
- distinguish safe from unsafe material,
- separate accurate from noisy data,
- encode stylistic and personality preferences,
- and generate outputs conditioned on a target CAS vector.

Why CAS Matters

CAS makes it possible to train on real-world data—messy, diverse, inconsistent—without losing control over the model's behavior. Instead of filtering datasets or relying on brittle prompt engineering, CAS teaches the model to *understand* the cognitive properties of its own outputs.

During generation, a target CAS vector acts as a structural guide:

"Produce safe, accurate, relevant content in a concise and technical style."

CAS becomes an architectural steering mechanism, not an afterthought.

In the remainder of this chapter, we explore how CAS is implemented, how CAS-N extends it into a society of cognitive minds, and how CAS forms the epistemic layer of Hybrid Intelligence.

12.3 CAS as a Cognitive Interface for Mixed and Hybrid Data

Real-world datasets are rarely clean. They contain contradictions, stylistic variation, inconsistent quality, and material that ranges from highly reliable to deeply unreliable. Traditional models treat all data as equal: every token contributes equally to the loss, regardless of whether it is safe, accurate, relevant, or stylistically appropriate.

CAS changes this. By attaching a cognitive vector to each training example, CAS provides a structured interface for learning from heterogeneous data. Instead of forcing the model to infer quality, safety, or style implicitly, CAS makes these properties explicit and learnable.

12.3.1 CAS as a Labeling Layer

A CAS vector can be attached to any training sample:

$$(x, y, \mathrm{CAS}(v)).$$

Each dimension of the vector represents a cognitive property that the model should learn to recognize and reproduce. Examples include:

- **safety**: whether the content is safe, harmful, or ambiguous,
- **accuracy**: whether the information is reliable or noisy,
- **relevance**: whether the sample belongs to a specific domain,
- **style**: formal, informal, poetic, technical, narrative,
- **personality**: concise, verbose, warm, neutral, humorous.

This allows a single dataset to contain:

- high-quality expert material,
- low-quality or noisy material,
- safe and unsafe examples,
- multiple writing styles,
- multiple domains,
- multiple personalities.

Instead of filtering or discarding data, CAS turns heterogeneity into a training signal.

12.3.2 Learning Safety and Accuracy During Training

Safety and accuracy are often treated as post-hoc alignment problems: filters, classifiers, or reinforcement layers are added *after* the model is trained. CAS integrates these dimensions directly into the model's internal representation.

A model trained with CAS learns:

- what safe content looks like,
- what unsafe content looks like,
- how to distinguish reliable from unreliable information,
- how to express uncertainty through its accuracy channel.

This makes safety and accuracy *structural* rather than external. The model does not need to be corrected after generation; it learns to produce safe, accurate content by design.

12.3.3 Relevance and Domain Conditioning

CAS also enables domain-aware training. A single model can learn to operate across multiple domains—mathematics, storytelling, science, law—without confusing them.

A relevance dimension in the CAS vector allows the model to:

- identify which domain a sample belongs to,
- learn domain-specific patterns,
- avoid cross-domain contamination,
- switch domains during generation by changing the target vector.

This is especially powerful in modular systems where different pages or experts specialize in different semantic regions.

12.3.4 Style and Personality as Cognitive Dimensions

Style and personality are often controlled through prompting, but prompts are ambiguous, brittle, and difficult to interpret. CAS provides a structural alternative.

A CAS vector can encode:

- concise vs. verbose,
- warm vs. neutral,
- humorous vs. serious,
- poetic vs. technical,
- narrative vs. analytical.

During training, the model learns to associate these stylistic properties with specific patterns in the data. During generation, a target CAS vector allows precise control over the output style without relying on fragile textual instructions.

12.3.5 Why CAS Works for Hybrid and Contaminated Datasets

The strength of CAS lies in its ability to turn messy data into structured cognitive signals. Instead of discarding low-quality or unsafe material, CAS teaches the model to:

- recognize it,
- classify it,
- separate it,
- and avoid reproducing it unless explicitly requested.

This makes CAS a practical tool for training on real-world corpora, where perfect curation is impossible.

CAS is therefore not merely an epistemic mechanism. It is a general cognitive interface that allows models to learn from heterogeneous data while remaining steerable, interpretable, and safe.

In the next section, we examine how CAS is implemented at the model level through multi-output architectures known as CAS-Networks.

12.4 CAS-Networks — A Minimal Architecture for Cognitive Vectors

CAS becomes operational when it is embedded directly into the model architecture. A CAS-Network is a neural model that does not merely predict a token or value, but also produces a cognitive vector describing the properties of that output. This transforms the model from a passive predictor into an active cognitive agent: a system that knows what kind of output it is generating.

12.4.1 Multi-Output Architecture

A CAS-Network maps an input x to two outputs:

$$f_{\text{CAS}}(x) \to (y_{\text{content}}, \text{CAS}(v)).$$

The first component is the primary prediction—typically a token, a regression value, or a classification label. The second component is a v-dimensional cognitive vector encoding the model's assessment of the output's properties.

Each dimension of the CAS vector has its own activation geometry:

- **content** — linear or softmax, depending on the task,
- **safety** — sigmoid, representing safe/unsafe,
- **accuracy** — sigmoid, representing reliable/unreliable,
- **style/personality** — tanh or sigmoid, depending on scale,
- **domain markers** — soft indicators for domain relevance.

This separation is not cosmetic. It forces the network to learn distinct cognitive functions rather than entangling them in a single output channel.

12.4.2 Why CAS-Networks Work So Well

CAS-Networks are typically implemented as multilayer perceptrons (MLPs) or lightweight transformer blocks. Their effectiveness comes from a simple fact: MLPs are excellent universal function approximators for multi-output mappings.

A CAS-Network learns three things simultaneously:

- the primary mapping (content),
- the cognitive properties of that mapping (CAS),
- the relationship between content and cognition.

Because each output channel has its own activation geometry, the model naturally decomposes these roles during training. The result is not a bundle of correlated numbers, but a structured cognitive judgment.

12.4.3 CAS Without PRE

CAS-Networks do not require PRE, recursion, or propose–refine cycles. They function as standalone models:

- in classification tasks,
- in regression tasks,
- in token prediction,
- in sequence generation,
- in domain-specific reasoning.

This makes CAS broadly applicable. It can be added to existing architectures with minimal changes, turning any model into a cognitive system capable of expressing safety, accuracy, relevance, and style.

12.4.4 CAS as Structural Decomposition

The key insight is that CAS is not a heuristic or a post-processing layer. It is a structural decomposition of knowledge:

- **content** expresses what the model believes,
- **accuracy** expresses how reliable that belief is,
- **safety** expresses whether the situation is stable,
- **style/personality** expresses how the belief is framed,
- **domain markers** express where the belief belongs.

This decomposition gives the model a richer internal representation of its own outputs. It becomes capable of distinguishing:

- safe vs. unsafe content,
- accurate vs. unreliable information,
- relevant vs. irrelevant material,
- technical vs. narrative style,
- concise vs. verbose personality.

CAS-Networks therefore form the foundation of the cognitive vector layer. They provide the architectural substrate on which CAS-N, PRE, and Hybrid Intelligence are built.

In the next section, we extend this idea from a single cognitive agent to a society of agents: CAS-N, a distributed epistemic system where multiple CAS-Networks collaborate, disagree, and form consensus.

12.5 CAS-N — A Society of Cognitive Minds

A single CAS-Network can express cognitive properties of its outputs, but it remains one mind with one internal geometry. Real intelligence, however, benefits from diversity: different perspectives, different biases, different internal error landscapes. CAS-N extends the cognitive vector layer from a single agent to a distributed system of agents, each with its own interpretation of content, accuracy, safety, and style.

CAS-N is therefore not a new architecture, but a *collective configuration* of CAS-Networks. It is a society of cognitive minds.

12.5.1 Multiple CAS-Networks with Independent Histories

A CAS-N system consists of N independent CAS-Networks:

$$\{f_1, f_2, \ldots, f_N\}.$$

Each network:

- is trained independently,
- experiences different noise and initialization,
- develops its own internal error geometry,
- produces its own CAS vector for each output.

This independence is essential. It creates epistemic diversity: a set of cognitive agents that do not share the same biases or internal representations.

12.5.2 Consensus as Emergent Agreement

When the N agents agree, their outputs converge:

- content values cluster tightly,
- accuracy values are high,

- safety values are high,
- stylistic or personality dimensions align.

The arbiter observes this convergence and produces a high-confidence judgment. No special rule is needed; consensus is an emergent property of agreement across diverse cognitive agents.

12.5.3 Disagreement and Emergent Abort

When the agents disagree, their outputs diverge:

- content values spread apart,
- accuracy drops,
- safety collapses,
- stylistic dimensions become inconsistent.

The arbiter interprets this divergence as epistemic instability. Confidence collapses to zero, and the system produces an *abort* signal.

This is the safest possible form of refusal:

- not imposed by rules,
- not dependent on thresholds,
- not engineered through heuristics,
- but arising naturally from disagreement among minds.

Abort becomes a structural property of the architecture.

12.5.4 Internal Error Languages

Each CAS-Network develops its own internal error language:

- shaped by its training history,
- nonlinear and asymmetric,
- stable within the network,
- invisible to the arbiter.

The arbiter never sees these internal languages. It only sees their consequences:

- agreement,
- disagreement,
- stability,
- instability.

This is the first time machine-native epistemic diversity becomes visible in a structured system.

12.5.5 CAS-N as a Special Case of CAS(v)

In the modern architecture, CAS-N is no longer the default interpretation of CAS. It is a *special case*:

- CAS(v) defines the cognitive vector space,
- CAS-N defines a society of agents operating in that space.

CAS-N is particularly powerful when:

- epistemic safety is required,
- disagreement must be detectable,
- consensus must be meaningful,
- abort must be structural,
- or multiple cognitive styles must coexist.

But CAS(v) can also be used without CAS-N. A single CAS-Network is often sufficient for style control, safety conditioning, or training on mixed datasets.

CAS-N is therefore best understood as a distributed epistemic layer: a society of cognitive minds whose interactions reveal when the system should trust itself—and when it should not.

In the next section, we examine how CAS is used during training and generation, and how cognitive vectors provide a unified interface for safety, style, relevance, and domain control.

12.6 CAS in Practice — Training, Labeling, and Generation

CAS is not merely a conceptual layer. It is a practical interface that can be used during training, evaluation, and generation. By attaching cognitive vectors to data and conditioning models on target vectors during inference, CAS provides a unified mechanism for safety, style, relevance, and domain control.

This section describes how CAS is used in real systems: how datasets are labeled, how models learn cognitive properties, and how generation becomes steerable through target CAS vectors.

12.6.1 Training with CAS-Labeled Data

A CAS-labeled dataset consists of triples:

$$(x, y, \mathrm{CAS}(v)).$$

Each training example includes:

- the input x,
- the desired output y,
- a cognitive vector describing the properties of y.

This allows a single dataset to encode multiple cognitive dimensions:

- safe vs. unsafe content,
- accurate vs. unreliable information,
- relevant vs. irrelevant material,
- domain-specific vs. domain-agnostic samples,
- stylistic and personality traits.

Instead of curating separate datasets for each property, CAS allows all material to coexist in one unified corpus. The model learns to associate each cognitive dimension with its corresponding patterns in the data.

12.6.2 Learning Safety During Training

Safety is often treated as a post-hoc alignment problem. CAS integrates safety directly into the model's internal representation.

A CAS dimension can encode:

- safe content (1),
- unsafe content (0),
- ambiguous or borderline content (intermediate values).

During training, the model learns:

- what safe content looks like,
- what unsafe content looks like,
- how to detect transitions between safe and unsafe regimes,
- how to express uncertainty through its safety channel.

This makes safety a structural property of the model rather than an external constraint.

12.6.3 Accuracy and Reliability as Cognitive Dimensions

Accuracy is another dimension that benefits from explicit labeling. Instead of assuming all training data is equally reliable, CAS allows the model to learn:

- which samples are trustworthy,
- which samples are noisy or approximate,
- how to express uncertainty in its accuracy channel,
- how to avoid overconfident extrapolation.

This is particularly useful in hybrid datasets that mix:

- expert material,

- synthetic data,
- user-generated content,
- partially correct or approximate examples.

CAS teaches the model to treat these sources differently.

12.6.4 Relevance and Domain Conditioning

CAS can encode domain relevance:

$$\mathrm{CAS}_{\mathrm{relevance}} = 1 \quad \text{(in-domain)},$$

$$\mathrm{CAS}_{\mathrm{relevance}} = 0 \quad \text{(out-of-domain)}.$$

This allows the model to:

- learn domain boundaries,
- avoid cross-domain contamination,
- switch domains during generation,
- specialize without losing generality.

Domain conditioning becomes a structural property rather than a prompt engineering trick.

12.6.5 Style and Personality Control

CAS provides a stable mechanism for controlling style and personality. Instead of relying on textual prompts—which are ambiguous and prone to drift—CAS encodes stylistic properties directly in the cognitive vector.

Examples include:

- concise vs. verbose,
- poetic vs. technical,
- warm vs. neutral,
- humorous vs. serious,
- narrative vs. analytical.

During training, the model learns to associate these dimensions with specific stylistic patterns. During generation, a target CAS vector provides precise control over the output style.

12.6.6 Using CAS During Generation

During inference, the model receives:

$$(x, \mathrm{CAS}_{\mathrm{target}}(v)).$$

The target CAS vector acts as a cognitive guide:

> "Generate safe, accurate, relevant content in a concise and technical style."

This makes generation:

- more predictable,
- more controllable,
- more interpretable,
- and less dependent on fragile prompt engineering.

CAS becomes an architectural steering mechanism.

12.6.7 CAS as a Replacement for Prompt Engineering

Prompt engineering relies on natural language instructions, which are:

- ambiguous,
- context-dependent,
- sensitive to phrasing,
- and difficult to interpret.

CAS provides a structural alternative:

- numeric,
- explicit,
- stable,
- and directly tied to training.

Instead of writing:

> "Please answer safely, accurately, and in a formal tone."

one simply provides:

$$\mathrm{CAS}_{\mathrm{target}} = (1, 1, 1, 0.8, 0.2, \ldots).$$

CAS turns cognitive control into an architectural feature rather than a linguistic trick.

In the next section, we examine how CAS forms the epistemic layer of Hybrid Intelligence and how it integrates with PRE to create the PRE–CAS family of composite minds.

12.7 CAS as the Epistemic Layer of Hybrid Intelligence

Hybrid Intelligence is built on two complementary principles: *recursion* and *epistemics*. PRE provides the recursive structure: propose–refine cycles, local improvement, and iterative reasoning. CAS provides the epistemic structure: safety, accuracy, relevance, style, and domain awareness.

Together, they form a cognitive system that is both self-improving and self-aware in a structural sense.

12.7.1 Why Hybrid Systems Need Epistemics

A recursive system without epistemics is unstable. It can refine an unsafe idea, reinforce an inaccurate belief, or amplify irrelevant content. PRE alone cannot distinguish:

- safe from unsafe refinements,
- accurate from unreliable proposals,
- relevant from irrelevant directions,
- or appropriate from inappropriate styles.

CAS provides the missing layer. It gives each refinement step a cognitive context, allowing the system to evaluate not only *what* it is doing, but *how* it is doing it.

12.7.2 CAS as a Structural Safety Mechanism

Safety is not an afterthought in Hybrid Intelligence. It is embedded directly into the architecture through CAS:

- unsafe content is labeled during training,
- the model learns to detect unsafe regimes,
- safety becomes a dimension of the cognitive vector,
- unsafe refinements collapse the safety channel,
- CAS-N disagreement triggers emergent abort.

This makes safety a *structural property* rather than a filter or external constraint.

12.7.3 CAS as a Guide for Recursive Reasoning

During a PRE cycle, each refinement step receives:

$$(x, \mathrm{CAS}_{\mathrm{target}}(v)).$$

The CAS vector guides the refinement:

- accuracy encourages reliable improvements,
- safety prevents drift into unstable regions,
- relevance keeps the reasoning on-topic,
- style and personality shape the form of the output.

CAS therefore acts as a cognitive compass for recursive reasoning.

12.7.4 CAS-N as Distributed Epistemics

When multiple experts operate within a page, CAS-N provides:

- epistemic diversity,
- disagreement detection,
- consensus formation,
- structural abort,
- and distributed authority.

This is particularly important in Hybrid Intelligence, where many small agents collaborate. CAS-N ensures that the system does not rely on a single epistemic perspective.

12.7.5 Integration with PRE — The PRE–CAS Family

PRE and CAS integrate naturally:

- PRE provides recursive improvement,
- CAS provides cognitive evaluation,
- PRE–CAS combines both into a hybrid kernel,
- CAS-N adds distributed epistemics,
- PAGED(p) organizes experts into semantic chambers,
- M ensembles provide evolutionary diversity.

The result is the PRE–CAS family: a colony of composite minds, each guided by cognitive vectors and refined through recursive reasoning.

12.7.6 Why CAS Is the Epistemic Layer

CAS is the epistemic layer of Hybrid Intelligence because it provides:

- **structured uncertainty** through accuracy,
- **structural safety** through safety,
- **domain awareness** through relevance,
- **stylistic control** through personality and style,
- **alignment** through target vectors,
- **distributed epistemics** through CAS-N.

Without CAS, Hybrid Intelligence would be recursive but blind. With CAS, it becomes recursive, aware, steerable, and safe.

In the final section, we synthesize the role of CAS as a universal cognitive substrate that unifies safety, style, relevance, and epistemic judgment across all layers of the architecture.

12.8 Synthesis — CAS as a Universal Cognitive Substrate

CAS began as a minimal epistemic interface, but it has grown into something broader: a universal cognitive substrate for intelligent systems. It unifies safety, accuracy, relevance, style, and domain awareness into a single architectural layer that can be learned during training and controlled during generation.

CAS is not a filter, not a heuristic, and not a prompt trick. It is a structural representation of the cognitive properties of content—embedded directly into the model's outputs and internal representations.

A Unified Interface for Cognitive Control

Across all tasks and domains, CAS provides a consistent mechanism for:

- **safety**: identifying and avoiding unsafe regimes,
- **accuracy**: expressing uncertainty and reliability,
- **relevance**: maintaining domain boundaries,
- **style**: controlling tone, form, and personality,
- **alignment**: steering generation through target vectors.

This makes CAS a universal interface for cognitive control—usable in any model, any dataset, and any architecture.

A Bridge Between Data and Cognition

CAS transforms heterogeneous datasets into structured cognitive signals. Instead of discarding noisy or stylistically diverse material, CAS teaches the model to understand it:

- what is safe,
- what is accurate,
- what is relevant,
- what belongs to which domain,
- and how content should be framed.

This allows models to learn from real-world data without losing steerability or safety.

A Foundation for Distributed Epistemics

CAS-N extends CAS from a single agent to a society of agents. It provides:

- epistemic diversity,
- consensus formation,
- disagreement detection,
- and emergent abort.

This makes CAS not only a cognitive layer, but also an epistemic layer: a substrate for distributed reasoning and structural safety.

Integration with Hybrid Intelligence

In Hybrid Intelligence, CAS integrates seamlessly with PRE:

- PRE provides recursive refinement,
- CAS provides cognitive evaluation,
- PRE–CAS forms the hybrid kernel,
- CAS-N adds distributed epistemics,
- PAGED(p) organizes semantic structure,
- M ensembles provide evolutionary diversity.

CAS is therefore the epistemic backbone of the entire architecture.

The Role of CAS in the Triadic Cosmos

Within the broader unification presented in this book, CAS plays a structural role:

- it decomposes cognition into explicit dimensions,
- it provides a universal interface for control,
- it embeds safety and accuracy into the model itself,
- it enables distributed epistemics through CAS-N,
- and it unifies heterogeneous data into a coherent cognitive space.

CAS is not a detail of the architecture. It is one of its foundations.

Architectural Insight

CAS is a universal cognitive substrate: a vector space in which safety, accuracy, relevance, style, and alignment become structural properties of intelligent systems.

Chapter 13

The PRE–CAS Family: A General Framework for Modular Reasoning

13.1 Introduction

Contemporary machine learning systems are dominated by monolithic architectures: a single model, a single parameter space, and a single, entangled inductive bias. Such systems are powerful pattern recognizers, but they offer limited modularity, poor traceability, and little control over how constraints and objectives shape individual decisions. Reasoning, when it emerges, is implicit and opaque.

The PRE–CAS architecture takes a different approach. It is a general, representation-agnostic framework for structured decision-making. PRE–CAS does not assume a particular domain (language, control, planning), nor a particular granularity (tokens, actions, tool calls, state transitions). Instead, it defines a *cognitive pipeline* that can operate on arbitrary candidate actions, provided they can be evaluated and compared.

In later chapters, PRE–CAS will be instantiated in concrete miniature worlds. In this chapter, we treat it in its most general form: as an architectural pattern for modular reasoning, built from domain decomposition, iterative proposal refinement, orthogonal evaluators, pre-selection filtering, and controlled arbitration.

13.2 The PRE–CAS Stack

At its core, PRE–CAS consists of five interacting components:

1. **Paging** — a decomposition of the global action space into coherent domains, each handled by its own expert or set of experts.

2. **PRE** (Propose–Refine–Evaluate) — an iterative reasoning loop that generates, improves, and assesses candidate actions.

3. **Evaluators** — a collection of orthogonal constraint functions, heuristic or learned, that score candidates along independent dimensions.

4. **Filtering** — a mechanism that removes invalid or structurally inconsistent candidates *before* any final selection is made.

5. **The Arbiter** — an integrative decision-maker that selects the final action from a curated set of valid proposals, balancing correctness with controlled diversity.

These components form a modular pipeline:

$$\text{Context} \xrightarrow{\text{Paging}} \text{Domain Expert} \xrightarrow{\text{PRE}} \text{Candidate Set} \xrightarrow{\text{Evaluators + Filtering}} \text{Valid Set} \xrightarrow{\text{Arbiter}} \text{Action}.$$

The architecture is compatible with both single-model and multi-model settings. In the latter case, multiple PRE–CAS instances can run in parallel, each with its own parameters and history, forming a population of reasoning agents that can be analyzed at the ensemble level.

13.3 Design Principles

PRE–CAS is guided by four architectural principles:

- **Modularity.** Each component—pages, experts, evaluators, filtering, arbiter—is separable, inspectable, and replaceable. The architecture encourages local changes rather than global retraining.
- **Orthogonality.** Different evaluators encode different constraints; experts specialize in disjoint domains; refinement steps incorporate feedback without collapsing all signals into a single undifferentiated score.
- **Traceability.** Every proposal, score, and decision is explicit. PRE–CAS exposes its internal reasoning trajectory instead of hiding it inside a monolithic parameter space.
- **Plurality.** The architecture supports multiple experts per domain and multiple full models in parallel, enabling diversity, robustness, and population-level analysis.

The remainder of this chapter formalizes each component of the PRE–CAS stack in turn, starting from domain decomposition (Paging) and progressing through the PRE loop, evaluators, filtering, arbitration, and ensemble-level structure.

13.4 Paging: Domain Decomposition of the Action Space

A central design principle of PRE–CAS is that no single expert should be responsible for the entire action space. Instead, the architecture decomposes the global space of possible actions into a set of *pages*: coherent, non-overlapping domains, each handled by its own expert or set of experts.

Paging is representation-agnostic. A "page" may correspond to a subset of tokens, a family of actions, a class of state transitions, a tool interface, or any other structured region of the decision space. The PRE–CAS architecture does not prescribe how pages must be defined; it only requires that the decomposition be deterministic and that each page be internally coherent.

13.4.1 The Action Space

Let $\mathcal{A}$ denote the full action space of the system. An action may be:

- a discrete symbol,

- a structured object,
- a tool invocation,
- a state transition,
- a planning operator,
- or any other atomic decision unit.

The architecture assumes only that actions can be enumerated or generated, and that they can be evaluated by the system's evaluators.

13.4.2 Definition of a Page

A *page* is a subset of the action space:

$$P_i \subseteq \mathcal{A},$$

such that the collection of pages forms a partition:

$$\mathcal{A} = \bigcup_{i=1}^{p} P_i, \qquad P_i \cap P_j = \varnothing \text{ for } i \neq j.$$

Each page is associated with one or more *experts*, which are models specialized in predicting or generating actions within that page.

13.4.3 The Paging Function

Paging is governed by a deterministic routing function:

$$\pi : \mathcal{C} \to \{1, \ldots, p\},$$

where $\mathcal{C}$ is the space of contexts. Given the current context $c \in \mathcal{C}$, the paging function selects the index of the page whose experts should be activated.

The paging function may depend on:

- the current state or history,
- the last chosen action,
- structural constraints,
- domain-specific rules,
- or learned routing heuristics.

The architecture does not constrain how π is implemented; it only requires that the mapping be deterministic so that the reasoning process remains traceable.

13.4.4 Experts Within a Page

Each page P_i contains N experts:

$$E_{i,1}, E_{i,2}, \ldots, E_{i,N}.$$

Experts within the same page share:

- the same action domain,
- the same input structure,
- the same evaluator set,
- and the same PRE loop.

They differ only in their parameters or training histories. This local plurality provides robustness and controlled variation without requiring global model diversity.

13.4.5 Why Paging Matters

Paging provides several architectural advantages:

- **Modularity.** Each page isolates a coherent region of the action space, enabling specialization and simplifying expert design.
- **Safety.** Invalid or out-of-domain actions are never proposed, because experts operate only within their assigned page.
- **Scalability.** The action space can grow without requiring a single model to handle all possible decisions.
- **Interpretability.** The paging function makes the system's routing decisions explicit and traceable.
- **Parallelism.** Multiple experts can be trained or evaluated independently within each page, enabling efficient ensemble methods.

Paging is therefore the structural foundation of PRE–CAS: it decomposes the decision space into manageable domains and ensures that each expert operates within a well-defined region of competence.

13.5 The PRE Loop: Propose, Refine, Evaluate

At the heart of the PRE–CAS architecture lies the PRE loop: an iterative mechanism for generating, improving, and assessing candidate actions. PRE is a general cognitive operator. It does not assume that actions are tokens, moves, tool calls, or symbolic steps; it only assumes that actions can be proposed, refined, and evaluated.

The PRE loop transforms decision-making from a single-step prediction into a structured reasoning process. Each iteration produces a richer set of candidates, shaped by evaluator feedback and constrained by the domain selected through paging.

13.5.1 Input Structure

Each PRE iteration receives a structured input vector

$$x = (c, a_{\text{prev}}, a_{\text{prop}}, e, r),$$

where:

- c is the current context,
- a_{prev} is the previous action (or a null symbol at the start),
- a_{prop} is the current proposal being refined,
- e is the vector of evaluator scores from the previous iteration,
- $r \in \{\text{P}, \text{R}\}$ is a role indicator distinguishing between the initial *propose* step and subsequent *refine* steps.

This structure ensures that PRE has access to both the global context and the local refinement trajectory.

13.5.2 The Propose Step

The first iteration of PRE generates an initial proposal:

$$a_0 = \text{PRE}(c, a_{\text{prev}}, \varnothing, \varnothing, \text{P}).$$

This proposal is unconstrained except by the expert associated with the current page. It represents the model's first hypothesis about the next action.

13.5.3 The Refinement Step

Subsequent iterations refine the proposal using evaluator feedback. Given a proposal a_i and evaluator scores e_i, PRE produces a new candidate:

$$a_{i+1} = \text{PRE}(c, a_{\text{prev}}, a_i, e_i, \text{R}).$$

Refinement allows the system to correct weak proposals, incorporate structural or semantic constraints, and explore alternative trajectories within the same page.

13.5.4 Evaluator Feedback

After each proposal or refinement, the evaluators compute a score vector:

$$e_i = (e_i^{(1)}, e_i^{(2)}, \ldots, e_i^{(m)}),$$

where each component corresponds to a distinct constraint dimension.

Evaluator feedback is not aggregated inside PRE. Instead, PRE receives the full vector, preserving orthogonality and allowing the model to respond differently to different constraint types.

13.5.5 Accumulating Candidates

Each iteration produces a candidate action and its associated evaluator scores. After k refinement steps, the PRE loop yields a set:

$$\mathcal{C} = \{(a_0, e_0), (a_1, e_1), \ldots, (a_k, e_k)\}.$$

This set is passed to the filtering and arbitration stages. PRE itself does not select among candidates; it only generates and refines them.

13.5.6 PRE Pseudocode

Algorithm 1: General PRE Loop

Input: context c, previous action a_{prev}, refinement depth k
Output: candidate set $\mathcal{C}$
$\mathcal{C} \leftarrow \varnothing$;
$a \leftarrow \text{PRE}(c, a_{\text{prev}}, \varnothing, \varnothing, \text{P})$;
$e \leftarrow \text{Evaluate}(a, c)$;
$\mathcal{C} \leftarrow \mathcal{C} \cup \{(a, e)\}$;
for $i \leftarrow 1$ **to** k **do**
  $a \leftarrow \text{PRE}(c, a_{\text{prev}}, a, e, \text{R})$;
  $e \leftarrow \text{Evaluate}(a, c)$;
  $\mathcal{C} \leftarrow \mathcal{C} \cup \{(a, e)\}$;
return $\mathcal{C}$;

13.5.7 Why PRE Matters

The PRE loop provides several advantages over single-step prediction:

- **Structured reasoning.** Decisions emerge from iterative refinement rather than a single forward pass.
- **Constraint integration.** Evaluator feedback is incorporated explicitly at each step.
- **Robustness.** Weak initial proposals can be corrected during refinement.
- **Interpretability.** Each refinement step exposes the system's internal reasoning trajectory.
- **General applicability.** PRE operates on arbitrary actions and is not tied to any specific domain.

PRE transforms action generation into a multi-step cognitive process, laying the foundation for the filtering and arbitration mechanisms that follow.

13.6 Evaluators: Orthogonal Constraint Functions

Evaluators are the constraint-enforcing components of the PRE–CAS architecture. They assess candidate actions along independent dimensions and provide explicit feedback to the PRE loop. Evaluators do not generate actions themselves; they score them. Their role is to make constraints *visible* to the system and to ensure that reasoning is shaped by structured, interpretable signals.

Evaluators are representation-agnostic. They may operate on tokens, symbolic actions, state transitions, tool calls, or any other form of decision unit. The architecture imposes no restrictions on how evaluators are implemented, only that they return a vector of scores that PRE can use during refinement.

13.6.1 Evaluator Functions

An evaluator is a function

$$E_j : \mathcal{A} \times \mathcal{C} \to [0, 1],$$

mapping an action $a \in \mathcal{A}$ and context $c \in \mathcal{C}$ to a normalized score. A score of 1 indicates full compliance with the constraint, while a score of 0 indicates a violation.

Given m evaluators, the system computes a score vector

$$e = (E_1(a, c), E_2(a, c), \ldots, E_m(a, c)).$$

This vector is passed unchanged to the PRE loop and later to the filtering and arbitration stages. PRE–CAS does not collapse evaluator outputs into a single scalar; orthogonality is preserved throughout the pipeline.

13.6.2 Types of Evaluators

Evaluators may be:

- **Heuristic.** Rule-based functions encoding structural, logical, or domain-specific constraints.
- **Learned.** Neural or statistical models trained to assess coherence, plausibility, or other high-level properties.
- **Hybrid.** Combinations of deterministic rules and learned components.
- **Hard constraints.** Evaluators whose violations immediately invalidate a candidate.
- **Soft constraints.** Evaluators whose scores influence arbitration but do not eliminate candidates outright.

The architecture does not prescribe the number or type of evaluators. Different instantiations may use different sets depending on the domain.

13.6.3 Orthogonality

A defining property of evaluators in PRE–CAS is *orthogonality*. Each evaluator captures a distinct dimension of correctness or quality. This separation has several advantages:

- **Interpretability.** Each constraint is visible and independently measurable.
- **Modularity.** Evaluators can be added, removed, or replaced without affecting the rest of the architecture.
- **Fine-grained feedback.** PRE receives a full vector of scores, enabling targeted refinement.
- **Avoiding entanglement.** Constraints do not collapse into a single opaque signal.

Orthogonality ensures that the system's reasoning process remains transparent and that constraints can be analyzed individually.

13.6.4 Evaluator Interface

All evaluators share a common interface:

- **Input:** a candidate action a and the current context c.
- **Output:** a normalized score $s \in [0, 1]$.
- **Optional metadata:** evaluators may return auxiliary information (e.g., violation types), but only the score is required by the architecture.

This uniform interface allows PRE, filtering, and arbitration to treat all evaluators identically, regardless of their internal implementation.

13.6.5 Evaluator Set

Let $\mathcal{E} = \{E_1, \ldots, E_m\}$ denote the evaluator set. The PRE–CAS architecture assumes:

- $\mathcal{E}$ is fixed during inference,
- evaluators operate independently,
- evaluators do not communicate with one another,
- and evaluators do not modify candidate actions.

Evaluators are therefore *critics*, not generators.

13.6.6 Role in the PRE–CAS Pipeline

Evaluators shape the reasoning process in three ways:

1. **During PRE.** Evaluator scores guide refinement, allowing PRE to correct weak proposals.
2. **During filtering.** Hard constraints eliminate invalid candidates before arbitration.
3. **During arbitration.** Soft constraints influence the final selection among valid candidates.

Evaluators thus form the backbone of correctness and structure within the PRE–CAS architecture. They define what it means for an action to be valid, coherent, or aligned with the system's objectives.

13.7 Filtering: Validity Before Selection

After the PRE loop has generated a set of candidate actions and their evaluator scores, the next stage of the PRE–CAS pipeline is *filtering*. The purpose of filtering is to ensure that only structurally valid, contextually appropriate, and constraint-compliant candidates are passed to the arbiter.

Filtering is a pre-selection mechanism: it does not rank candidates, and it does not choose among them. Instead, it removes candidates that violate hard constraints or fall below minimal quality thresholds. This guarantees that the arbiter operates only on a curated set of valid actions.

13.7.1 Input to Filtering

Filtering receives the full candidate set produced by PRE:

$$\mathcal{C} = \{(a_0, e_0), (a_1, e_1), \ldots, (a_k, e_k)\},$$

where each pair consists of:

- a_i — a candidate action,
- e_i — the evaluator score vector for that action.

Filtering does not modify evaluator scores; it only uses them to determine whether a candidate should be retained.

13.7.2 Hard Constraints

A *hard constraint* is a rule that must be satisfied for a candidate to be considered valid. Hard constraints may include:

- structural validity,
- domain-specific legality,
- type or format requirements,
- safety or alignment constraints,
- evaluator scores that must equal 1.

Formally, let $\mathcal{H}$ denote the set of hard constraints. A candidate (a_i, e_i) is valid under hard constraints if:

$$h(a_i, e_i) = 1 \quad \text{for all } h \in \mathcal{H}.$$

Candidates failing any hard constraint are removed immediately.

13.7.3 Soft Thresholding

In addition to hard constraints, filtering may apply *soft thresholds* to evaluator scores. A soft threshold does not enforce strict validity; instead, it ensures that candidates meet a minimal quality level before reaching arbitration.

Given a threshold $\tau \in [0, 1]$, a candidate passes soft filtering if:

$$\min_j e_i^{(j)} \geq \tau,$$

or, more generally, if it satisfies a thresholding function:

$$T(a_i, e_i, \tau) = 1.$$

Soft thresholds allow the system to balance correctness and diversity by adjusting how permissive the filtering stage is.

13.7.4 Filtered Candidate Set

Let $\mathcal{F}$ denote the filtered set:

$$\mathcal{F} = \{(a_i, e_i) \in \mathcal{C} \mid \text{all hard constraints satisfied and soft thresholds met}\}.$$

If $\mathcal{F}$ is empty, the system may:

- relax thresholds,
- fall back to the highest-scoring candidate in $\mathcal{C}$,
- or trigger a recovery mechanism defined by the instantiation.

The PRE–CAS architecture does not prescribe a specific fallback strategy.

Algorithm 2: Filtering Stage

Input: candidate set $\mathcal{C}$, hard constraints $\mathcal{H}$, threshold τ
Output: filtered set $\mathcal{F}$
$\mathcal{F} \leftarrow \varnothing$;
foreach $(a, e) \in \mathcal{C}$ **do**
  if *exists* $h \in \mathcal{H}$ *such that* $h(a, e) = 0$ **then**
    continue;
  if $T(a, e, \tau) = 0$ **then**
    continue;
  $\mathcal{F} \leftarrow \mathcal{F} \cup \{(a, e)\}$;
return $\mathcal{F}$;

13.7.5 Why Filtering Matters

Filtering plays a crucial role in the PRE–CAS architecture:

- **Safety.** Invalid or harmful actions are removed before they can influence the final decision.
- **Structural integrity.** Only candidates that satisfy domain-specific rules reach the arbiter.
- **Efficiency.** The arbiter operates on a smaller, cleaner set of candidates.
- **Interpretability.** Filtering makes constraint violations explicit and traceable.
- **Control.** Thresholds allow fine-grained tuning of the trade-off between correctness and diversity.

Filtering ensures that the arbiter receives only candidates that are both structurally valid and minimally aligned with the system's objectives, enabling robust and interpretable decision-making.

13.8 The Arbiter: Integration and Controlled Variation

After filtering has removed invalid or low-quality candidates, the PRE–CAS pipeline reaches its final decision-making component: the *arbiter*. The arbiter selects the final action from the filtered candidate set, balancing correctness, evaluator alignment, and controlled variation.

Unlike PRE, which generates and refines candidates, or evaluators, which score them, the arbiter is a *selector*. Its role is to integrate multiple signals into a single decision while preserving transparency and traceability.

13.8.1 Input to the Arbiter

The arbiter receives the filtered candidate set:

$$\mathcal{F} = \{(a_i, e_i)\},$$

where each element consists of:

- a_i — a valid candidate action,
- e_i — its evaluator score vector.

All candidates in $\mathcal{F}$ satisfy hard constraints and meet the minimum threshold requirements. The arbiter therefore operates only on actions that are structurally valid and contextually appropriate.

13.8.2 Scoring Function

The arbiter computes a scalar score for each candidate using a scoring function:

$$S(a_i, e_i) = f(e_i),$$

where f is a deterministic function that aggregates evaluator scores.

The architecture does not prescribe a specific form for f. Common choices include:

- weighted sums,
- geometric or harmonic means,
- lexicographic ordering,
- evaluator-specific priority rules,
- or learned aggregation functions.

The only requirement is that f be transparent and interpretable.

13.8.3 Selecting the Best Candidate

The arbiter identifies the highest-scoring candidate:

$$(a^*, e^*) = \arg \max_{(a_i, e_i) \in \mathcal{F}} S(a_i, e_i).$$

If multiple candidates tie for the highest score, the arbiter may:

- select the earliest candidate,
- select the most refined candidate,
- or apply a controlled variation mechanism (see below).

13.8.4 Controlled Variation

To avoid deterministic collapse and to maintain diversity in the action distribution, the arbiter may introduce controlled variation. This is achieved by selecting from a *score cluster* rather than a single maximum.

Let

$$\mathcal{F}_\delta = \{(a_i, e_i) \in \mathcal{F} \mid S(a^*, e^*) - S(a_i, e_i) \leq \delta\},$$

where $\delta \geq 0$ is a variation parameter.

The arbiter then samples from $\mathcal{F}_\delta$ according to a distribution defined by the instantiation (uniform, softmax, etc.).

Controlled variation provides:

- robustness against overfitting to narrow attractors,
- increased diversity in long sequences of decisions,
- and a mechanism for exploring alternative valid actions.

13.8.5 Arbiter Pseudocode

Algorithm 3: Arbiter

Input: filtered set $\mathcal{F}$, scoring function S, variation parameter δ
Output: final action a_{final}
Compute $S(a, e)$ for all $(a, e) \in \mathcal{F}$;
$(a^*, e^*) \leftarrow \arg\max_{(a,e)\in\mathcal{F}} S(a, e)$;
$\mathcal{F}_\delta \leftarrow \{(a, e) \in \mathcal{F} \mid S(a^*, e^*) - S(a, e) \leq \delta\}$;
Select $(a_{\text{final}}, e_{\text{final}})$ from $\mathcal{F}_\delta$;
return a_{final};

13.8.6 Why the Arbiter Matters

The arbiter is the final decision-maker in the PRE–CAS pipeline. Its role is crucial for several reasons:

- **Integration.** It combines evaluator signals into a single, interpretable decision.
- **Correctness.** It ensures that the chosen action is the best-scoring valid candidate.
- **Diversity.** Controlled variation prevents deterministic collapse and encourages exploration of alternative valid actions.
- **Traceability.** The scoring function and selection mechanism make the final decision fully transparent.
- **Modularity.** The arbiter can be replaced or extended without modifying PRE, evaluators, or paging.

Together with filtering, the arbiter transforms the PRE-generated candidate set into a single, high-quality action that respects both structural constraints and evaluator-defined objectives.

13.9 Ensembles: Plurality of Minds

Although PRE–CAS can operate as a single-model architecture, it naturally extends to a multi-model setting. An *ensemble* consists of M independent PRE–CAS instances, each with its own parameters, training history, and internal representations. Ensembles introduce plurality into the reasoning process, providing robustness, diversity, and a foundation for population-level analysis.

The ensemble layer is optional but conceptually important: it transforms PRE–CAS from a single cognitive pipeline into a small society of interacting reasoners.

13.9.1 Definition of an Ensemble

An ensemble is a collection:

$$\mathcal{M} = \{M_1, M_2, \ldots, M_M\},$$

where each M_i is a complete PRE–CAS system with its own:

- paging function,
- experts and parameters,
- evaluator set,
- filtering mechanism,
- arbiter.

The architecture does not require ensemble members to share parameters or training data. They may be identical, partially aligned, or entirely distinct.

13.9.2 Parallel Reasoning

Given a context c, each model M_i independently produces an action:

$$a_i = M_i(c).$$

This yields an action set:

$$\mathcal{A}_{\text{ens}} = \{a_1, a_2, \ldots, a_M\}.$$

The ensemble therefore provides M parallel perspectives on the same context.

13.9.3 Ensemble Arbitration

The architecture allows, but does not require, a *global arbiter* that aggregates the outputs of all models. A global arbiter may:

- select the majority action,
- choose the highest-scoring action under a shared evaluator set,
- apply diversity-aware selection rules,
- or simply record the distribution for analysis.

Formally, a global arbiter is a function:

$$A_{\text{global}} : \mathcal{A}_{\text{ens}} \to \mathcal{A}.$$

The PRE–CAS architecture does not prescribe a specific aggregation rule.

13.9.4 Benefits of Ensembles

Ensembles provide several architectural advantages:

- **Robustness.** Independent models reduce the risk of systematic failure modes.
- **Diversity.** Different parameterizations yield different attractors and reasoning styles.
- **Plurality.** Multiple models provide multiple hypotheses, enabling richer decision-making.
- **Traceability.** Differences between models can be analyzed to understand uncertainty, disagreement, or structural bias.
- **Evolvability.** Ensemble members can be retrained, replaced, or mutated independently.

Ensembles turn PRE–CAS into a population-based reasoning system, enabling ecological dynamics such as convergence, divergence, specialization, and collective behavior.

13.9.5 Architectural Neutrality

The ensemble layer is deliberately neutral:

- It does not assume communication between models.
- It does not require shared evaluators or shared paging.
- It does not impose a specific aggregation mechanism.

This neutrality allows PRE–CAS to support a wide range of multi-agent configurations, from loosely coupled populations to tightly integrated collective systems.

13.9.6 Summary

The ensemble layer extends PRE–CAS from a single reasoning pipeline to a pluralistic architecture. By running multiple PRE–CAS instances in parallel, the system gains robustness, diversity, and the ability to analyze reasoning at the population level. This plurality is a natural complement to the modular structure of PRE–CAS and plays an important role in large-scale deployments.

13.10 Summary and Architectural Outlook

The PRE–CAS architecture provides a general, modular, and interpretable framework for structured decision-making. It decomposes reasoning into transparent components: domain routing through paging, iterative proposal generation through PRE, orthogonal constraint evaluation, validity filtering, and controlled arbitration. Each component is independent yet tightly integrated, forming a cognitive pipeline that is both flexible and robust.

13.10.1 Architectural Summary

The key elements of PRE–CAS can be summarized as follows:

- **Paging** partitions the action space into coherent domains, enabling specialization, safety, and modularity.
- **PRE** transforms decision-making into an iterative reasoning process, where proposals are refined using explicit evaluator feedback.
- **Evaluators** provide orthogonal constraint signals, ensuring that correctness, structure, and alignment remain visible and measurable.
- **Filtering** enforces validity before selection, guaranteeing that only structurally sound candidates reach the arbiter.
- **The Arbiter** integrates evaluator signals into a final decision, balancing correctness with controlled variation.
- **Ensembles** extend the architecture from a single reasoning pipeline to a pluralistic system, enabling robustness, diversity, and population-level analysis.

Together, these components form a modular reasoning architecture that is transparent, extensible, and representation-agnostic.

13.10.2 Design Properties

PRE–CAS exhibits several desirable properties for large-scale reasoning systems:

- **Modularity.** Components can be added, removed, or replaced without altering the overall structure.
- **Traceability.** Every proposal, score, and decision is explicit and inspectable.
- **Orthogonality.** Constraints remain independent, avoiding entanglement and preserving interpretability.
- **Plurality.** Multiple experts and multiple models enable diverse reasoning styles and robust decision-making.
- **General applicability.** PRE–CAS operates on arbitrary actions and is not tied to any specific domain or representation.

These properties make PRE–CAS suitable for a wide range of reasoning tasks, from symbolic manipulation to generative modeling, planning, and multi-agent systems.

13.10.3 Outlook

The PRE–CAS architecture is intentionally general. It defines a pattern for modular reasoning that can be instantiated in many different environments. Subsequent chapters will explore concrete realizations of this architecture in specific miniature worlds, demonstrating how the abstract components of PRE–CAS behave in practice.

These instantiations are not part of the architecture itself; they are applications. The architecture remains independent of any particular domain, granularity, or training regime. Its purpose is to provide a principled, transparent, and extensible foundation for structured cognition.

In summary, PRE–CAS offers a blueprint for building reasoning systems that are modular, interpretable, and adaptable. It replaces monolithic prediction with structured decision-making, enabling systems that can reason, refine, and evaluate their own actions in a transparent and principled manner.

13.11 PRE–CAS Overview Table

Component	Input	Output	Role	Guarantees	Variations
Paging	Context c	Page index p	Selects domain for next action	Deterministic routing; domain coherence	Rule-based, learned, hybrid
Expert	c, a_{prev}	Proposal logits / action	Generates actions within one domain	No cross-domain errors	Single or multiple experts per page
PRE–Propose	$(c, a_{\text{prev}}, \varnothing, \varnothing)$	Initial proposal a_0	Starts refinement loop	Traceable, domain-bounded proposal	Neural, symbolic, heuristic
PRE–Refine	$(c, a_{\text{prev}}, a_j, e_j)$	Refined a_{j+1}	Improves proposal using evaluator feedback	Corrects weak proposals	Any refinement depth k
Evaluators	Action a, context c	Score vector e	Assess orthogonal constraints	Independent signals; full traceability	Heuristic, learned, hybrid
Filtering	Candidates (a_j, e_j)	Filtered set F	Removes invalid / low-quality actions	Hard constraints enforced	Threshold tuning; fallback rules
Arbiter	Filtered set F	Final action a_{final}	Selects best action with variation control	Transparent scoring; bounded diversity	Score clusters; sampling rules
Ensemble	Context c	$\{a_1, \ldots, a_M\}$	Parallel independent reasoning	Plurality; robustness; ecology	With/without global arbiter

Table 13.1: Concise overview of the PRE–CAS modular reasoning architecture.

Architectural Insight

PRE–CAS is not a monolithic intelligence. It is a composition of minds: independent experts, evaluators, and arbiters whose coordinated interaction produces coherent reasoning.

Chapter 14

Toy Story: Generation Using PAGED-PRE-CAS

14.1 Introduction: Why Toy Story?

Toy Story is a miniature generative universe designed to make the behaviour of a modular reasoning system concrete, observable, and scientifically tractable. Where modern large language models rely on opaque high-dimensional transformations, Toy Story takes the opposite approach: it is deliberately small, modular, transparent, and fully interpretable. It is not a toy in the trivial sense, but a didactic laboratory—a controlled microcosm in which the dynamics of structure, constraint, and recursive improvement can be studied with unusual clarity.

The world is simple enough to expose every decision the system makes, yet rich enough to reveal how coherent actions emerge, how errors arise, and how iterative refinement shapes the expressive landscape of a population of models. Toy Story was built to answer a simple question: *What happens when we construct a generative system that reasons instead of samples?* The result is a model that not only produces thousands of coherent stories, but also reveals—in full detail—how those stories are constructed and how the system navigates its own space of possibilities.

This chapter introduces the Toy Story world and the mechanisms through which the system interacts with it. It provides the empirical foundation for the ecological and theoretical insights developed in the next chapter, where Toy Story becomes a microscope for studying collapse, attractors, diversity, and the dynamics that govern intelligent systems.

14.2 Tokenization and the World Space

Toy Story begins with a radically simple but structurally powerful tokenization scheme. Where modern language models rely on complex subword encodings such as BPE or SentencePiece, Toy Story uses a fully deterministic, curriculum-aligned vocabulary of only 144 tokens. This minimalism is not a limitation but a design choice: it creates a world that is small enough to be understood in full detail, yet rich enough to support hundreds of trillions of valid stories. The tokenization pipeline defines the ontology of the Toy Story universe and therefore determines what the model can express, what it can learn, and how its stories unfold.

At the core of this world lie three control tokens:

- **EMPTY** — a placeholder used to pad the context window,
- **EOL** — an explicit end-of-line marker,

- **EOS** — an explicit end-of-sequence marker.

These tokens provide the structural backbone of the system. They make line boundaries explicit, allow the context window to be represented in fixed position, and enable simple rules about timing, actor consistency, and line count to be enforced directly in the token stream. Unlike transformers, which must infer structure implicitly from patterns in data, Toy Story encodes structure explicitly.

All tokens are stored in a *TokenDictionary*, a transparent mapping between textual forms and integer identifiers. Token IDs are generated deterministically using a simple hashing function, ensuring that the vocabulary is stable across runs and independent of dataset order. Each token also carries a numerical suffix (0–4) that represents a stylistic dimension used throughout the Toy Story world.

The result is a compact but expressive world space:

- 144 total tokens,
- 34 possible token transitions observed in the dataset,
- 155 million curriculum stories,
- approximately 470 trillion valid generator stories.

This combination of simplicity and expressive power is central to Toy Story's purpose. Because the token space is small, every transition can be inspected, every rule can be understood, and every error can be traced. Because the world is large, the model can still exhibit meaningful variation, specialization, and emergent behaviour. Tokenization is therefore not merely a preprocessing step: it is the foundation of the Toy Story universe itself.

14.3 Embeddings and the Context Window

Toy Story represents language using an intentionally simple and fully interpretable embedding scheme. Instead of the high-dimensional learned embeddings used in modern transformers, Toy Story employs two compact representational layers—a one-dimensional input embedding and a small normalized output embedding—combined with a fixed-size sliding context window. This design makes the entire representational pipeline transparent, debuggable, and easy to inspect.

Input Embeddings: A Minimal Semantic Trace

Each token in the vocabulary is assigned a single scalar input embedding, generated deterministically using a Gaussian distribution seeded by the token's identifier. This one-dimensional representation is surprisingly effective in the constrained world of Toy Story: it provides enough variation for the system to learn local semantic continuity, while remaining simple enough to visualize and reason about directly.

The input embeddings populate the context window and serve as the primary representation of the story's history. Because the embedding space is one-dimensional, every feature of the representation can be examined without ambiguity.

Output Embeddings: Lightweight Semantic Geometry

In addition to the scalar input embedding, each token is assigned a small output embedding vector (e.g., 16 dimensions), also generated from a normalized Gaussian distribution. These vectors provide a compact semantic geometry that supports contrastive learning: the system learns which transitions are natural and which represent small structural or semantic deviations.

The Sliding Context Window: A Fixed-Position Working Memory

Toy Story maintains a sliding window of the last 35 tokens, forming the system's working memory. Unlike attention-based architectures, which dynamically compute relevance across the entire sequence, Toy Story uses a fixed-position representation: each position in the window corresponds to a specific slot in the input vector. This design has several advantages:

- **Transparency** — the full context is visible and interpretable at all times.
- **Determinism** — the same context always produces the same input encoding.
- **Trainability** — models see each structural feature in a consistent position.
- **Simplicity** — no attention mechanisms, no positional embeddings, no hidden state.

The window stores:

- the input embeddings of the last 35 tokens,
- the current line index and position index,
- the ID and output embedding of the most recent token.

EMPTY tokens fill unused positions at the beginning of generation, ensuring that the input vector always has the same dimensionality.

Why This Representation Works

Despite its simplicity, the embedding and context system is remarkably expressive. The evaluators rely on the context window to enforce actor consistency, timing structure, line boundaries, and stylistic constraints. The output embeddings provide a fine-grained semantic signal that helps distinguish natural transitions from micro-errors.

The result is a representational scheme that is:

- small enough to be fully understood,
- structured enough to support meaningful constraints,
- expressive enough to generate thousands of coherent stories,
- and stable enough to train efficiently on CPU.

Toy Story demonstrates that meaningful language generation does not require massive embeddings or attention mechanisms. It requires the right structure: a clear, interpretable working memory and a representational scheme that exposes its internal dynamics.

14.4 Paging: Decomposing the Vocabulary into Experts

Toy Story uses a simple but powerful paging mechanism to partition the vocabulary into a set of small, non-overlapping *pages*, each handled by its own expert network. Instead of training a single model to predict the entire vocabulary, the system assigns each token to exactly one page, transforming generation from a monolithic classification problem into a modular process in which each expert is responsible for a coherent semantic domain.

The motivation is straightforward: different kinds of tokens behave differently. Timing words, actors, objects, locations, and structural markers follow distinct transition patterns and obey different constraints. A single softmax over the entire vocabulary forces a model to learn all of these patterns simultaneously, increasing interference and making errors harder to diagnose. Paging avoids this problem by ensuring that each expert only needs to reason about a small, semantically consistent subset of the world.

Toy Story uses twelve pages in total: seven non-deterministic pages with their own neural networks, and five deterministic pages that contain only a single possible output token. The non-deterministic pages typically contain between 4 and 40 tokens, resulting in extremely small softmax layers that are easy to train, easy to debug, and resistant to noise. Deterministic pages require no model at all: when such a page is selected, the next token is known with certainty.

Constructing Pages from Successor Structure

Pages are constructed using an *overlap-minimizing* algorithm based on successor maps extracted from the curriculum. For each token, the system records all tokens that can legally follow it in the dataset. Tokens with similar successor sets are grouped together, ensuring that each page corresponds to a coherent semantic domain. The algorithm proceeds by:

1. ranking tokens by the size of their successor sets,
2. selecting the most complex token as a page root,
3. adding all tokens whose successors lie within the emerging page,
4. removing these tokens from the pool,
5. repeating until all tokens are assigned.

This process yields pages that reflect the underlying structure of the Toy Story universe. Timing words cluster together; actor tokens cluster together; object and location tokens form their own pages; and structural tokens such as EOL and EOS form deterministic pages. The resulting decomposition mirrors the semantic topology of the world.

Paging as a Structural Constraint

Paging is not merely an optimization; it also enforces strong structural constraints. By selecting the page based on the last token in the context window, the system determines *which expert is allowed to speak next.* This ensures that:

- timing words appear only where timing words are expected,
- actors appear only in actor positions,

- objects and locations appear only in semantically valid contexts,
- structural tokens appear only at line boundaries.

This eliminates entire classes of errors that transformer models routinely make. A page that contains only timing words cannot accidentally output an actor; a page that contains only objects cannot output EOS; a deterministic page cannot produce invalid structure. Paging therefore acts as a built-in correctness layer, keeping generation within the boundaries of the world's rules.

Why Paging Matters

Paging provides several key advantages:

- **Modularity** — each expert specializes in a narrow domain.
- **Interpretability** — errors can be traced to specific pages.
- **Stability** — small softmax layers converge quickly and reliably.
- **Scalability** — new domains can be added by introducing new pages.
- **Safety** — structural errors become impossible by construction.

In Toy Story, paging is the mechanism that keeps the world coherent. It ensures that each part of the vocabulary behaves according to its own rules, and that generation remains aligned with the structure of the universe.

14.5 PRE: Propose, Refine, Evaluate

Toy Story does not generate tokens in a single step. Instead, it uses a small multi-stage process in which the system proposes a candidate token, evaluates it, and then refines it through several iterations. This mechanism—PRE, for *Propose–Refine–Evaluate*—turns generation into a short deliberative loop rather than a one-shot prediction. The goal is simple: produce a token that fits the structure, timing, and semantics of the Toy Story world.

The Propose Step

In the propose phase, the active page-expert produces an initial candidate token based on the current context window. This first proposal is not taken as final. Instead, it is immediately checked by a set of evaluators that assess:

- semantic coherence,
- stylistic proximity,
- actor consistency,
- timing structure,
- line structure.

The result is a structured evaluation object that records both the token and its constraint profile.

The Refinement Steps

After the initial proposal, the system performs several refinement iterations. Each refinement step receives additional information: the previous proposal and its evaluator scores. This allows the expert to adjust its prediction based on explicit feedback, effectively learning to correct weak or borderline choices.

Each refinement produces a new candidate token, which is again evaluated by the full set of evaluators. All valid proposals across all refinement steps are retained, forming a small pool of filtered candidates. Refinement therefore expands the search space rather than overwriting earlier ideas.

Evaluation as a First-Class Signal

In Toy Story, evaluation happens before selection. The evaluators act as gatekeepers: they filter out invalid tokens, enforce structural rules, and provide continuous feedback during refinement. This ensures that:

- incoherent tokens are rejected immediately,
- structurally invalid tokens never enter the candidate pool,
- refinement is guided by explicit semantic and structural signals,
- the final choice is made from candidates that respect the world's rules.

Evaluation is therefore not a post-processing step but an integral part of the generation loop.

PRE as a Deliberative Process

PRE turns generation into a short sequence of reasoning steps:

1. propose an initial candidate,
2. evaluate it through multiple constraints,
3. refine based on explicit feedback,
4. accumulate all valid proposals,
5. select the final token from the candidate set.

This mirrors human reasoning: we generate an idea, check it, revise it, and consider alternatives before committing.

Why PRE Matters

PRE provides several advantages over single-pass generation:

- **Correctness** — invalid tokens are filtered before selection.
- **Robustness** — refinement corrects weak or borderline proposals.
- **Diversity** — multiple valid candidates survive to the final step.

- **Interpretability** — every proposal and refinement step is traceable.

In Toy Story, PRE is the mechanism that makes the system's behaviour visible. Every proposal, every correction, and every rejected candidate can be inspected, revealing how the model navigates its own space of possibilities.

14.6 Evaluators: The Mini-Experts

Toy Story enforces structure and semantics through a set of five evaluators, each responsible for a distinct dimension of correctness. These evaluators act as independent checks on every proposed token, ensuring that generation remains coherent, consistent, and aligned with the rules of the Toy Story universe.

Unlike transformer-based models, which rely on implicit statistical correlations to maintain structure, Toy Story makes structure explicit. Each evaluator encodes one aspect of the curriculum—style, identity, timing, line structure, or semantic continuity—and contributes its own judgment to the decision process.

The CAS Evaluator: Enforcing Stylistic Consistency

The CAS evaluator measures how closely a proposed token matches the target CAS value specified in the prompt. Each token carries a numerical suffix (0–4) representing its stylistic variant. The evaluator computes a score based on the distance between the token's CAS value and the target CAS, producing a continuous signal between 0 and 1.

This evaluator ensures that the story maintains a consistent stylistic tone and supports controlled stylistic variation across thousands of possible stories.

The Actor Evaluator: Maintaining Identity

The actor evaluator enforces identity consistency. It extracts the actor from the prompt (e.g., `alice`, `bob`, `carol`) and ensures that every story line refers to the same actor. If a proposed token introduces a different actor, the evaluator assigns a score of zero, effectively disqualifying the token.

This simple rule prevents one of the most common failure modes in generative models: unintended shifts in narrative focus.

The Timing Evaluator: Enforcing Narrative Order

Toy Story stories follow a strict temporal structure:

- line 1 begins with `first`,
- lines 2–4 begin with `then`,
- line 5 begins with `finally`.

The timing evaluator checks whether a proposed token respects this ordering. Any violation results in a score of zero. This ensures that the narrative unfolds in a logically ordered sequence and prevents temporally inconsistent stories.

The Line Evaluator: Enforcing Structural Form

The line evaluator enforces the structural constraints of the story format. It ensures that:

- each line ends with an `EOL` token,
- the story contains exactly five story lines,
- the `EOS` token appears only at the end,
- no double `EOL` tokens occur.

This evaluator guarantees that the generated story adheres to the formal structure defined by the curriculum.

The Coherence Evaluator: Learning Semantic Continuity

The coherence evaluator is the only learned evaluator. It is a small MLP trained using contrastive micro-errors: token swaps, incorrect transitions, and structural violations. The model outputs a continuous score between 0 and 1, representing how semantically coherent a proposed token is with respect to the current context window.

This evaluator captures local semantic relationships that are not explicitly encoded in the curriculum, such as natural ordering of actions or plausible transitions between objects and locations.

Orthogonality and Collective Judgment

Each evaluator examines a different dimension of correctness:

Evaluator	**Dimension**	**Type**
CAS	Style consistency	heuristic
Actor	Identity consistency	heuristic
Timing	Narrative order	heuristic
Line	Structural form	heuristic
Coherence	Semantic continuity	learned

Because the evaluators are orthogonal, their judgments combine into a multi-dimensional assessment of each proposed token. This modularity makes Toy Story fully interpretable: every decision can be traced back to the evaluator that influenced it.

Why Evaluators Matter

The evaluators are essential for keeping Toy Story's generation process aligned with the rules of the world. They provide:

- **hard constraints** that prevent invalid tokens from being considered,
- **soft signals** that guide refinement toward better proposals,
- **interpretability** by exposing the rationale behind each decision,
- **robustness** by isolating failure modes to specific evaluators,
- **modularity** by separating concerns across independent checks.

Together, the evaluators ensure that every generated story is structurally sound and semantically meaningful.

14.7 Token Filtering Before Selection

Toy Story introduces a distinctive mechanism for controlling generation: candidate tokens are filtered *before* any selection is made, using both logit ranking and evaluator constraints. Unlike transformer-based models, which rely on a probabilistic softmax distribution and apply heuristics after sampling, Toy Story performs deterministic logit ranking and applies structural and semantic filters prior to choosing a token. This ensures that only valid candidates ever enter the decision process.

The filtering mechanism is implemented by the *TokenSelector*, which acts as a gatekeeper between the page-expert's raw logits and the subsequent reasoning steps. The TokenSelector examines the ranked logits produced by the active page-expert and applies all evaluators to each candidate token. Only tokens that satisfy every constraint are retained.

Filtering via Ranked Logits

The filtering process proceeds in two stages:

1. **Identify the highest-logit valid token.** The TokenSelector iterates through the logits in descending order. The first token that passes all evaluator checks becomes the *anchor* candidate.

2. **Collect additional valid candidates using a score threshold.** Each candidate has an evaluator-based score

   $$s = \text{coherence} + \text{styleScore},$$

   where actor, timing, and line constraints must already be satisfied. A threshold is computed as

   $$s_{\min} = s_{\text{anchor}} \times \text{scoreThreshold}.$$

   Any subsequent token whose evaluator score satisfies

   $$s \geq s_{\min}$$

   and passes all hard constraints is added to the candidate set.

This produces a curated pool of candidates that are both structurally valid and semantically coherent. Tokens that violate actor consistency, timing rules, line structure, CAS style, or coherence are discarded immediately.

Why Filtering Before Selection Matters

Filtering before selection provides several advantages:

- **Structural safety.** Invalid tokens never enter the generation loop.
- **Semantic robustness.** The coherence evaluator rejects transitions that break local continuity.
- **Interpretability.** Every rejected token can be traced to the evaluator that disqualified it.
- **Controlled diversity.** Multiple high-quality candidates survive, enabling variation without chaos.

- **Deterministic behaviour.** The filtering process is fully reproducible and independent of sampling noise.

This approach stands in contrast to transformer-based generation, where the model must implicitly learn structural constraints and where sampling heuristics attempt to correct errors after the fact. Toy Story inverts this logic: constraints come first, logits second.

Filtering as a Cognitive Analogy

Token filtering is not merely an engineering technique; it also mirrors a familiar cognitive pattern: we discard impossible or illogical options before considering the remaining alternatives. In Toy Story, the evaluators act as fast, specialized critics that eliminate invalid proposals early, allowing the system to focus on meaningful choices.

By enforcing constraints at the earliest possible stage, Toy Story ensures that every step of generation remains within the boundaries of the world's rules.

A Foundation for Structured Generation

Filtering before selection ensures that:

- page-experts remain domain-specific,
- evaluators enforce orthogonal constraints,
- only valid proposals proceed to later steps,
- the candidate set remains small, clean, and interpretable.

Filtering transforms raw logits into meaningful proposals and keeps generation disciplined, rule-governed, and fully traceable.

14.8 The Arbiter: Scoring and Variation Injection

After the system has produced a curated set of valid candidate tokens—each filtered through the evaluators and refined across multiple iterations—the final decision is delegated to the *Arbiter*. The arbiter aggregates all candidate proposals, scores them using evaluator signals, and selects the next token in a way that balances correctness with controlled diversity.

Where the evaluators enforce structure and PRE generates alternatives, the arbiter acts as the decision-maker. Its role is not merely to choose the highest-scoring token, but to preserve variation among high-quality alternatives, preventing premature convergence and enabling expressive diversity within the structural boundaries of the Toy Story universe.

Aggregating Candidate Proposals

The arbiter receives a list of candidate outputs from the refinement loop. Each entry contains:

- the proposed token,
- the evaluator scores associated with that token,
- the logit value from the page-expert,

- metadata about the refinement step that produced it.

All candidates from the propose step and all refinement steps are pooled into a single list. Because invalid tokens have already been filtered out, the arbiter operates exclusively on structurally and semantically valid proposals.

Scoring Candidates

Each candidate is assigned a scalar score based on its evaluator signals. Hard constraints (actor, timing, line) must already be satisfied; the arbiter therefore focuses on the two continuous dimensions:

$$s = \text{coherence} + \text{styleScore}.$$

This score reflects both semantic continuity and stylistic alignment with the target CAS value. Candidates are sorted in descending order of s, producing a ranked list of high-quality alternatives.

Thresholding and Variation Injection

To avoid deterministic collapse into a single preferred token, the arbiter introduces controlled variation. After sorting, it computes a score threshold:

$$s_{\min} = s_{\text{best}} \times \text{scoreThreshold},$$

where s_{best} is the score of the top-ranked candidate. All candidates satisfying

$$s \geq s_{\min}$$

form the *top cluster*. If the cluster contains only one candidate, that token is selected deterministically. If multiple candidates qualify, the arbiter selects uniformly at random from the cluster.

This mechanism ensures:

- **quality** — only high-scoring candidates are eligible,
- **diversity** — multiple strong candidates can be chosen,
- **stability** — randomness is bounded by evaluator scores,
- **expressiveness** — the model avoids premature convergence.

The arbiter therefore injects variation without sacrificing structural correctness or semantic coherence.

A Cognitive Interpretation

The arbiter embodies a familiar cognitive principle: *deliberation among multiple good options.* Humans rarely choose the single best option in a purely deterministic way; we consider a small set of acceptable alternatives and select among them based on context, preference, or stochasticity. Toy Story mirrors this behaviour by:

- gathering all viable proposals,

- scoring them along meaningful dimensions,
- restricting attention to a high-quality subset,
- selecting one option with controlled randomness.

This makes the arbiter an essential part of Toy Story's decision process.

Why the Arbiter Matters

The arbiter is crucial for maintaining both correctness and diversity:

- It prevents the system from collapsing into deterministic, repetitive patterns.
- It ensures that variation arises only among structurally valid candidates.
- It integrates multiple signals into a single coherent decision.
- It provides a transparent, interpretable mechanism for token selection.

The arbiter transforms a set of filtered, refined proposals into a single next token, balancing precision with expressive flexibility.

14.9 The Ensemble: A Population of Models

Toy Story does not rely on a single generative model. Instead, it uses an *ensemble* of independently trained models, each exposed to a different randomized subset of the curriculum. In the standard configuration, the ensemble contains twenty models, but this number is arbitrary: any population size may be used. What matters is not the specific number, but the fact that Toy Story treats generation as the collective behaviour of a population rather than the output of a single monolithic network.

Each model in the ensemble shares the same structure—identical pages, identical refinement steps, identical evaluators—but differs in its training history. Because each model is trained on a different random ordering of a different subset of the dataset, the ensemble develops natural variation: differences in stylistic preference, transition likelihoods, and coherence judgments. This variation is essential for Toy Story's role as a reasoning laboratory, as it allows the system to exhibit diversity, specialization, and emergent population-level dynamics.

Story-Level Ensemble Operation

The ensemble operates at the *story level*, not the token level. For each prompt and target CAS value, every model in the population independently generates a complete story. There is no cross-model voting, averaging, or token-level blending. Instead:

1. each model receives the same prompt and CAS target,
2. each model runs its own generation process,
3. each model produces a full story or aborts if no valid continuation exists.

This design preserves the individuality of each model. The ensemble is therefore not a committee that negotiates token-by-token, but a population of independent agents that express their own interpretations of the same task.

Why a Population Matters

Using a population of models provides several advantages:

- **Diversity.** Different training subsets lead to different stylistic and structural preferences.
- **Robustness.** Failures in one model do not propagate to others.
- **Interpretability.** Variation across models reveals which behaviours are stable and which are contingent.
- **Ecological dynamics.** The ensemble forms a miniature cognitive ecosystem, enabling the study of collapse, attractors, and evolutionary behaviour in later chapters.
- **Scalability.** The population size can be increased or decreased without structural changes.

The ensemble therefore serves as a computational analogue of a biological or cognitive population: a set of agents with shared structure but different experiences.

Population-Level Behaviour

Across models, the ensemble exhibits population-level patterns that do not appear in any single model. Differences in training history lead to subtle specializations, biases, and stylistic tendencies. When viewed collectively, these differences form a small cognitive ecology in which variation, drift, and convergence can be observed directly.

Preparation for Collective Recursion

The ensemble is also the foundation for Toy Story's second training phase: *collective recursion*. Because each model generates its own stories, the ensemble produces a rich pool of candidate training data. In the next chapter, we examine how this population behaves when it is trained on its own output, revealing phenomena such as:

- rapid convergence,
- loss of diversity,
- collapse into shared attractors,
- and the effects of mutation on population dynamics.

The ensemble is therefore not merely a convenience; it is an empirical tool. It transforms Toy Story from a single-model generator into a full cognitive ecology, enabling the study of reasoning, structure, and evolution within a controlled miniature universe.

14.10 Collective Recursion (With and Without Mutation)

Toy Story's second training phase, *collective recursion*, transforms the ensemble from a set of independently trained models into a population capable of self-improvement. Where curriculum training teaches the basic structure, semantics, and stylistic rules of the Toy Story universe, collective recursion exposes the ensemble to its own generated stories, allowing it to refine its behaviour through iterative self-training.

Collective recursion is an optional mechanism that demonstrates how a population of models behaves when trained on its own output. This section describes the mechanics of recursion, while the next chapter analyzes its ecological consequences.

Overview of the Recursion Cycle

Each recursion epoch proceeds through the following steps:

1. **Generation.** Every model in the ensemble generates stories for all prompts and CAS values. Each model produces a full story or aborts if no valid continuation exists.

2. **Selection.** Only stories that pass full structural and semantic validation are retained. Invalid stories are discarded.

3. **Normalization and Tokenization.** The selected stories are normalized, converted to the canonical line format, and re-tokenized using the same deterministic tokenization pipeline.

4. **Dataset Construction.** The validated stories from all models are pooled into a new dataset, subject to a maximum size per epoch to prevent runaway overfitting.

5. **Retraining.** Each model in the ensemble is retrained on the new dataset, using the same training procedure as in the curriculum phase.

6. **Iteration.** The process repeats for multiple epochs.

This cycle allows the ensemble to refine its behaviour based on its own output, creating a closed-loop learning system.

Recursion Without Mutation

In the simplest form of collective recursion, the ensemble is trained exclusively on its own validated stories, without any artificial variation. This setup reveals how a population of models behaves when exposed to a fixed world and a fixed set of structural constraints.

Mechanically, recursion without mutation is straightforward:

- each model generates stories,
- only fully correct stories are retained,
- the retained stories form the next training dataset,
- all models are retrained on this dataset.

Because the dataset consists only of correct stories, the ensemble gradually converges toward the most stable and easily reproducible patterns in the world. The result is a rapid reduction in expressive diversity and a strong increase in correctness. This behaviour is analyzed in detail in the next chapter.

Recursion With Mutation

Toy Story also supports a variant of collective recursion that introduces controlled mutations into the generated stories before they are added to the training dataset. These mutations are *macro-level* transformations that preserve the structural validity of the story while injecting variation into the dataset.

The mutation operators include:

- **Actor swap** — replacing the actor with another valid actor token.
- **CAS mutation** — randomly altering the CAS suffix of several tokens.
- **Line reordering** — permuting the middle lines while preserving the `first/then/finally` timing structure.

These mutations are applied only to stories that are already structurally valid, ensuring that the resulting dataset remains safe for training. The purpose of mutation is not to break the rules of the world, but to introduce diversity into the training signal.

Mechanically, recursion with mutation follows the same steps as recursion without mutation, with the additional mutation stage inserted between selection and tokenization.

Why Collective Recursion Matters

Collective recursion serves several purposes within the Toy Story framework:

- **It demonstrates how a population behaves under self-training.**
- **It reveals the stability and fragility of structural constraints.**
- **It provides a controlled environment for studying convergence and collapse.**
- **It enables the introduction of artificial variation through mutation.**
- **It prepares the ground for ecological analysis in the next chapter.**

From a scientific perspective, collective recursion provides a miniature laboratory for studying the dynamics of populations of generative models and the forces that shape their evolution.

From Mechanics to Behaviour

The mechanics of collective recursion complete the description of how Toy Story models generate, validate, and refine their own data. We now turn from the mechanism itself to its consequences. When a population of models trains on its own output, new patterns emerge: convergence, loss of diversity, shared attractors, and the effects of mutation on variation. The remainder of this chapter examines these population-level behaviours in detail.

14.11 Summary: What Toy Story Reveals

Example Generated Story

Prompt: `Tell a story about Alice`

Generated Story:
First Alice meets child3.
Then Alice is in tower3.
Then Alice searches3 with lantern3.
Then Alice sees stone3.
Finally Alice plays with cat3.

Toy Story brings together a set of simple, transparent mechanisms into a fully interpretable generative system. Tokenization defines the ontology of the world; a fixed-position context window provides a visible working memory; lightweight embeddings encode the semantic geometry; paging decomposes the vocabulary into domain-specific experts; evaluators enforce structural, stylistic, and semantic constraints; refinement introduces deliberation; filtering removes invalid candidates; the arbiter balances correctness with controlled variation; and the ensemble introduces natural diversity through independent training histories.

Taken together, these components show that expressive generative behaviour does not require massive models or opaque representations. Toy Story demonstrates that a small, modular system with explicit structure can produce coherent, combinatorial stories while remaining fully interpretable. Every decision is traceable, every error diagnosable, and every mechanism visible.

14.12 Toy Story as a Microscope for Generative Behaviour

Toy Story is not merely a toy model; it is a scientific instrument. Its small size, full transparency, and modular structure turn it into a microscope for studying the fundamental forces that shape generative systems. Where large language models hide their internal dynamics behind billions of parameters and opaque attention patterns, Toy Story exposes every mechanism, every decision, and every failure mode.

Because Toy Story is compact and fully interpretable, it makes visible what large models obscure—how correctness interacts with diversity, how attractors form, how collapse emerges, and how variation sustains a population of generative agents.

Toy Story reveals three things with exceptional clarity:

- **Why collapse becomes visible.** In large models, collapse is buried under scale, noise, and sampling heuristics. In Toy Story, collapse is sharp, measurable, and inevitable.
- **Why attractors can be observed directly.** Transformers distribute their inductive biases across millions of parameters. Toy Story isolates them into explicit structural components, making attractors observable as behavioural phenomena rather than statistical artifacts.
- **Why modular systems reveal their own laws.** Because Toy Story is decomposed into explicit parts, each mechanism leaves a distinct signature on the system's behaviour. This makes it possible to study generative dynamics as an ecological process.

In this chapter, we now turn from mechanism to behaviour. We examine:

- convergence under self-training,
- collapse into monoculture,
- the narrowing of attractors,
- the role of mutation in sustaining diversity,
- the emergence of niches and specializations,
- and the population-level dynamics of the ensemble.

Toy Story thus becomes a controlled environment for uncovering the laws that govern generative systems. It reveals that collapse is not an accident but a structural force; that variation is essential; and that populations of small, transparent models can illuminate principles that remain hidden in large, monolithic architectures.

14.13 Curriculum Learning Results in the Toy Story Universe

To evaluate the effect of curriculum learning on a population of models with identical architecture but different world-experience, we generate stories for all possible prompts in the Toy Story universe. Each model is queried with 3 prompts for each of the 5 actors, and for target CAS values from 0.00 to 1.00 in steps of 0.01. This yields $15 \times 101 = 1515$ requested stories per model. For an ensemble of 20 models, the total evaluation set consists of 30 300 requested stories.

14.13.1 Effect of Curriculum Dataset Size

To study how curriculum dataset size influences correctness, diversity, and attractor formation, we train three ensembles of models on curriculum sets of size 1 000, 10 000, and 100 000 stories respectively. All models share the same architecture and training procedure; the only difference is the amount of world-experience they receive.

1 000-Story Curriculum

Training on only 1 000 stories yields weak models with highly unstable behavior. Across the 20 models, we observe:

- **Correctness range:** 0% to 47%.
- **Written stories:** 15% to 89%.
- **Unique stories:** 0 to 252 per model.
- **Combined unique stories:** 1 828.

The models frequently abort generation, fail to converge to stable attractors, and exhibit extremely low correctness. Diversity is limited because the world-experience is too small to form meaningful attractors.

10 000-Story Curriculum

Increasing the curriculum to 10 000 stories produces a dramatic improvement across all metrics:

- **Correctness range:** 7% to 93%.
- **Written stories:** 75% to 100%.
- **Unique stories:** 52 to 869 per model.
- **Combined unique stories:** 11 238.

The jump from 1k to 10k is substantial: correctness increases by a factor of 2–5, and diversity increases by nearly an order of magnitude. Models begin to form recognizable attractors, and world-views become more coherent.

100 000-Story Curriculum

Expanding the curriculum to 100 000 stories yields further improvements, but with much smaller relative gains compared to the 1k→10k jump:

- **Correctness range:** 32% to 97%.
- **Written stories:** 72% to 97%.
- **Unique stories:** 254 to 1 172 per model.
- **Combined unique stories:** 11 904.

The improvement from 10k to 100k is modest: correctness increases slightly, and diversity grows only marginally (from 11 238 to 11 904 unique stories). This indicates diminishing returns: once the world-experience is sufficiently rich, additional data refines attractors rather than creating new ones.

The spread is however substantial: the best models produce more than four times as many unique stories as the weakest ones, and correctness varies by a factor of three. This confirms that identical architectures trained on different slices of the same world converge to different attractors.

14.13.2 Effect of Refinement Steps

A core component of PRE (Proposal–Refinement–Evaluation) is the use of refinement steps. Each refinement step generates additional candidate continuations, allowing the arbiter to select the best token among all proposals and refinements. Importantly, even when later refinement steps no longer improve candidate quality, they do not harm performance: the arbiter always selects the best candidate across all steps, not merely the final refinement.

We evaluate the effect of using $k = 0$, $k = 1$, and $k = 2$ refinement steps. Previous results used $k = 4$, which performs slightly better than $k = 2$; we do not repeat those numbers here.

No Refinements (k = 0)

Without refinement, models rely solely on their initial proposal. This severely limits the arbiter's ability to correct weak or unstable trajectories.

- **Correctness range:** 0% to 98%.
- **Written stories:** 51% to 97%.
- **Unique stories:** 0 to 333 per model.
- **Combined unique stories:** 1 628.

The absence of refinement leads to unstable behavior: many models collapse to near zero correctness, and diversity is extremely low. The arbiter has too little choice to steer generation toward stable attractors.

One Refinement (k = 1)

Introducing a single refinement step dramatically improves both correctness and diversity. The arbiter now has multiple candidates to choose from, enabling recovery from weak proposals.

- **Correctness range:** 13% to 97%.
- **Written stories:** 67% to 98%.
- **Unique stories:** 121 to 1 063 per model.
- **Combined unique stories:** 9 183.

The improvement from $k = 0$ to $k = 1$ is substantial: correctness stabilizes, diversity increases by more than a factor of five, and attractors become much more consistent across models.

Two Refinements (k = 2)

Using two refinement steps yields further improvements, though the gains are smaller than the jump from $k = 0$ to $k = 1$. This indicates diminishing returns and shows that $k = 2$ is close to optimal.

- **Correctness range:** 27% to 97%.
- **Written stories:** 70% to 96%.
- **Unique stories:** 210 to 1 174 per model.
- **Combined unique stories:** 11 388.

The improvement over $k = 1$ is clear but modest. Most of the benefit of refinement comes from the first step; additional steps refine the attractor landscape rather than expanding it.

Refinement plays a central role in PRE: it stabilizes generation, increases the arbiter's decision power, and enables models to reach stronger attractors even with limited world-experience.

14.13.3 Effect of Evaluators

The PRE architecture relies on multiple evaluators to constrain generation and guide the arbiter toward coherent, stylistically aligned, and semantically valid stories. To understand their contribution, we disable specific evaluators and measure the resulting correctness, diversity, and stability across the 20-model ensemble.

Disabling the Coherence Evaluator

Removing the coherence evaluator increases creativity but reduces sentence-level consistency. The arbiter no longer penalizes incoherent continuations, which leads to a large increase in diversity but a noticeable drop in correctness.

- **Correctness range:** 7% to 91%.
- **Written stories:** 100% for all models.
- **Unique stories:** 90 to 1 249 per model.
- **Combined unique stories:** 11 553.

The extremely high uniqueness percentages (77%–94%) show that disabling coherence removes structural constraints, allowing models to explore a much larger portion of the manifold. However, many stories become grammatically or logically inconsistent, reducing correctness.

Disabling the CAS Style Evaluator

The CAS evaluator enforces the target style value. Disabling it removes stylistic pressure and therefore increases the number of unique stories substantially.

- **Correctness range:** 44% to 99%.
- **Written stories:** 78% to 100%.
- **Unique stories:** 490 to 1 365 per model.
- **Combined unique stories:** 16 965.

The jump in diversity (from 11 553 to 16 965 unique stories) confirms that CAS is a strong stylistic constraint. Removing it allows the model to generate a much wider variety of stories, but at the cost of ignoring the requested CAS target.

Disabling the Heuristic Semantic Evaluators

The three heuristic evaluators (actor, timing, lines) enforce basic semantic rules of the Toy Story universe. Disabling them has a drastic effect on quality: many stories violate fundamental world constraints and are therefore invalid.

- **Correctness range:** 7% to 52%.
- **Written stories:** 69% to 95%.
- **Unique stories:** 77 to 638 per model.

- **Combined unique stories:** 5 915.

Correctness collapses because the model frequently produces stories with wrong sentences. Diversity also decreases because many invalid stories are filtered out by the arbiter.

14.13.4 Effect of Score Threshold

The score threshold determines how strict the arbiter is when selecting candidate tokens. A high threshold filters aggressively and prioritizes correctness, while a lower threshold allows more exploratory candidates to pass through, increasing diversity at the cost of correctness. This trade-off becomes especially important later in the analysis of collective recursion.

We compare two settings: a very conservative threshold of 0.98 and a very permissive threshold of 0.6.

Very Conservative Threshold (0.98)

A threshold of 0.98 accepts only the highest-scoring candidates. This produces very stable attractors and high correctness, but severely restricts diversity.

- **Correctness range:** 21% to 98%.
- **Written stories:** 72% to 97%.
- **Unique stories:** 107 to 781 per model.
- **Combined unique stories:** 6 897.

The extremely low diversity shows that the manifold collapses under strong filtering. Most alternative continuations are rejected, and models converge to narrow attractors.

Very Permissive Threshold (0.6)

Lowering the threshold to 0.6 allows more candidate tokens to pass the arbiter. This increases diversity dramatically, but reduces correctness.

- **Correctness range:** 39% to 93%.
- **Written stories:** 81% to 96%.
- **Unique stories:** 510 to 1 345 per model.
- **Combined unique stories:** 16 576.

The uniqueness percentages approach 100% for nearly all models, showing that the manifold becomes much broader when the arbiter is less strict. However, correctness drops because many lower-quality candidates are now accepted.

14.14 Collective Recursion Training

Collective recursion is a population-based training method in which all models in the ensemble generate stories each epoch, and all unique stories produced by the ensemble are added to the training set for the next epoch. Each model is retrained on the combined set of unique stories. This process is repeated for 15 epochs.

Unless stated otherwise, a score threshold of 0.95 is used during generation. We evaluate two settings: collective recursion *without* mutation and collective recursion *with* mutation. These experiments form the empirical foundation for the dataset scaling law introduced in the next section.

14.14.1 Collective Recursion Without Mutation

Without mutation, collective recursion rapidly increases correctness but causes a steep collapse in diversity. The number of unique stories generated per epoch drops dramatically:

$$11822 \to 8558 \to 3507 \to 2077 \to 1230 \to 972 \to 826 \to 755 \to 683 \to 628 \to 581 \to 534 \to 534 \to 554 \to 534$$

At test time, only 550 unique stories remain across all 20 models.

- **Correctness range:** 87% to 100%.
- **Written stories:** 88% to 97%.
- **Unique stories per model:** 113 to 154.
- **Combined unique stories:** 550.

Correctness becomes extremely high, but diversity collapses by more than a factor of 20. All models converge to the same narrow attractor set, despite starting from different world-views.

14.14.2 Collective Recursion With Mutation

Adding mutation slows the collapse of diversity and allows correctness to reach 100% for all models. However, diversity still declines sharply, though less catastrophically than without mutation.

Unique stories per epoch evolve as:

$$11822 \to 9044 \to 3833 \to 3516 \to 3095 \to 2549 \to 2117 \to 2030 \to 1934 \to 1938 \to 2135 \to 2036 \to 2712 \to 2888 \to 2422$$

At test time, 2430 unique stories remain.

- **Correctness range:** 99.7% to 100%.
- **Written stories:** 92% to 98%.
- **Unique stories per model:** 176 to 289.
- **Combined unique stories:** 2430.

Mutation prevents total collapse and maintains a broader manifold, but the attractor set still shrinks significantly. Collective recursion acts as a strong attractor amplifier: correctness increases, diversity decreases.

14.14.3 Effect of Lowering the Threshold After Recursion

After collective recursion, models become more precise and logits sharpen. This allows the score threshold to be lowered without collapsing correctness. We compare the default threshold of 0.95 with a permissive threshold of 0.60.

Threshold 0.60 Without Mutation

Lowering the threshold increases diversity but correctness remains high due to the strong attractors formed during recursion.

- **Correctness range:** 89% to 99%.
- **Unique stories per model:** 282 to 367.
- **Combined unique stories:** 1 408.

Diversity increases compared to the 550 stories at threshold 0.95, but remains far below pre-recursion levels.

Threshold 0.60 With Mutation

With mutation, lowering the threshold produces a dramatic increase in diversity while maintaining 100% correctness.

- **Correctness:** 100% for all models.
- **Unique stories per model:** 579 to 1 037.
- **Combined unique stories:** 9 698.

This is a factor of 4 increase compared to threshold 0.95 with mutation (2 430 stories), and a factor of 18 compared to threshold 0.95 without mutation (550 stories). Mutation keeps the manifold open, and the lower threshold allows this diversity to surface.

14.14.4 Interpretation

Collective recursion has three consistent effects:

1. **Correctness increases rapidly.** Recursion amplifies attractors and sharpens logits.
2. **Diversity collapses.** Without mutation, the collapse is extreme; with mutation, it is slower but still substantial.
3. **Threshold sensitivity changes.** After recursion, models are precise enough to tolerate lower thresholds without losing correctness. This effect is only meaningful when mutation preserves manifold width.

These results demonstrate a fundamental principle: identical world-experience forces models toward the same attractors. Mutation or different world-experience is required to maintain diversity. Collective recursion is therefore a powerful mechanism for increasing correctness, but it inevitably reduces diversity unless counterbalanced by mutation.

This dynamic forms the empirical basis of the dataset scaling law introduced in the next section.

14.15 The Triadic Cosmos Dataset Scaling Law

The collective recursion experiments reveal a structural regularity that extends far beyond the Toy Story universe. It applies to any population of generative models that share an architecture and learn from the same world. This regularity can be formulated as a general law: the Dataset Scaling Law for populations of reasoning systems.

The Triadic Cosmos Dataset Scaling Law

Full Formulation
For a population of individuals with similar architecture and identical world-experience, there exists a finite set of stable attractors. With sufficient training, the entire population converges to this attractor set, regardless of initial variation, mutation, or threshold settings.

Compact Form

$$\text{Same world} \rightarrow \text{same attractors} \rightarrow \text{same models.}$$

Variation requires different world experience.

Interpretation

The Dataset Scaling Law states that collapse is not a quirk of small models, nor an artifact of toy experiments, but a structural property of generative systems. Whenever multiple models are trained on the same dataset, under the same constraints, they are funneled toward the same stable patterns of behavior.

The collective recursion results make this collapse visible with unprecedented clarity:

- **Without mutation, diversity collapses by a factor of 20.** The ensemble goes from 11 822 unique stories to only 534 after 15 epochs.
- **With mutation, collapse is slower but still inevitable.** Diversity drops from 11 822 to 2 430 unique stories.
- **Correctness reaches 100% in both cases.** Collective recursion acts as a correctness amplifier and a diversity suppressor.
- **Lower thresholds only help after recursion.** Before recursion: lowering the threshold destroys correctness. After recursion: correctness remains 100% and diversity increases (up to 9 698 stories with mutation).
- **Mutation helps, but cannot defeat the world.** It slows collapse and widens the manifold, but the attractor landscape is determined by the underlying world, not by the mutation mechanism.

The law therefore states that:

$$\text{World-experience determines the attractor set.}$$

Architectures, thresholds, refinements, and evaluators only determine *how fast* the population converges, not *where* it converges.

Why the Law Matters

The Dataset Scaling Law reframes generative AI as an ecological system. It shows that diversity cannot be maintained by architecture alone; it requires exposure to different worlds, different datasets, or different constraints.

The Toy Story experiments make this visible:

- Curriculum learning shows that world-size determines attractor richness.
- Evaluators show that constraints shape attractor geometry.
- Refinement shows that search amplifies attractor strength.
- Thresholds show that correctness and diversity trade off.
- Collective recursion shows that shared world-experience forces collapse.
- Mutation shows that variation slows collapse but cannot prevent it.

Toy Story is small, modular, and transparent — but the law itself is universal.

Do Large LLMs Converge Toward Each Other?

This is not a public claim, but a direct consequence of mainstream ML assumptions.
Modern LLMs are trained on:

- highly overlapping world experience (web-scale text),
- similar Transformer architectures,
- similar objectives (next-token prediction + instruction tuning),
- similar optimization and alignment pipelines.

Under these conditions, the Dataset Scaling Law predicts that *different LLMs converge toward the same attractor landscape of behaviors and internal representations.*
Apparent differences arise mostly from sampling noise, alignment layers, and surface-level tuning—not from fundamentally different attractors.

14.16 Conclusion: The Lesson of Toy Story

Toy Story began as a miniature universe — a controlled environment for studying the mechanics of modular reasoning. But through recursion, collapse, attractors, and mutation, it revealed something far deeper: the fundamental ecological forces that govern all generative systems.

The Dataset Scaling Law shows that collapse is not a failure mode but a natural law. A population trained on a single world converges toward a single mind. Correctness narrows, filtering sharpens, and attractors dominate. Variation slows the descent but cannot overturn the gravitational pull of shared experience.

This insight reframes the entire landscape of generative AI. It explains why large LLMs increasingly resemble one another, why scale alone cannot produce diversity, and why pluralistic architectures are essential for robust intelligence.

Toy Story makes these forces visible. The Triadic Cosmos formulates them as general laws. Together, they point toward a new paradigm: intelligence as ecology, not machinery.

One-Line Summary

A single world creates a single mind — only plurality keeps intelligence alive.

Chapter 15

Domain-Bound AGI: A Composite Reasoning Architecture

15.1 Introduction

Artificial intelligence has advanced rapidly through the development of large transformer-based language models (LLMs), which demonstrate remarkable capabilities in pattern recognition, linguistic generalization, and zero-shot adaptation. These models compress vast statistical regularities into a single parameter space, enabling broad competence across many tasks. Yet their monolithic structure imposes fundamental limitations: they lack explicit mechanisms for branching search, structured memory, multi-step control flow, and interpretable reasoning. As a result, they struggle with tasks that require deliberate planning, symbolic manipulation, or the exploration of alternative futures.

This chapter introduces a compositional alternative: a *domain-bound AGI architecture* that separates domain-agnostic reasoning from domain-specific computation. Instead of relying on a single model to internalize all cognitive functions, the architecture decomposes intelligence into interacting modules: a reasoning language model implemented using the Paged–PRE–CAS framework, a machine-native script engine, an object-referenced memory map, and a collection of expert modules that provide domain-specific or algorithmic capabilities. Deep combinatorial exploration is delegated to explicit search engines such as Monte Carlo simulation or Beam Search, which operate independently of the reasoning model.

A central innovation of this architecture is the treatment of *context* as a first-class computational object. Contexts are stored in a persistent memory map under unique identifiers, enabling the system to maintain multiple concurrent reasoning flows, spawn hierarchical sub-contexts, and resume or combine them as needed. This stands in contrast to transformer models, which encode all state implicitly in a growing token sequence.

Equally important is the mechanism of *callback-driven self-prompting*. Experts do not merely return data; they generate structured prompts that are routed back to the reasoning model through context identifiers. This creates a closed-loop cognitive process in which reasoning, memory, and computation interact autonomously, without requiring synchronous API calls or external orchestration.

The architecture is not positioned as a rejection of transformer models. Instead, it *generalizes* them. By embedding a transformer-based reasoning model inside a modular cognitive system—equipped with explicit memory, algorithmic search, and procedural control—the model acquires reasoning capabilities that cannot emerge from scale alone. The relationship is bidirectional:

the architecture becomes more powerful through the transformer, and the transformer becomes more capable through the architecture. This two-directional enhancement provides a practical path toward hybrid systems that combine statistical fluency with structured, interpretable reasoning.

Roadmap of the Chapter

To guide the reader, the chapter proceeds as follows:

- **Core Principles** introduce the conceptual foundations: compositionality, hybrid intelligence, and the Society of Mind perspective.
- **Architecture Overview** describes the system's main components: the reasoning core, script engine, memory map, and expert modules.
- **Depth Search and Callback Dynamics** explain how explicit search engines and callback-driven continuation enable multi-step, distributed reasoning.
- **Hierarchical and Asynchronous Contexts** show how the system supports parallel reasoning flows and long-lived cognitive processes.
- **Self-Learning Without Weight Updates** presents the mechanism through which the system expands its capabilities procedurally rather than parametrically.
- **Comparison With Monolithic Transformers** synthesizes the architectural contrast and clarifies the bidirectional benefits.
- **Domain-Bound AGI as a Recipe** distills the architecture into a reusable pattern for constructing new domains.
- **Learned Cognitive Mechanisms** detail the internal reasoning behaviors—canonical rewriting, variable binding, flow-based attention, and repeat–prompt recursion—that provide coherence and stability.

From Theory to Practice: The Toy Board Demonstration

While this chapter develops the architecture conceptually, the next chapter (*The Toy Board*) provides a concrete operational demonstration. There, we instantiate the architecture in multiple miniature domains—sorting, planning, simulation, and game-like environments—to show how contexts, experts, search, and callback-driven reasoning behave in practice. The Toy Board serves as a didactic bridge between the abstract blueprint presented here and its real-world implementation, illustrating how domain-bound AGI systems can be constructed step by step.

Together, this chapter and the next form a complete introduction to the architecture: first the conceptual foundations, then the operational realization.

15.2 Core Principles

The domain-bound AGI architecture is grounded in three foundational principles: *compositionality*, *hybrid intelligence*, and the *Society of Mind* perspective. Together, these principles provide the conceptual scaffolding for a cognitive system that is modular, interpretable, and capable of general-purpose reasoning without relying on monolithic parameter updates. They also illuminate how transformer models can be embedded within a broader cognitive structure, gaining new capabilities through architectural context rather than scale alone.

15.2.1 Compositionality

Compositionality is the idea that complex cognitive behavior emerges from the interaction of simpler components. Rather than embedding all reasoning, knowledge, and search capabilities inside a single model, the architecture decomposes intelligence into modular units: a reasoning engine, a script interface, an object-referenced memory map, and a collection of expert modules that implement domain-specific or algorithmic operations.

This modular structure offers several advantages. It enables the system to extend itself by adding new experts without retraining the reasoning model. It supports transparency, since each component can be inspected independently. And it allows the system to adapt to new domains by composing existing capabilities in novel ways. Compositionality thus serves as both an engineering principle and a cognitive principle: complex reasoning arises from the structured interaction of simpler processes.

Importantly, compositionality also benefits transformer models. When embedded within a modular architecture, a transformer no longer needs to internalize all domain logic or simulate search through statistical continuation. Instead, it can specialize in what it does best—pattern recognition, semantic compression, and prompt rewriting—while relying on explicit modules for memory, search, and control flow.

15.2.2 Hybrid Intelligence

Hybrid intelligence refers to the integration of multiple forms of computation within a single cognitive system. In this architecture, symbolic reasoning, statistical inference, and algorithmic search coexist and reinforce one another. The reasoning model provides high-level planning and metacognitive control; expert modules supply domain-specific transformations and evaluations; and depth-search engines such as Monte Carlo simulation or Beam Search provide the computational substrate for exploring large decision spaces.

This division of labor mirrors human cognition, which combines intuitive pattern recognition with deliberate reasoning and procedural knowledge. It also addresses the limitations of monolithic transformer models: while transformers excel at linguistic generalization, they struggle with deep combinatorial reasoning. By delegating search and domain-specific computation to explicit modules, the architecture achieves a form of intelligence that is both more powerful and more interpretable than what either component could provide alone.

At the same time, hybrid intelligence enhances the transformer itself. Freed from the burden of simulating search or maintaining long-term state, the model can focus on learned cognitive mechanisms such as canonical rewriting, variable binding, and flow-based attention—capabilities that emerge naturally when the model operates within a structured cognitive environment.

15.2.3 Society of Mind

The Society of Mind perspective views intelligence as the emergent result of many small processes interacting in a coordinated fashion. This architecture operationalizes that idea by treating experts as autonomous cognitive agents that contribute to the reasoning process. Experts do not merely return data; they generate structured callback prompts that are routed back to the reasoning model through context identifiers. This callback-driven mechanism enables a form of self-prompting in which the system continues reasoning autonomously, without requiring external intervention.

Contexts themselves are first-class objects in the memory map, allowing the system to maintain multiple concurrent reasoning flows and to spawn hierarchical sub-contexts. This structure supports

parallelism, task decomposition, and multi-step reasoning in a way that is difficult to achieve with purely token-based models.

The Society of Mind principle also clarifies the role of the transformer within the architecture. Rather than functioning as a monolithic intelligence, the transformer becomes one agent among many—responsible for rewriting, planning, and integrating information, while relying on other agents for computation, memory, and search. This repositioning transforms the transformer from a general-purpose approximator into a specialized cognitive subsystem embedded within a larger, more powerful whole.

Together, these principles form the conceptual foundation of the domain-bound AGI architecture. They enable a system that is modular, transparent, extensible, and capable of structured reasoning across a wide range of domains, while simultaneously offering a path for transformer models to evolve into more powerful and cognitively grounded components.

15.3 Architecture Overview

The domain-bound AGI architecture is a modular cognitive system composed of several independent subsystems that interact through explicit, structured interfaces. Only one of these subsystems—the reasoning language model—uses the Paged–PRE–CAS framework. All other components, including search engines, parsers, evaluators, and domain-specific tools, operate independently and communicate with the reasoning model through a machine-native scripting interface. This separation ensures that the reasoning model remains domain-agnostic, while the system as a whole can incorporate specialized computational capabilities without modifying the model's parameters.

The architecture is designed not only to overcome the limitations of monolithic transformer models, but also to enhance them. By embedding a transformer-based reasoning core within a structured cognitive environment—equipped with explicit memory, algorithmic search, and modular experts—the transformer gains access to capabilities that cannot emerge from next-token prediction alone. The result is a hybrid system in which statistical fluency and symbolic structure reinforce one another.

15.3.1 Reasoning Language Model (Paged–PRE–CAS)

The reasoning component of the architecture is implemented as a Paged–PRE–CAS model. Its role is not to perform domain-specific computation, but to orchestrate the cognitive process. It is responsible for:

- decomposing tasks into smaller steps,
- generating structured instructions for experts,
- interpreting expert feedback,
- synthesizing partial results,
- and determining when a reasoning flow is complete.

Paged token partitioning provides structural constraints and computational efficiency; PRE (Propose–Refine) enables local improvement cycles; and CAS vectors (optional) provide alignment or evaluation signals. Crucially, the reasoning model contains *no domain-specific knowledge*. All domain operations—parsing, evaluation, simulation, search—are delegated to external experts.

This design repositions the transformer from a monolithic problem solver to a specialized cognitive subsystem. Freed from the burden of simulating search or maintaining long-term state, the model can focus on learned cognitive mechanisms such as canonical rewriting, variable binding, and flow-based attention. These mechanisms emerge naturally when the model operates within a structured cognitive architecture.

15.3.2 Script Engine

The script engine is the communication layer that connects the reasoning model to the rest of the system. It defines a minimal, machine-native instruction language for:

- invoking experts,
- creating or updating objects in memory,
- linking contexts,
- and routing callback prompts.

All interactions between components are explicit, deterministic, and auditable. The script engine ensures that the reasoning model never interacts with raw domain logic or unstructured data. Instead, it issues high-level instructions such as "simulate," "evaluate," or "plan," which the script engine binds to the appropriate expert or search module.

This separation of concerns enables extensibility: new experts can be added without modifying the reasoning model, and new search engines can be integrated without retraining. It also enhances transparency and governance, since every operation is represented as an explicit script instruction rather than an implicit activation pattern inside a neural network.

15.3.3 Object-Referenced Memory Map

All cognitive entities are stored in an object-referenced memory map. Each entity—whether a domain object, intermediate result, or reasoning context—is assigned a unique identifier. This provides stable references that persist across reasoning steps and can be accessed or updated by experts.

A key innovation is that *contexts* themselves are objects in memory. A context represents an ongoing reasoning flow and may contain:

- references to domain objects,
- partial computations or intermediate results,
- links to sub-contexts,
- pending tasks or goals,
- and metadata relevant to the reasoning process.

Because contexts are stored as structured objects rather than text, they persist independently of token windows and can be manipulated by experts without requiring the reasoning model to reconstruct state from a growing prompt. This enables hierarchical reasoning, parallel flows, and long-lived cognitive processes that are not constrained by sequence length.

The memory map also provides a foundation for explicit governance and interpretability. Every state transition is visible, inspectable, and traceable—properties that are difficult to achieve in monolithic transformer architectures, where state is encoded implicitly in high-dimensional activations.

Together, the reasoning model, script engine, and memory map form the core of the architecture. They establish a structured cognitive environment in which transformer-based reasoning, algorithmic search, and domain-specific expertise can interact in a modular and interpretable manner.

15.3.4 Expert Modules

Expert modules are independent computational components that implement domain-specific or algorithmic capabilities. They operate outside the Paged–PRE–CAS reasoning core and do not participate in its token-based reasoning dynamics. Instead, they receive structured instructions from the script engine and return results through callback prompts that reference the appropriate context.

Experts may include:

- Monte Carlo and Beam Search engines for depth exploration,
- heuristic evaluators and scoring functions,
- rule-based or grammar-based parsers,
- constraint solvers and optimization routines,
- simulation engines for dynamic environments,
- domain-specific transformation or validation tools,
- or even specialized language models trained for a particular domain.

Although expert modules are *not required* to use the Paged–PRE–CAS framework, they *may* be implemented as Paged–PRE–CAS models when this is advantageous. For example, a domain-specific expert—such as a chemistry parser, a legal reasoning assistant, or a symbolic algebra engine—can itself be a small transformer trained on domain data. In such cases, the expert behaves as a specialized language model embedded within the broader architecture, while the central reasoning model remains domain-agnostic.

This flexibility ensures that the architecture can integrate a wide spectrum of computational tools: from deterministic algorithms and simulators to neural models and hybrid systems. Regardless of their internal implementation, all experts share the same interface: they operate on structured objects stored in the memory map and communicate exclusively through explicit script instructions and callback prompts.

The modularity of expert modules offers several advantages. New experts can be added without modifying or retraining the reasoning model. Existing experts can be replaced, extended, or audited independently. This supports transparency, safety, and governance: each expert is an isolated, inspectable component whose behavior is fully explicit.

Importantly, expert modules also enhance the transformer-based reasoning core. By delegating domain-specific computation to experts—whether algorithmic or neural—the reasoning model is freed from the burden of simulating algorithms through statistical continuation. This allows the

transformer to focus on higher-level cognitive functions such as canonical rewriting, variable binding, and flow control. In this way, the architecture not only overcomes the limitations of monolithic transformers but also provides a structured environment in which transformer-based experts can operate more effectively.

Expert modules therefore serve as the computational backbone of the domain-bound AGI architecture. They provide the concrete operations that the reasoning model orchestrates, enabling the system to perform complex, multi-step tasks across diverse domains without embedding domain logic inside a single monolithic model.

15.3.5 Generic Depth Search

A core component of the architecture is a domain-agnostic depth-search mechanism that provides the computational substrate for exploring large decision spaces. Transformer models excel at pattern recognition but lack explicit mechanisms for branching, simulation, or evaluating alternative futures. Depth search fills this gap by supplying a structured exploration process that the reasoning model can invoke whenever a task requires multi-step evaluation or combinatorial analysis.

The architecture treats the choice of search method as a *domain-level property*. Each domain specifies the search strategy that best matches its structural characteristics, allowing the script engine to automatically route search requests to the appropriate module. This ensures that the reasoning model remains entirely domain-agnostic: it issues abstract instructions such as "simulate" or "plan," while the script engine binds these instructions to the domain's preferred search engine.

Monte Carlo simulation is well suited for domains with large branching factors and shallow evaluation horizons. It provides a stochastic estimate of the value of a single action by sampling one-step or short-horizon futures. This makes it effective for local tactical exploration, where the goal is to evaluate immediate consequences rather than construct long-term plans.

Beam Search, by contrast, supports *multi-step planning*. It maintains a frontier of the most promising partial plans, guided by a simple domain heuristic. This enables deeper exploration of action sequences while pruning unpromising branches. Domains with structured combinatorial spaces—such as sorting, constraint puzzles, or symbolic workflows—benefit from Beam Search because it can construct multi-step plans that Monte Carlo cannot reliably discover.

Treating search as a modular component yields several advantages:

- **Flexibility**: domains can express their preferred planning style.
- **Transparency**: search statistics and traces are explicit and auditable.
- **Replaceability**: search engines can be swapped or extended without retraining.
- **Scalability**: deep reasoning is offloaded to specialized modules.

This separation of concerns allows the architecture to combine statistical reasoning with explicit algorithmic search—a capability that pure transformer models lack. At the same time, depth search enhances the transformer-based reasoning core: by delegating combinatorial exploration to external modules, the transformer is freed from the impossible task of simulating search through next-token prediction. This enables the reasoning model to focus on higher-level cognitive functions such as canonical rewriting, variable binding, and flow control.

Generic depth search therefore acts as a universal computational substrate for the architecture. It provides the raw exploration power that complements the reasoning model's ability to structure tasks, interpret results, and orchestrate multi-step workflows across diverse domains.

15.3.6 Callback-Driven Self-Prompting

A defining feature of the architecture is its callback-driven mechanism for autonomous continuation of reasoning. Unlike monolithic transformer systems, which operate as single-pass sequence generators, the domain-bound AGI architecture supports multi-step workflows in which experts actively participate in the control flow. Experts do not simply return raw data; they produce structured callback prompts that instruct the reasoning model on how to proceed. This transforms the system from a passive text generator into an active, event-driven cognitive process.

When an expert completes an operation, it generates a callback containing:

- the identifier of the context to update,
- the result of the computation,
- and a structured instruction describing the next reasoning step.

The reasoning model processes the callback, updates the relevant context in the memory map, and issues new script instructions. This creates a closed-loop interaction pattern:

$$\text{Reasoning Model} \rightarrow \text{Expert Instruction} \rightarrow \text{Expert Execution} \rightarrow \text{Callback Prompt} \rightarrow \text{Reasoning Model}.$$

This loop continues until the reasoning model emits a termination signal. The architecture therefore supports long-lived, multi-step reasoning flows that are not constrained by token windows or sequential inference.

Callback-driven self-prompting provides several advantages:

- **Autonomy**: the system continues reasoning without external orchestration or synchronous API calls.
- **Hierarchical control**: complex tasks can be decomposed into sub-contexts, each driven by its own callback chain.
- **Parallelism**: multiple experts can operate on different contexts simultaneously, producing callbacks in any order.
- **Transparency**: every reasoning step is explicit and auditable.

This mechanism also enhances the transformer-based reasoning model. In monolithic architectures, transformers must implicitly simulate control flow through token continuation, which leads to drift, instability, and brittle long-range reasoning. In contrast, callback-driven self-prompting provides an external control structure that the transformer can rely on. The model no longer needs to remember its own output or maintain state through a growing prompt; instead, it receives structured callbacks that anchor each reasoning step in a stable context.

By embedding the transformer within a callback-driven architecture, the system achieves a form of procedural intelligence that cannot emerge from next-token prediction alone. The transformer becomes a cognitive controller that advances the reasoning process step by step, guided by explicit signals from experts and structured memory. This bidirectional interaction—experts driving callbacks, the transformer integrating them—forms the backbone of the architecture's multi-step reasoning capabilities.

15.4 Hierarchical Contexts, Parallel Reasoning, and Asynchronous Processing

A central innovation of the architecture is the treatment of *contexts* as first-class computational objects stored in the memory map. Unlike transformer-based systems, which rely on a single linear prompt as their working memory, the architecture supports multiple concurrent reasoning flows, each represented by a structured context object with a unique identifier. This enables hierarchical reasoning, parallel task execution, asynchronous processing, and long-lived cognitive processes that are not constrained by token windows or sequential inference.

15.4.1 Contexts as First-Class Cognitive Objects

A context is an explicit object in the memory map that encapsulates the state of an ongoing reasoning process. It may contain:

- references to domain objects,
- partial computations or intermediate results,
- links to sub-contexts,
- pending tasks or goals,
- and metadata relevant to the reasoning flow.

Because contexts are stored as structured objects rather than text, they persist across reasoning steps and can be accessed or updated by experts without requiring the reasoning model to reconstruct state from a growing prompt. This provides a stable foundation for multi-step reasoning and eliminates the drift and fragility associated with token-based memory.

15.4.2 Hierarchical Context Trees

Contexts may reference other contexts, forming a tree or graph structure. This enables the system to decompose complex tasks into smaller sub-tasks, each handled by its own context. For example:

- a top-level context may represent a global planning task,
- child contexts may represent subtasks such as parsing, evaluation, or simulation,
- and leaf contexts may represent atomic operations delegated to experts.

The reasoning model can spawn new contexts, update existing ones, or merge results from multiple sub-contexts. This hierarchical structure supports recursive reasoning patterns and multi-step workflows that are difficult to achieve with monolithic transformer architectures.

15.4.3 Parallel Reasoning Flows

Because contexts are independent objects, multiple reasoning flows can proceed in parallel. The architecture does not enforce a single-threaded execution model. Instead:

- experts may operate on different contexts simultaneously,

- callback prompts may arrive in any order,
- and the reasoning model may resume or suspend contexts as needed.

Parallelism emerges naturally from the independence of context objects and the callback-driven control mechanism. This allows the system to explore multiple hypotheses, evaluate alternatives, or perform background computations without blocking the main reasoning flow.

15.4.4 Asynchronous Processing

A key advantage of the architecture is its support for *asynchronous* processing. Expert modules operate independently and may complete their work at different times. When an expert finishes, it does not interrupt the reasoning model or require synchronous waiting. Instead, it emits a callback prompt that is routed to the appropriate context.

This enables several powerful behaviors:

- **Non-blocking execution**: the reasoning model can continue working on other contexts while experts perform long-running computations.
- **Out-of-order completion**: callbacks may arrive in any order, and the reasoning model integrates them as they appear.
- **Dynamic task spawning**: experts may create new child contexts that perform additional reasoning or computation asynchronously.
- **Event-driven cognition**: reasoning progresses in response to completed tasks rather than a fixed sequence of steps.

Asynchronous processing transforms the architecture into a distributed cognitive system. The reasoning model acts as a central controller that integrates results, while experts function as autonomous workers that contribute to the overall reasoning process at their own pace.

15.4.5 Callback-Driven Continuation

The callback mechanism ties hierarchical contexts and asynchronous processing together. When an expert completes an operation, it generates a structured callback prompt containing:

- the identifier of the context to update,
- the result of the computation,
- and an instruction for how the reasoning model should proceed.

The reasoning model processes the callback, updates the relevant context, and issues new instructions. This creates a closed-loop reasoning process in which control flow is driven by the arrival of callbacks rather than by a fixed sequence of API calls.

15.4.6 Advantages Over Token-Based Reasoning

Hierarchical and asynchronous context management offers several advantages over traditional prompt-based reasoning:

- **Persistence**: contexts persist independently of token windows.
- **Structure**: reasoning state is stored as objects, not text.
- **Parallelism**: multiple reasoning flows can run concurrently.
- **Asynchronicity**: experts operate independently and return results when ready.
- **Decomposition**: tasks can be broken into sub-contexts.
- **Scalability**: long-lived processes are not constrained by sequence length.

These properties enable the architecture to support complex, multi-step reasoning tasks that are difficult or impossible to implement with monolithic transformer models.

15.4.7 A Bidirectional Benefit for Transformers

Hierarchical and asynchronous contexts not only overcome the limitations of transformer-based reasoning but also enhance the transformer itself. By externalizing state into structured objects and delegating computation to asynchronous experts, the transformer no longer needs to maintain long-term memory or simulate concurrency through token continuation. This reduces cognitive load and allows the model to focus on higher-level reasoning operations such as rewriting, planning, and integrating expert feedback.

In this way, hierarchical and asynchronous contexts provide a structured cognitive environment in which transformer models can operate more effectively, while simultaneously enabling forms of reasoning that cannot emerge from next-token prediction alone.

15.4.8 Summary

Hierarchical contexts, parallel reasoning, and asynchronous processing transform the architecture from a linear prompt-driven system into a distributed, event-driven cognitive environment. By representing reasoning flows as persistent objects, enabling parallel and asynchronous execution, and coordinating progress through callback prompts, the system achieves a form of structured, domain-agnostic reasoning that is both scalable and interpretable. At the same time, this structure enhances the transformer-based reasoning core, providing the stability and modularity required for advanced cognitive behavior.

15.5 Monte Carlo and Domain-Specific Search as Depth Reasoning Mechanisms

Transformer-based models excel at pattern recognition and linguistic generalization, but they struggle with deep combinatorial reasoning. Their architecture does not support branching, simulation, or the exploration of alternative futures. For tasks that require evaluating many possible action sequences, a separate computational mechanism is needed. The architecture therefore incorporates a family of depth-search modules—ranging from generic Monte Carlo simulation to domain-specific search engines such as Beam Search, A* search, and constraint-based planners.

These search modules operate independently of the reasoning model and provide the computational substrate for exploring large decision spaces. The reasoning model delegates search tasks through the script engine, receives structured results via callback prompts, and integrates them into the ongoing reasoning flow. This separation ensures that the reasoning model remains lightweight and domain-agnostic, while the search modules supply the computational power needed for deep reasoning.

15.5.1 Monte Carlo as a Domain-Agnostic Search Mechanism

Monte Carlo simulation provides a simple, unbiased, and transparent method for exploring decision spaces. It requires only three components:

- a state representation,
- a function that enumerates legal actions,
- and a termination condition.

Given these, the search module repeatedly samples action sequences and records their outcomes. Monte Carlo is fully domain-agnostic: it does not rely on heuristics, learned parameters, or domain-specific knowledge. This makes it an ideal default search mechanism for new domains or environments where no structured heuristics are available.

Monte Carlo is particularly effective for:

- local tactical exploration,
- evaluating short-horizon futures,
- stochastic or uncertain environments,
- and domains with large branching factors.

Its transparency is a major advantage. Each decision is supported by explicit statistics:

- number of simulations,
- distribution of outcomes,
- estimated value of each action,
- and variance or confidence intervals.

These statistics can be inspected, logged, or audited—properties that are difficult to achieve with monolithic transformer models.

15.5.2 Domain-Specific Search: Beam Search, A*, and Beyond

While Monte Carlo provides a robust default, many domains benefit from more structured search strategies. The architecture therefore supports the integration of domain-specific search engines, including:

- **Beam Search** for multi-step planning in structured spaces,

- **A* search** for heuristic-guided optimal planning,
- **Dijkstra-style search** for cost-based exploration,
- **constraint solvers** for combinatorial optimization,
- **SAT/SMT solvers** for logical or symbolic domains,
- **domain-specific planners** with custom heuristics.

These search engines can exploit domain structure to prune the search space, construct long-horizon plans, or guarantee optimality. For example:

- Beam Search excels in symbolic workflows such as sorting, scheduling, or program synthesis.
- A* search is ideal for navigation, pathfinding, and structured optimization problems.
- Constraint solvers are effective for puzzles, scheduling, and resource allocation.

The architecture treats the choice of search engine as a *domain-level property*. Each domain specifies its preferred search strategy, and the script engine automatically routes search requests to the appropriate module. This ensures that the reasoning model remains entirely domain-agnostic.

15.5.3 Integration with the Reasoning Model

The interaction between the reasoning model and the search modules follows a simple pattern:

1. The reasoning model issues a script instruction requesting a search operation on a given context.
2. The search module performs the required simulations or expansions.
3. The module stores the results in the memory map and generates a callback prompt.
4. The reasoning model processes the callback and incorporates the results into its ongoing reasoning flow.

This callback-driven loop enables the system to perform deep reasoning without requiring the transformer to simulate search through next-token prediction.

15.5.4 Replaceability and Extensibility

Search modules are fully modular. They can be:

- replaced with more efficient algorithms,
- augmented with heuristics or learned policies,
- specialized for particular domains,
- or combined into hybrid search strategies.

Because all communication occurs through the script engine and memory map, search modules can be integrated without modifying the reasoning model or the Paged–PRE–CAS components.

15.5.5 A Bidirectional Benefit for Transformers

Depth search not only compensates for the limitations of transformer models but also enhances them. By delegating combinatorial exploration to explicit search modules, the transformer is freed from the impossible task of simulating search through statistical continuation. This allows the reasoning model to focus on higher-level cognitive functions such as canonical rewriting, variable binding, and flow control.

In this way, depth search provides a structured computational environment in which transformer models can operate more effectively, while simultaneously enabling forms of reasoning that cannot emerge from next-token prediction alone.

15.5.6 A Computational Substrate for General Reasoning

Monte Carlo, Beam Search, A*, and other search engines together form a generic computational substrate for exploring decision spaces. They complement the reasoning model's ability to plan, decompose tasks, and synthesize results. This division of labor enables the architecture to perform structured, multi-step reasoning across diverse domains without relying on monolithic parameter updates or opaque internal representations.

15.6 Self-Learning Without Weight Updates

A defining property of the architecture is that it can learn and expand its capabilities without modifying the parameters of the reasoning model. Instead of relying on gradient-based training or large-scale fine-tuning, the system acquires new skills procedurally: by generating new experts, integrating them through explicit interfaces, and incorporating them into its reasoning flows. Learning becomes a matter of *growing the system*, not altering its internal representations.

15.6.1 Procedural Learning Through Expert Acquisition

In traditional transformer architectures, learning requires updating the model's weights. This couples knowledge, reasoning, and computation into a single parameter space, making it difficult to add new capabilities without retraining the entire model. In contrast, the domain-bound AGI architecture separates these concerns. The reasoning model remains domain-agnostic, while new capabilities are introduced by adding expert modules that implement specific operations.

Experts can be:

- algorithmic modules (e.g., search engines, evaluators),
- deterministic tools (e.g., parsers, simulators),
- or neural models trained for a specific domain.

Once an expert is added, the reasoning model can immediately use it through the script engine, without requiring any parameter updates. This enables rapid adaptation to new domains and tasks.

15.6.2 Learning Through Specification-Driven Expert Generation

A powerful consequence of this modular design is that the system can generate new experts directly from *formal specifications*. Given a structured description of a domain—its objects, operations, constraints, and evaluation rules—the reasoning model can produce:

- interface-compliant expert code,
- domain-specific transformation functions,
- validation routines,
- or simulation logic.

These generated experts are then integrated into the architecture by registering them with the script engine and memory map. Because all experts share a common interface, the reasoning model can immediately orchestrate them through canonical rewriting and variable binding.

This mechanism enables a form of self-expansion: the system can extend its own computational substrate by generating new tools that conform to the architecture's interfaces. Learning becomes a process of *adding new modules*, not modifying existing ones.

15.6.3 Procedural Memory Instead of Parametric Memory

Traditional transformer models store knowledge implicitly in their parameters. This makes knowledge opaque, difficult to update, and tightly coupled to the model's internal representations. In contrast, the architecture uses *procedural memory*: knowledge is stored in the form of explicit experts, structured objects, and domain specifications.

Procedural memory has several advantages:

- **Transparency**: each expert is an inspectable component.
- **Replaceability**: experts can be updated without affecting the reasoning model.
- **Extensibility**: new experts can be added at any time.
- **Governance**: domain logic is explicit and auditable.

This decoupling allows the system to grow indefinitely without accumulating instability or drift.

15.6.4 Self-Improvement Through Structured Interaction

The reasoning model improves its performance not by changing its parameters but by learning how to orchestrate experts more effectively. Through repeated interaction with experts, the model acquires stable procedural patterns:

- how to decompose tasks,
- how to bind variables,
- how to sequence operations,
- how to integrate expert feedback,
- and how to terminate reasoning flows.

These patterns are encoded in the learned cognitive mechanisms described earlier—canonical rewriting, flow-based attention, and repeat–prompt recursion. As the system gains more experts, the reasoning model becomes capable of solving increasingly complex tasks by composing these experts in novel ways.

15.6.5 A Scalable Path to Domain Expansion

Because new experts can be generated from specifications and integrated through stable interfaces, the architecture supports continuous domain expansion. The system can:

- incorporate new computational tools,
- adopt new search strategies,
- integrate new symbolic or neural modules,
- and extend its reasoning capabilities without retraining.

This provides a scalable path toward domain-bound general intelligence: the system becomes more capable not by modifying its core, but by enriching the ecosystem of experts it can orchestrate.

15.6.6 A New Paradigm for Learning

Self-learning without weight updates represents a shift from parametric to procedural intelligence. Instead of embedding all knowledge inside a single model, the architecture distributes knowledge across explicit components that can be added, replaced, or audited independently. The reasoning model acts as a stable cognitive core that coordinates these components through structured interaction.

This paradigm enables a form of continual learning that is transparent, governable, and modular—qualities that are difficult to achieve with monolithic transformer architectures. It also provides a practical path toward systems that can grow indefinitely, adapting to new domains through explicit integration rather than implicit parameter change.

15.7 Comparison With Monolithic Transformer Architectures

Large transformer-based language models have reshaped the landscape of artificial intelligence by demonstrating broad generalization, linguistic fluency, and powerful pattern recognition. Their success stems from a simple design principle: compress as much statistical structure as possible into a single, monolithic parameter space. This approach has proven remarkably effective for tasks that can be framed as sequence prediction. Yet the same design principle imposes structural constraints that limit their ability to perform explicit reasoning, modular extension, and governed computation.

The architecture presented in this chapter represents a different paradigm. It does not compete with transformer models; it *repositions* them within a broader cognitive system. The contrast between the two approaches can be summarized along several conceptual axes.

15.7.1 Monolithic Compression vs. Compositional Structure

Transformers unify all cognitive functions—memory, reasoning, knowledge, and decision-making—inside a single parameter space. This yields powerful emergent behavior but makes the system opaque, brittle, and difficult to extend.

The proposed architecture distributes cognitive functions across explicit components: a reasoning core, a script engine, a structured memory map, and a collection of expert modules. Each component is optimized for its role, and the interactions between components are governed by explicit interfaces. This compositional structure enables transparency, modularity, and controlled extension.

15.7.2 Implicit Emergence vs. Explicit Mechanisms

Transformer reasoning emerges implicitly from statistical continuation. This makes it difficult to enforce procedural structure, maintain stable state, or perform multi-step workflows without drift.

In contrast, the architecture implements reasoning through explicit mechanisms: canonical rewriting, variable binding, callback-driven continuation, hierarchical contexts, and asynchronous expert coordination. These mechanisms provide predictable, auditable control flow that does not depend on emergent token dynamics.

15.7.3 Static Models vs. Dynamic Systems

Monolithic transformers are static artifacts: once trained, their capabilities are fixed unless the entire model is fine-tuned. Domain adaptation requires parameter updates, which are expensive, opaque, and potentially destabilizing.

The modular architecture is a dynamic system. It grows by integrating new experts, new search strategies, and new domain specifications—without modifying the reasoning model. Learning occurs through system expansion rather than parameter change. This enables continual adaptation while preserving stability.

15.7.4 Opaque Internal State vs. Externalized Cognitive State

Transformers encode all state in high-dimensional activations and token sequences. This makes intermediate reasoning steps difficult to inspect or govern.

The proposed architecture externalizes cognitive state into structured objects stored in the memory map. Contexts, variables, intermediate results, and search traces are explicit and inspectable. This supports governance, safety, and interpretability in ways that monolithic models cannot achieve.

15.7.5 Single-Threaded Generation vs. Distributed Cognition

Transformers operate as single-threaded sequence generators. They cannot natively support parallel reasoning, asynchronous workflows, or multi-agent coordination.

The modular architecture is inherently distributed. Experts operate asynchronously, contexts evolve independently, and reasoning flows can proceed in parallel. The system behaves as a network of interacting cognitive processes rather than a single generative stream.

15.7.6 A Bidirectional Relationship

The contrast between the two approaches is not adversarial. Transformer models remain essential: they provide the reasoning core that orchestrates the architecture. At the same time, the architecture enhances transformers by providing:

- explicit memory,
- structured control flow,
- domain-specific computation,
- and transparent search mechanisms.

Transformers become more capable within the architecture than they are on their own. The relationship is bidirectional: the architecture generalizes the transformer, and the transformer empowers the architecture.

15.7.7 Synthesis

Monolithic transformer architectures excel at compressing knowledge and recognizing patterns, but they are not designed for structured reasoning, modular extension, or governed computation. The architecture presented here offers a complementary paradigm: a compositional cognitive system in which reasoning is explicit, memory is structured, search is transparent, and learning occurs through modular integration. Together, these properties provide a scalable and interpretable foundation for domain-bound general intelligence.

15.8 Domain-Bound AGI as a Recipe

The architecture introduced in this chapter is best understood not as a single system but as a *pattern*: a reusable cognitive recipe for constructing intelligent systems across diverse domains. Rather than embedding all knowledge and reasoning inside one monolithic model, the architecture defines a set of interacting components—each with a clear role and interface—that can be instantiated, combined, and extended to form domain-bound intelligent systems.

This recipe-based perspective shifts the focus from training a universal model to assembling a coherent cognitive system. A new domain does not require a new reasoning model; it requires a new configuration of components that the reasoning model can orchestrate.

15.8.1 A Domain-Agnostic Cognitive Core

At the center of every instantiation lies a domain-agnostic reasoning core. Its function is not to contain domain knowledge but to provide the procedural scaffolding for reasoning:

- interpreting tasks,
- structuring workflows,
- coordinating computational modules,
- and maintaining coherent control flow.

Because the core is independent of domain content, it can be reused across all domains. What changes from domain to domain is not the reasoning process but the computational substrate on which it operates.

15.8.2 Domain Logic as Modular Ingredients

Domain-specific behavior is supplied by modular components that implement the operations, transformations, and evaluations relevant to a particular domain. These components act as the *ingredients* of the recipe. They may be algorithmic, symbolic, or neural, and they can be generated from formal specifications or implemented manually.

What matters is not their internal design but their adherence to the architecture's interfaces. Once integrated, they become part of the system's computational vocabulary—tools that the reasoning core can invoke as needed.

15.8.3 Search as a Pluggable Exploration Strategy

Many domains require exploration of alternative futures or combinatorial structures. The architecture treats search as a pluggable component: each domain may specify the exploration strategy that best fits its structure.

This design avoids prescribing a single universal search mechanism. Instead, it provides a place in the recipe where domain-appropriate exploration can be inserted—whether Monte Carlo sampling, heuristic search, constraint solving, or another method.

15.8.4 Memory as a Structured Cognitive Workspace

The architecture externalizes cognitive state into a structured workspace. This workspace stores objects, intermediate results, and reasoning contexts in a form that is persistent, inspectable, and independent of token sequences.

In the recipe metaphor, memory is the *work surface*: the place where ingredients are combined, transformed, and assembled into coherent results.

15.8.5 Instantiating a New Domain

Constructing a new domain-bound system becomes a procedural act rather than a training exercise. The recipe can be summarized as:

1. Define the domain's objects, operations, and constraints.
2. Implement or generate experts that realize these operations.
3. Register these experts through the architecture's interfaces.
4. Specify the domain's preferred search strategy.
5. Initialize contexts that represent the tasks to be solved.
6. Allow the reasoning core to orchestrate the system.

The reasoning model requires no modification; the system adapts through integration rather than parameter change.

15.8.6 A General Pattern for Constructing Intelligence

The result is a general pattern for building intelligent systems:

- a reusable reasoning core,
- a structured workspace,
- a communication layer,
- and a set of domain-specific computational tools.

Each instantiation follows the same recipe but differs in the ingredients it uses. This provides a scalable, interpretable, and governable path toward domain-bound general intelligence—one in which systems grow by acquiring new modules rather than modifying their internal parameters.

15.9 Learned Cognitive Mechanisms in the Reasoning Core

Beyond the structural components of the architecture—experts, memory, search, and the script engine—the reasoning model contributes a lightweight cognitive layer that provides coherence, stability, and procedural structure. These mechanisms do not encode domain knowledge; instead, they define how reasoning unfolds. They arise from curriculum learning within the Paged–PRE–CAS framework and enable the model to function as a domain-agnostic controller.

15.9.1 Canonicalization Through Prompt Rewriting

The reasoning model begins each reasoning cycle by rewriting natural-language inputs into canonical machine forms. This process extracts structure, removes linguistic variability, and produces stable representations that downstream components can interpret reliably. Canonicalization ensures that reasoning is grounded in explicit structure rather than emergent token patterns.

15.9.2 Variable Binding Through Local Prompt Maps

During rewriting, the model constructs a local variable map that binds symbols, identifiers, and references to stable names. These bindings persist across the reasoning flow and provide referential coherence without relying on long context windows. Each context maintains its own variable map, enabling independent and parallel reasoning processes.

15.9.3 Flow-Based Procedural Attention

Instead of generic self-attention, the model employs flow-based attention patterns learned through curriculum. These patterns encode procedural regularities—such as classification before evaluation or evaluation before simulation—and guide the sequencing of operations. Flow-based attention provides predictable, auditable control flow without embedding domain logic in the model's parameters.

15.9.4 Callback Integration Through Self-Prompting

Experts return structured callbacks that signal the next reasoning step. The model integrates these callbacks through learned rewriting and variable binding, allowing it to continue reasoning autonomously. This self-prompting mechanism turns the model into a procedural controller capable of coordinating asynchronous expert activity and advancing multi-step workflows.

15.9.5 Context–Output Separation

A foundational principle of the architecture is the strict separation between internal reasoning context and external output. The model does not accumulate a growing transcript; it reconstructs its reasoning state from structured memory at every step. This prevents drift, eliminates dependency on past text, and enables arbitrarily long reasoning processes without degradation.

15.9.6 Repeat–Prompt Recursion

Repeat–Prompt Recursion operationalizes this separation. At each step, the model regenerates a fresh canonical prompt that encodes the current task, symbolic state, variable bindings, and required next operation. This creates a controlled recursive loop:

$$\text{Context} \rightarrow \text{Rewrite} \rightarrow \text{Expert Call} \rightarrow \text{Callback} \rightarrow \text{Context Reconstruction}.$$

The result is a stable, deterministic reasoning process that does not depend on long-context attention or transcript accumulation.

15.9.7 Reasoning as a Controlled Recursive Process

Together, these mechanisms transform the reasoning model into a symbolic controller that:

1. rewrites its own prompt,
2. binds variables,
3. invokes experts,
4. integrates callbacks,
5. reconstructs context,
6. and repeats.

This controlled recursion provides the semantic stability and procedural structure required for modular, interpretable, and domain-agnostic reasoning. The cognitive layer does not solve tasks by itself; it orchestrates the components that do. The architecture provides the structure, the model provides the flexibility, and their interaction produces coherent cognition.

15.10 Conclusion

The architecture presented in this chapter reframes intelligence as a modular interaction between a domain-agnostic reasoning core and a set of explicit, inspectable computational components. Rather than concentrating all knowledge and reasoning inside a single model, the system distributes cognition across experts, structured memory, search procedures, and a lightweight cognitive layer implemented within the Paged–PRE–CAS framework.

This separation of concerns yields several advantages. The reasoning core remains stable and reusable across domains, while domain-specific behavior is supplied by modular experts that can be added, replaced, or generated from formal specifications. Memory is externalized into structured objects that persist across reasoning steps, enabling long-lived, hierarchical, and parallel workflows. Search is transparent and pluggable, allowing each domain to employ the exploration strategy best suited to its structure.

The learned cognitive mechanisms of the reasoning model—canonical rewriting, variable binding, flow-based attention, self-prompting, and repeat–prompt recursion—provide the procedural glue that binds these components into a coherent system. They ensure that reasoning is explicit, stable, and interpretable, even as the system grows and adapts.

Taken together, these elements form a general recipe for constructing domain-bound intelligent systems. The architecture does not rely on monolithic parameter updates or opaque internal representations. Instead, it builds intelligence through composition: a reasoning core that orchestrates explicit tools within a structured cognitive environment. This approach offers a scalable, governable, and transparent foundation for artificial systems capable of sustained, multi-step reasoning across diverse domains.

Architectural Insight

Domain-bound intelligence emerges when a stable, domain-agnostic reasoning core orchestrates explicit experts, structured memory, and transparent search within a controlled recursive process. The architecture separates context from output, distributes cognition across modular components, and relies on learned canonicalization, variable binding, and flow-based control to maintain coherence. Intelligence is not concentrated in a single model but constructed through composition: a reasoning core that coordinates explicit tools inside a governable cognitive environment.

Chapter 16

Toy Board: A Practical Domain-Bound AGI Example

16.1 Introduction: Why Toy Board?

Toy Board is a compact but fully operational example of *domain-bound artificial general intelligence*. It demonstrates how a small, modular reasoning system can solve heterogeneous problems—games, puzzles, planning tasks, and even sorting—through a unified cognitive architecture. Unlike transformer-based language models, which internalize all world knowledge into a single monolithic network, Toy Board separates *reasoning* from *domain logic*. The result is a transparent, interpretable, and extensible system that behaves like a miniature cognitive agent.

At its core, Toy Board implements a universal *board-game ontology*. Every domain, whether it is Connect Four, Nim, MiniChess, Sliding Puzzle, Sudoku, or even a sorting task, is mapped onto the same abstract interface: a `BoardState`, a set of legal `BoardMove`s, and a deterministic `BoardDomain` that defines the transition rules. This abstraction is powerful: it allows a single reasoning model to operate across many domains without ever learning their internal mechanics. The model does not know how Sudoku works or how to sort a list; instead, it learns how to *think* about problems expressed through this ontology.

Toy Board's reasoning engine is built on the PAGED–PRE–CAS architecture introduced in Toy Story, but repurposed for procedural cognition. Instead of generating text, the model generates *programs*: structured sequences of `STORE` and `CALL` instructions, variable bindings, search invocations, evaluations, and termination markers. These programs are executed by a deterministic Script Engine that coordinates memory, search engines, domain transitions, and failure detection. The system therefore does not merely predict the next token; it constructs and executes a plan.

This architectural separation—a small neural core for reasoning, surrounded by modular domain engines for world dynamics—has profound implications. A Toy Board model with approximately one megabyte of parameters can coordinate eight heterogeneous domains. Extrapolated to full AGI, the same architectural principles would scale to terabyte-sized systems: large, but feasible. In contrast, a transformer-based AGI attempting to internalize all domain logic would require exabyte-scale models, pushing far beyond the limits of practical computation. Toy Board thus provides a concrete, operational argument for why AGI must be modular, structured, and tool-driven rather than monolithic.

This chapter presents Toy Board as a practical, working example of domain-bound AGI. It introduces the cognitive vocabulary, paging structure, PRE reasoning loop, evaluator system, arbiter, script engine, and search integration that together form a complete reasoning pipeline. It

also shows how diverse domains can be mapped onto a single ontology, and why this mapping is a powerful abstraction for scalable intelligence.

Toy Board is not a toy in the trivial sense. It is a blueprint: a minimal, executable realization of an AGI architecture that separates reasoning from world knowledge, delegates complexity to modular engines, and uses explicit structure instead of statistical correlation. It demonstrates that intelligence does not require massive models, but the right architecture— one that reasons, plans, and decides.

16.2 The Domain-Bound AGI Problem

Artificial General Intelligence is often framed as a monolithic challenge: a single model that must internalize all world knowledge, all reasoning strategies, all domains, and all forms of problem solving. This framing implicitly assumes that intelligence is a property of a single, undifferentiated function approximator. Toy Board challenges this assumption. It demonstrates that AGI can instead be approached as a *domain-bound* problem: a system that reasons universally, but acts within well-defined interfaces that encapsulate domain-specific rules.

Domain-bound AGI is not a weaker form of intelligence. It is a more structured one. Humans do not store the rules of chess, arithmetic, navigation, and language in a single undifferentiated neural substrate. We rely on modular cognitive processes, external tools, symbolic representations, and learned procedures that interact through shared working memory and control flow. Toy Board adopts the same principle. It separates the *reasoning core* from the *domain engines*, allowing each to specialize without interference.

The central challenge of domain-bound AGI is therefore not to learn the rules of every possible domain, but to learn the *cognitive glue* that binds domains together: classification, planning, search invocation, evaluation, memory management, and termination. These are the universal operations that allow an agent to operate across heterogeneous environments. Toy Board shows that these operations can be learned by a small PRE–CAS model, provided that the domains themselves expose a uniform interface.

This leads to a crucial insight: the difficulty of AGI does not lie in the complexity of the domains, but in the complexity of the *interfaces* that connect reasoning to action. Once a domain is expressed through a simple ontology—a board state, a set of legal moves, and a deterministic transition function—the reasoning model can treat it as just another environment. Connect Four, Nim, MiniChess, Sliding Puzzle, Sorting, and Sudoku all become instances of the same abstract problem: *given a state, propose and evaluate actions until a goal is reached or failure is detected.*

Toy Board therefore reframes AGI as an architectural problem rather than a scaling problem. Instead of increasing model size to absorb more domains, it increases the *expressive power of the interfaces* that connect a small reasoning core to a growing collection of domain engines. This modularity is what makes Toy Board scalable, interpretable, and computationally feasible. It is also what makes it a compelling example of domain-bound AGI: a system that reasons universally, but acts locally.

In the sections that follow, we examine how Toy Board implements this principle in practice. We introduce the cognitive vocabulary, paging structure, PRE reasoning loop, evaluators, arbiter, script engine, and search integration that together form a complete domain-bound AGI pipeline. We also show how diverse domains can be mapped onto a single board-game ontology, and why this mapping is a powerful abstraction for general-purpose problem solving.

16.3 The Toy Board Architecture Overview

Toy Board is built on a simple but powerful idea: a small reasoning core can coordinate multiple cognitive processes—classification, search, evaluation, memory, and control flow—provided that each process is exposed through a clean, uniform interface. The architecture therefore consists of a compact neural component surrounded by a set of deterministic modules that implement domain logic, search strategies, and program execution. Together, these components form a complete domain-bound AGI pipeline.

At a high level, Toy Board operates in three phases:

1. **Reasoning:** The PRE–CAS model generates a structured program consisting of `STORE` and `CALL` instructions, variable bindings, and termination markers. This program is not text but executable cognitive code.

2. **Execution:** The Script Engine interprets the program deterministically. It manages variables, invokes search engines, queries domain rules, updates board states, and detects termination or failure.

3. **World Interaction:** Domain engines implement the rules of each environment. They define legal moves, state transitions, heuristic scores, and win conditions. The reasoning model never learns these rules; it merely orchestrates their use.

This separation of concerns is the defining feature of Toy Board. The neural model is responsible for *how to think*, while the domain engines are responsible for *what happens* when an action is taken. The architecture therefore avoids the monolithic entanglement of reasoning and world knowledge that characterizes large language models.

Components of the Architecture

Toy Board consists of the following major components:

- **Token Dictionary and Cognitive Vocabulary** A compact set of control tokens, variable tokens, and domain-agnostic words that define the language of reasoning.

- **Paging System** A decomposition of the vocabulary into small expert pages, each responsible for a coherent subset of cognitive operations. Paging enforces structural correctness and reduces interference between unrelated token types.

- **PRE Reasoning Loop** A propose–refine–evaluate cycle that generates candidate tokens, filters them through evaluators, and accumulates valid proposals for the arbiter.

- **Evaluators** A set of orthogonal micro-experts that enforce variable integrity, script structure, domain consistency, procedural coherence, and syntactic correctness.

- **Arbiter** A decision-maker that integrates evaluator scores, injects controlled variation, and selects the next token from the refined candidate set.

- **Script Engine** A deterministic interpreter that executes the generated program, manages memory, invokes search engines, updates board states, and detects abort conditions.

- **Domain Engines** Modular implementations of `BoardDomain`, `BoardState`, and `BoardMove` for each environment. These engines encapsulate all domain rules and provide a uniform interface for reasoning.

A Cognitive Pipeline

The interaction between these components forms a complete cognitive pipeline:

1. The user provides a natural-language question.
2. The PRE–CAS model generates a reasoning program token by token.
3. The Script Engine executes the program deterministically.
4. Domain engines compute legal moves, transitions, and evaluations.
5. Search engines explore the state space when required.
6. The Script Engine reports the result or aborts if no solution exists.

Every step is transparent, interpretable, and reproducible. There are no hidden states, no implicit memories, and no statistical shortcuts. Toy Board therefore provides a rare opportunity to observe a complete AGI-style reasoning loop in a fully inspectable form.

Why This Architecture Matters

Toy Board demonstrates that AGI does not require a monolithic model that internalizes all domains. Instead, it shows that a small reasoning core can coordinate modular processes through explicit structure, paging, evaluators, and deterministic execution. This architecture scales with the number of domains not by increasing model size, but by adding new domain engines that implement the shared ontology.

In the sections that follow, we examine each component of this architecture in detail, beginning with the cognitive vocabulary and the paging system that routes control between expert networks.

16.4 Tokenization and the Cognitive Vocabulary

Toy Board does not operate on natural language tokens or subword fragments. Instead, it uses a compact, fully deterministic *cognitive vocabulary*: a set of control tokens, variable tokens, and domain-agnostic words that together form a procedural language for reasoning. This vocabulary is not designed for expressiveness in the linguistic sense, but for expressiveness in the *cognitive* sense: it encodes the operations, memory structures, and control flow required to solve problems across heterogeneous domains.

The tokenization scheme is deliberately minimal. Each token corresponds to a discrete cognitive action or symbol, and each is mapped to a stable integer identifier through a deterministic hashing function. There is no ambiguity, no learned embedding space, and no context-dependent interpretation. The vocabulary defines the ontology of Toy Board's reasoning process.

Control Tokens

Toy Board introduces a small set of control tokens that form the backbone of its procedural language:

- `STORE` — binds a natural-language question to a variable.
- `CALL` — invokes a cognitive subroutine (classification, search, evaluation, repeat).

- `EOL` — marks the end of a reasoning line.
- `EOR` — marks the end of the reasoning program.

These tokens define the structure of the reasoning trace. They are not optional: every valid program must begin with a `STORE`, proceed through a sequence of `CALL` instructions, and terminate with `EOR`. The Script Engine interprets these tokens deterministically, making the reasoning process transparent and reproducible.

Variable Tokens

Toy Board uses explicit variable tokens to represent identifiers and intermediate results:

- `$BID` — the board identifier.
- `$PID` — the prompt identifier.
- `$MOVE` — the move proposed by a search engine.
- `$PLAYER` — the active player in the domain.

These variables form the working memory of the system. They are created, updated, and queried by the Script Engine, and their integrity is enforced by the Variable Evaluator. This explicit memory mechanism stands in sharp contrast to transformer-based models, which must represent such information implicitly in hidden activations.

Domain-Agnostic Cognitive Words

In addition to control and variable tokens, Toy Board includes a small set of domain-agnostic words that describe cognitive operations:

- `classify` — determine the domain of a board.
- `search` — invoke a domain-specific search engine.
- `evaluate` — determine whether the game is won, lost, or ongoing.
- `repeat` — re-enter the reasoning loop with updated state.

These words are not tied to any specific environment. They represent universal cognitive operations that apply equally to Connect Four, Nim, MiniChess, Sudoku, and Sorting. This is what makes Toy Board domain-bound rather than domain-specific: the reasoning model learns the *structure* of problem solving, not the rules of any particular domain.

A Procedural Language for Reasoning

Taken together, the cognitive vocabulary forms a small procedural language. A typical reasoning trace might look like:

```
<STORE> what should be the next move for $BID
<CALL> classify $BID into $PID
<CALL> search $MOVE for $BID into $PID
<CALL> evaluate $BID into $PID
<CALL> repeat prompt into $PID
<EOR>
```

This is not text generation. It is program synthesis. The PRE–CAS model constructs a program token by token, the Script Engine executes it, and the domain engines provide the world dynamics. The vocabulary is therefore not a linguistic medium but a cognitive one.

Why This Vocabulary Matters

The cognitive vocabulary is the foundation of Toy Board's architecture:

- It defines the space of possible reasoning programs.
- It enforces structural constraints through paging and evaluators.
- It enables explicit memory through variable tokens.
- It separates reasoning from domain logic.
- It allows the Script Engine to execute reasoning deterministically.

By reducing reasoning to a small, interpretable language, Toy Board achieves something rare: a complete AGI-style reasoning loop that is transparent, modular, and fully inspectable. In the next section, we examine how paging decomposes this vocabulary into expert networks that enforce structural correctness and cognitive specialization.

16.5 Paging: Routing Cognitive Control

Toy Board inherits the paging mechanism from Toy Story, but repurposes it for a very different cognitive landscape. In Toy Story, paging decomposed the vocabulary into semantic domains such as actors, timing words, and structural markers. In Toy Board, paging becomes a *control-flow router*: it determines which expert network is allowed to speak next, based on the structure of the reasoning program being constructed.

The vocabulary of Toy Board is not linguistic but procedural. It consists of control tokens (`STORE`, `CALL`, `EOL`, `EOR`), variable tokens (`$BID`, `$PID`, `$MOVE`), and domain-agnostic cognitive words (`classify`, `search`, `evaluate`, `repeat`). These tokens do not form sentences; they form *programs*. Paging ensures that each token is produced by the appropriate expert, enforcing structural correctness and preventing invalid transitions.

A Decomposition of Cognitive Roles

Toy Board partitions its vocabulary into a set of small, non-overlapping pages, each corresponding to a distinct cognitive role:

- **STORE Page** — responsible for variable-binding instructions.
- **CALL Page** — responsible for invoking cognitive subroutines.

- **Variable Page** — responsible for producing variable identifiers.
- **Cognitive-Word Page** — responsible for tokens such as `classify`, `search`, `evaluate`, and `repeat`.
- **Structural Pages** — deterministic pages for `EOL` and `EOR`.

Each page contains only the tokens relevant to its role. The PRE–CAS model therefore never faces a large softmax over the entire vocabulary; it selects from a small, coherent subset of tokens that are structurally valid in the current context.

Paging as a Structural Constraint

Paging enforces strong structural constraints on the reasoning program:

- After `STORE`, only variable tokens or cognitive words may appear.
- After `CALL`, the next token must be a cognitive word or a variable.
- After a cognitive word such as `search`, the next token must be a variable.
- `EOL` may only appear at the end of a reasoning line.
- `EOR` may only appear when the program is complete.

These constraints eliminate entire classes of errors. A page that contains only variable tokens cannot accidentally output `CALL`; a page that contains only structural tokens cannot output `search`. Paging therefore acts as a built-in safety mechanism that keeps the reasoning program within the boundaries of the cognitive language.

Paging as Cognitive Routing

In Toy Board, paging is not merely an optimization; it is a form of *cognitive routing*. It determines which expert network is responsible for the next step in the reasoning process. This routing is based on the last token in the context window, which encodes the current phase of the program:

- `STORE` activates the variable-binding expert.
- `CALL` activates the subroutine-invocation expert.
- Cognitive words activate the variable or structural experts.
- `EOL` activates the expert responsible for line termination.

This mechanism ensures that the PRE–CAS model always operates within a valid procedural context. It also reduces interference between unrelated cognitive operations, making the system easier to train, debug, and interpret.

Why Paging Matters

Paging provides several key advantages in the context of domain-bound AGI:

- **Structural Safety** — invalid token transitions become impossible by construction.
- **Modularity** — each expert specializes in a narrow cognitive role.
- **Interpretability** — errors can be traced to specific pages.
- **Stability** — small softmax layers converge quickly and reliably.
- **Scalability** — new cognitive operations can be added by introducing new pages.

In Toy Board, paging is the mechanism that transforms a small neural network into a structured reasoning engine. It ensures that the PRE–CAS model generates programs rather than text, and that these programs adhere to the rules of the cognitive language.

In the next section, we examine how the PRE reasoning loop uses paging, evaluators, and feedback to construct valid reasoning programs token by token.

16.6 PRE: Propose–Refine–Evaluate for Procedural Reasoning

Toy Board uses the same PRE mechanism introduced in Toy Story, but in a radically different context. Instead of generating narrative tokens, PRE constructs *programs*: structured sequences of `STORE` and `CALL` instructions, variable bindings, search invocations, evaluations, and termination markers. PRE is therefore not a stylistic refinement loop but a *procedural reasoning engine*. It transforms token prediction into a multi-step cognitive process that resembles planning, correction, and decision-making.

PRE operates at the heart of Toy Board's architecture. It is the mechanism that allows a one-megabyte model to produce valid reasoning traces across heterogeneous domains without ever learning the rules of those domains. PRE ensures that each token is proposed, evaluated, and refined within the structural constraints of the cognitive language.

The Propose Step

In the propose phase, the active page-expert receives:

- the fixed-position encoding of the context window,
- the target CAS value,
- a role indicator specifying that this is an initial proposal.

The expert produces a softmax distribution over the tokens in its page. The highest-logit token becomes the initial proposal, but it is not accepted immediately. Instead, it is passed to the evaluators, which assess:

- variable integrity,
- script structure,
- domain consistency,

- procedural coherence,
- CAS alignment.

The result is a structured evaluation object that captures both the token and its constraint profile.

The Refinement Steps

After the initial proposal, PRE performs a sequence of refinement iterations. In each iteration, the expert receives a richer input:

- the context window,
- the target CAS value,
- the previous proposal,
- the evaluator scores associated with that proposal.

This feedback loop allows the expert to correct its own mistakes. A refinement step may:

- replace an invalid variable with a valid one,
- adjust a CALL structure to match the expected syntax,
- select a different cognitive word that better fits the context,
- avoid premature termination,
- improve procedural coherence.

Each refinement produces a new candidate token, which is again evaluated. All valid proposals across all refinement steps are accumulated into a candidate pool.

Evaluation as a Procedural Constraint System

Unlike transformer-based models, where constraints are applied after sampling, Toy Board applies constraints *before* selection. The evaluators act as independent micro-experts that enforce:

- syntactic correctness,
- variable consistency,
- domain validity,
- structural form,
- procedural continuity.

Invalid proposals are discarded immediately. Refinement is guided by explicit feedback, ensuring that the candidate pool contains only structurally valid and semantically coherent tokens.

PRE as a Cognitive Process

PRE transforms token generation into a multi-step cognitive process:

1. propose an initial candidate,
2. evaluate it through orthogonal constraints,
3. refine based on explicit feedback,
4. accumulate all valid proposals,
5. pass the candidate set to the arbiter.

This mirrors human reasoning: we generate an idea, check it against constraints, revise it, and consider alternatives before committing. PRE is therefore a computational realization of deliberation.

Why PRE Matters for Domain-Bound AGI

PRE is essential for Toy Board's ability to operate across heterogeneous domains:

- **Correctness** — invalid tokens are filtered before selection.
- **Robustness** — refinement corrects weak or borderline proposals.
- **Generalization** — the same reasoning loop applies to all domains.
- **Interpretability** — every proposal and refinement step is traceable.
- **Modularity** — evaluators and experts remain independent agents.

In Toy Board, PRE is the mechanism that turns a small neural network into a structured, domain-agnostic reasoning engine. It is the bridge between paging, evaluators, and the arbiter, and it is the foundation of the system's ability to generate executable cognitive programs.

In the next section, we examine the evaluators that provide the constraint signals guiding PRE's refinement process.

16.7 Evaluators: Coherence as the Only Learned Critic

In Toy Story, multiple evaluators explicitly encoded style, timing, line structure, and actor consistency. Toy Board takes a different route. Here, almost all structure is enforced *architecturally* rather than through learned evaluators. The only true evaluator in the system is a small learned *Coherence model*; all other constraints emerge from the combination of prompt rewriting, explicit variables, paging, and the deterministic Script Engine.

This design choice is deliberate. Toy Board is a domain-bound AGI system: it relies on explicit program structure, a fixed cognitive vocabulary, and a strict execution model. Most of what transformer-based systems would need to learn as "soft constraints" is instead hard-coded into the language of reasoning and the interpreter that executes it.

Structural Constraints Without Evaluators

Toy Board enforces the majority of its correctness properties without learned evaluators:

- **Program grammar** is enforced by the Script Engine. Invalid sequences (e.g., `CALL` without a verb, missing variables, misplaced `EOR`) simply cannot be executed and are treated as failures.
- **Variable discipline** is enforced by explicit variable tokens and the interpreter. Variables such as `$BID`, `$PID`, and `$MOVE` must be bound before use; otherwise, execution fails.
- **Domain consistency** is enforced by the domain interfaces. Each `BoardDomain` implementation defines which operations are valid; illegal calls result in execution errors or abort.
- **Control flow** is enforced by the cognitive vocabulary and paging. The model can only choose from structurally valid token types in each context, making many syntactic errors impossible by construction.
- **Attention-like behavior** is achieved through prompt rewriting and variable reuse. The model repeatedly sees the current board, the last move, and the relevant identifiers in a structured prompt, focusing its reasoning on the right parts of the state.

These mechanisms act as *implicit evaluators*: they do not score tokens, but they constrain what can be generated and what can be executed. The only component that actually assigns a learned score to a proposal is the Coherence model.

The Coherence Model: Learning Procedural Continuity

The Coherence model is a small neural network trained to distinguish good reasoning steps from bad ones. It is the only evaluator in Toy Board in the strict sense: it takes a candidate token (together with its context) and outputs a continuous score between 0 and 1 that reflects how *procedurally coherent* that token is.

Training data for the Coherence model is constructed from:

- valid reasoning traces produced by the system,
- contrastive micro-errors such as:
  - misplaced `CALL` instructions,
  - incorrect variable usage,
  - premature or missing `EOR`,
  - unnatural ordering of cognitive operations.

The Coherence model does not enforce hard rules; those are already handled by the architecture. Instead, it captures *soft regularities* in procedural behavior, such as:

- `classify` tends to appear early in the program,
- `search` usually follows classification,
- `evaluate` often precedes a decision to repeat or terminate,
- `repeat` is unlikely to appear before any search has been performed.

These patterns are not strictly required by the grammar, but they make reasoning more natural, efficient, and robust.

How Coherence Interacts with PRE

Within the PRE loop, the Coherence model provides a scalar signal for each proposed token. This signal is used to:

- down-rank proposals that are procedurally odd or implausible,
- up-rank proposals that fit well with the surrounding context,
- guide refinement toward more coherent CALL sequences,
- help the arbiter distinguish between multiple structurally valid options.

Crucially, Coherence does not replace structural checks. A token that violates the program grammar or variable discipline is rejected outright by the architecture, regardless of its coherence score. The evaluator's role is to shape behavior *within* the space of structurally valid programs.

Why a Single Evaluator Is Enough

Toy Board shows that a rich evaluator stack is not always necessary. Because so much structure is made explicit in:

- the cognitive vocabulary,
- the paging layout,
- the Script Engine,
- the domain interfaces,
- and the prompt rewriting scheme,

a single learned Coherence model is sufficient to capture the remaining degrees of freedom in procedural behavior. This keeps the system small, interpretable, and easy to extend.

In the next section, we examine how Toy Board filters candidate tokens *before* selection, combining structural constraints and coherence scores into a curated set of valid proposals for the arbiter.

16.8 Filtering and Arbitration: A Unified Decision Mechanism

Toy Board inherits the general PRE–CAS decision structure from Toy Story, but its implementation is significantly simpler and more architectural. Because Toy Board has only one learned evaluator (the Coherence model) and relies heavily on structural constraints enforced by paging, variables, and the Script Engine, the distinction between "filtering" and "arbitration" becomes conceptually thinner than in Toy Story. Both mechanisms work together to ensure that only structurally valid and procedurally coherent tokens are ever selected.

This section describes the unified decision mechanism that governs how Toy Board chooses the next token in a reasoning program.

Structural Filtering Before Scoring

Before any scoring or arbitration occurs, Toy Board applies a strict structural filter. This filter is not learned; it emerges directly from the architecture:

- **Paging** restricts the active vocabulary to the correct token type.
- **The program grammar** rejects illegal sequences (e.g., `CALL` without a verb).
- **Variable discipline** ensures that identifiers such as `$BID` and `$MOVE` are bound before use.
- **Domain interfaces** reject operations that are not supported by the current board.
- **The Script Engine** enforces execution validity and detects impossible actions.

These constraints eliminate the majority of invalid proposals *before* the Coherence model is even consulted. As a result, Toy Board does not require a large evaluator stack: most errors are impossible by construction.

Coherence as the Only Learned Signal

After structural filtering, the remaining candidates are scored by the Coherence model. This model provides a single scalar:

$$s = \text{coherenceScore},$$

which reflects how procedurally natural the token is within the current context. The score captures soft regularities in reasoning behavior—for example, that `classify` tends to appear early, or that `search` typically follows classification.

Because all hard constraints are enforced architecturally, Coherence does not need to police syntax or domain rules. Its role is to guide the system toward more fluent and robust reasoning patterns.

Candidate Pool Construction

Toy Board constructs a candidate pool in two steps:

1. **Select the first structurally valid token as the anchor.** Tokens are considered in descending logit order. The first token that passes all structural checks becomes the anchor candidate.
2. **Add additional candidates based on a coherence threshold.** For each remaining structurally valid token, Toy Board includes it in the candidate pool if:

$$s \geq s_{\text{anchor}} \times \text{scoreThreshold}.$$

This ensures that the arbiter receives a curated set of high-quality alternatives: all structurally valid, all procedurally coherent, and all close in quality to the anchor.

Arbitration: Controlled Variation Among Valid Options

Once the candidate pool is constructed, the arbiter performs a simple but effective selection:

- If the pool contains only one candidate, it is selected deterministically.
- If multiple candidates qualify, the arbiter selects uniformly at random.

This mechanism injects controlled variation without sacrificing correctness. Because all candidates have already passed structural filtering and coherence thresholding, the arbiter's randomness is bounded and safe.

A Unified Decision Pipeline

In Toy Board, filtering and arbitration form a single cognitive pipeline:

1. eliminate structurally invalid tokens,
2. score remaining tokens with the Coherence model,
3. construct a high-quality candidate pool,
4. select one candidate with bounded randomness.

This pipeline is simpler than Toy Story's evaluator-rich architecture, but also more powerful in a procedural setting. By shifting most constraints into the architecture itself, Toy Board reduces the burden on the neural model and ensures that every selected token is both executable and coherent.

In the next section, we examine the Script Engine, which executes the generated program and connects reasoning to world dynamics.

16.9 The Script Engine: A Deterministic Interpreter for Cognitive Programs

Toy Board differs fundamentally from transformer-based systems in one respect: its output is not text but an *executable program*. Every reasoning trace produced by the PRE–CAS model is a sequence of `STORE` and `CALL` instructions, variable bindings, and termination markers. The Script Engine is the component that interprets this program, executes its cognitive operations, and connects the reasoning model to the underlying domain engines.

The Script Engine is deterministic. Given the same program and the same board state, it will always produce the same result. This determinism is essential: it ensures that Toy Board's reasoning is transparent, reproducible, and debuggable. It also allows the system to treat reasoning as a form of program synthesis rather than probabilistic sampling.

Programs, Not Predictions

A typical Toy Board reasoning trace looks like:

```
<STORE> what should be the next move for $BID
<CALL> classify $BID into $PID
<CALL> search $MOVE for $BID into $PID
```

```
<CALL> evaluate $BID into $PID
<CALL> repeat prompt into $PID
<EOR>
```

This is not a sequence of linguistic tokens. It is a *procedural script*. The Script Engine executes it line by line, performing the following operations:

- binding variables,
- invoking domain classifiers,
- calling search engines,
- updating board states,
- evaluating win/loss conditions,
- repeating the reasoning loop when necessary,
- terminating when the program ends.

The Script Engine therefore acts as the cognitive substrate of Toy Board: it is the mechanism that turns symbolic reasoning into concrete action.

Variable Memory as Working Memory

Toy Board uses explicit variables such as `$BID`, `$PID`, and `$MOVE` to represent identifiers and intermediate results. The Script Engine maintains a small, deterministic memory map that stores these variables. It enforces:

- variables must be bound before use,
- variable names must remain consistent,
- variable updates must follow the program grammar,
- variable values must be compatible with the domain.

This explicit working memory is one of the key differences between Toy Board and transformer-based models. Instead of relying on implicit attention patterns, Toy Board manipulates variables directly, making reasoning interpretable and reliable.

Domain Interaction Through a Uniform Ontology

The Script Engine interacts with domain engines through a uniform interface consisting of:

- `BoardState`,
- `BoardMove`,
- `BoardDomain`.

Each domain engine implements:

- legal move generation,
- state transitions,
- win/loss evaluation,
- search integration.

Because all domains conform to the same ontology, the Script Engine can execute the same program structure across Connect Four, Nim, MiniChess, Sliding Puzzle, Sudoku, Sorting, and other environments. This is what makes Toy Board a domain-bound AGI system: the reasoning model is domain-agnostic, while the Script Engine and domain engines provide the necessary specialization.

Search as a Cognitive Action

When the Script Engine encounters a search instruction:

```
<CALL> search $MOVE for $BID into $PID
```

it invokes a domain-specific search engine (Monte Carlo or Beam Search). The search engine explores the state space, selects a promising move, and returns it as the value of `$MOVE`. The reasoning model does not learn how to play the game; it learns when to invoke search.

This separation of reasoning and search is one of Toy Board's defining features. It allows a small model to solve complex problems by delegating combinatorial exploration to deterministic tools.

Abort as Failure Awareness

If a search engine cannot find a valid move, or if the program requests an impossible operation, the Script Engine produces an explicit abort:

```
search is unable to find solution
```

Abort is not an error. It is a cognitive signal: the system recognizes that its reasoning strategy cannot succeed and terminates gracefully. This form of failure awareness is a key property of intelligent systems and is difficult to achieve in transformer-based models.

Determinism and Interpretability

The Script Engine is fully deterministic:

- the same program always produces the same result,
- the same board state always leads to the same search behavior,
- variable updates are predictable and traceable,
- abort conditions are explicit and reproducible.

This determinism makes Toy Board:

- easy to debug,

- easy to analyze,
- safe to extend,
- suitable for scientific study.

It also provides a clear separation between the neural and symbolic components of the system, allowing each to be understood independently.

Why the Script Engine Matters

The Script Engine is the component that transforms Toy Board from a language model into a cognitive agent. It provides:

- explicit control flow,
- explicit memory,
- explicit domain interaction,
- explicit search invocation,
- explicit failure detection.

It is the operational core of Toy Board's domain-bound AGI architecture.

In the next section, we examine the search engines themselves and how they function as cognitive tools within this architecture.

16.10 Search Engines as Cognitive Tools

Toy Board does not attempt to learn the internal logic of its domains. Instead, it delegates combinatorial reasoning to external search engines that operate through a uniform interface. The PRE–CAS model learns *when* to invoke search, not *how* to perform it. This separation is central to Toy Board's domain-bound AGI architecture: the neural model handles symbolic reasoning and control flow, while deterministic search engines handle combinatorial exploration.

The system currently supports two search engines:

- a Monte Carlo search engine for adversarial or strategic domains,
- a Beam Search engine for puzzles, sorting, and deterministic planning tasks.

Both engines operate on the same `BoardState`/`BoardMove`/`BoardDomain` ontology, making them interchangeable from the perspective of the Script Engine.

Monte Carlo Search: Exploration Under Uncertainty

The Monte Carlo search engine is used for domains where the agent must evaluate strategic consequences of moves, such as:

- Connect Four,
- Checkers,

- MiniChess,
- Nim,
- Chomp.

The engine performs repeated playouts from the current state, sampling moves until a terminal state is reached or a cutoff depth is exceeded. Each playout contributes a score that reflects the likelihood of winning from that position. The engine then selects the move with the highest empirical value.

Toy Board's Monte Carlo engine includes several practical enhancements:

- **Immediate win detection** to avoid unnecessary playouts.
- **Rollout cutoff** to prevent unbounded simulations.
- **Heuristic evaluation** when terminal states are not reached.
- **Configurable playout count** to trade off speed and accuracy.

These features make the engine robust across a wide range of board sizes and branching factors. The reasoning model does not learn any of these heuristics; it simply invokes `search` when appropriate.

Beam Search: Deterministic Planning and Optimization

Beam Search is used for domains where the environment is deterministic and the goal is to reach a specific configuration:

- Sliding Puzzle,
- Sorting,
- Sudoku (partial),
- other planning-style domains.

Beam Search maintains a fixed-size frontier of the most promising states, expanding them layer by layer until a solution is found or the search space is exhausted. This approach is particularly effective for:

- puzzles with large branching factors,
- optimization tasks with clear heuristics,
- domains where depth is more important than breadth.

Toy Board's Sorting domain is a striking example: the system solves sorting not through a learned algorithm, but through search over swap operations. This demonstrates the power of the board-game ontology: even non-game tasks can be expressed as state transitions and solved through planning.

A Uniform Interface for All Domains

Both search engines operate through the same interface:

- `BoardState` represents the current world configuration.
- `BoardMove` represents an action.
- `BoardDomain` defines legal moves, transitions, and win conditions.

This uniformity allows the Script Engine to invoke search without knowing anything about the domain. The reasoning model simply emits:

```
<CALL> search $MOVE for $BID into $PID
```

and the Script Engine handles the rest.

Search as a Cognitive Action

In Toy Board, search is not an internal neural operation. It is an explicit cognitive action, analogous to a human deciding to:

- calculate variations in a chess position,
- try out moves in a puzzle,
- simulate outcomes before acting.

The reasoning model learns the *conditions* under which search is useful:

- after classification,
- before evaluation,
- when a decision must be made,
- when the board state is ambiguous.

This makes search a first-class citizen in the cognitive architecture.

Abort as a Search Outcome

If a search engine cannot find a valid move or reaches its computational limits, the Script Engine returns an explicit abort:

```
search is unable to find solution
```

This is not an error but a form of metacognition: the system recognizes that its strategy cannot succeed and terminates gracefully. Abort is therefore a meaningful cognitive signal, not a failure of the architecture.

Why Search Engines Matter

Search engines give Toy Board capabilities far beyond what a small neural model could achieve alone:

- **Combinatorial reasoning** without learning domain rules.
- **Generalization** across games, puzzles, and optimization tasks.
- **Scalability** through configurable search depth and breadth.
- **Interpretability** through explicit playouts and frontier expansions.
- **Modularity** through the uniform board-game ontology.

Search is the bridge between symbolic reasoning and world dynamics. It allows Toy Board to solve complex problems with a tiny model by delegating exploration to deterministic tools.

In the next section, we examine how Toy Board maps diverse domains onto a single board-game ontology, enabling this unified search interface.

16.11 A Unified Board-Game Ontology for Heterogeneous Domains

One of Toy Board's most surprising results is that a single, minimal ontology—originally designed for board games—turns out to be expressive enough to represent a wide range of domains, including puzzles, optimization tasks, and sorting. This ontology consists of only three core abstractions:

- `BoardState` — a representation of the current world configuration,
- `BoardMove` — an action that transforms one state into another,
- `BoardDomain` — the rules governing legal moves, transitions, and evaluation.

Despite its simplicity, this interface is powerful. It allows Toy Board to treat Connect Four, Nim, MiniChess, Sliding Puzzle, Sudoku, and Sorting as structurally identical problems: each domain becomes a state-transition system with legal actions and terminal conditions. The reasoning model never learns domain rules; it learns how to operate on this ontology.

A Minimal Ontology with Maximum Reach

The ontology is intentionally small. A `BoardState` may contain:

- a grid of pieces (Connect Four, Checkers, MiniChess),
- a list of numbers (Sorting),
- a tile configuration (Sliding Puzzle),
- a constraint grid (Sudoku),
- or any other structured representation.

A `BoardMove` may represent:

- dropping a piece,
- swapping two elements,
- sliding a tile,
- filling a cell,
- or any domain-specific action.

A `BoardDomain` defines:

- how to enumerate legal moves,
- how to apply a move to a state,
- how to detect terminal states,
- how to evaluate intermediate states,
- which search engine to use.

This uniformity is what makes Toy Board domain-bound AGI: the reasoning model is domain-agnostic, while the domain engines provide the specialization.

Mapping Games, Puzzles, and Sorting onto the Same Structure

The most striking demonstration of the ontology's generality is the Sorting domain. Sorting is not a board game, yet it fits naturally into the ontology:

- `BoardState` is the current list,
- `BoardMove` is a swap operation,
- `BoardDomain` defines legal swaps and a heuristic for sortedness.

Toy Board solves sorting not through a learned algorithm, but through search over swap operations. This is conceptually interesting: the system treats sorting as a planning problem on a board, solved through the same mechanisms as Connect Four or Sliding Puzzle.

Similarly:

- Sliding Puzzle becomes a deterministic planning domain,
- Sudoku becomes a constraint-satisfaction domain,
- Nim and Chomp become impartial combinatorial games,
- MiniChess becomes a strategic adversarial domain.

All of these domains share the same interface, allowing the Script Engine and search engines to operate uniformly across them.

Why a Board-Game Ontology Works

The success of this ontology is not accidental. Many cognitive tasks—games, puzzles, optimization problems—can be expressed as:

1. a state,
2. a set of legal actions,
3. a transition function,
4. a goal condition.

This is precisely the structure of a board game. By adopting this ontology, Toy Board gains:

- **Modularity** — new domains can be added by implementing the interface.
- **Uniformity** — the reasoning model uses the same program structure everywhere.
- **Scalability** — domains do not require retraining the model.
- **Search compatibility** — both Monte Carlo and Beam Search operate seamlessly.
- **Interpretability** — states and moves are explicit and inspectable.

This ontology is the conceptual backbone of Toy Board's domain-bound AGI architecture.

A Cognitive Interface, Not a Data Structure

The board-game ontology is not merely a data structure; it is a *cognitive interface*. It defines how the reasoning model interacts with the world:

- classify the domain,
- search for a move,
- evaluate the resulting state,
- repeat until termination.

Because the ontology is stable and minimal, the PRE–CAS model can learn these operations once and apply them everywhere.

Why This Ontology Matters for AGI

Toy Board demonstrates that AGI does not require a massive model that internalizes all domains. Instead, it requires:

- a small reasoning core,
- a uniform interface for world interaction,
- modular domain engines,
- and deterministic execution.

The board-game ontology is the key that makes this possible. It is the bridge between symbolic reasoning and domain-specific computation.

In the next section, we illustrate this ontology in action through a complete reasoning trace, showing how Toy Board solves a problem from start to finish.

16.12 Full Example: A Complete Reasoning Trace

To illustrate Toy Board's architecture in action, this section presents a complete reasoning trace from input prompt to final answer. The example demonstrates how the PRE–CAS model synthesizes a program, how the Script Engine executes it, and how the domain engines and search engines contribute to the final result. The trace is representative of the system's behavior across all supported domains.

User Prompt

```
What is the best next move for this Connect Four board?
```

The user provides a natural-language question. The PRE–CAS model does not answer this question directly. Instead, it constructs a procedural script that describes how to answer it.

Generated Reasoning Program

Toy Board produces the following program token by token:

```
<STORE> what should be the next move for $BID
<CALL> classify $BID into $PID
<CALL> search $MOVE for $BID into $PID
<CALL> evaluate $BID into $PID
<CALL> repeat prompt into $PID
<EOR>
```

This program is not text; it is a sequence of instructions. Each line corresponds to a cognitive action that the Script Engine will execute deterministically.

Step 1: STORE

The Script Engine binds the user's question to `$BID`. This variable now contains the board identifier and the associated state.

Step 2: Domain Classification

```
<CALL> classify $BID into $PID
```

The Script Engine invokes the domain classifier. It inspects the board and determines that it belongs to the Connect Four domain. The result is stored in `$PID`.

Step 3: Search for a Move

```
<CALL> search $MOVE for $BID into $PID
```

The Script Engine invokes the Monte Carlo search engine associated with Connect Four. The engine performs:

- immediate win detection,
- playout simulations,

- heuristic evaluation for non-terminal playouts,
- selection of the highest-value move.

The chosen move is stored in `$MOVE`.

Step 4: Evaluate the Resulting State

```
<CALL> evaluate $BID into $PID
```

The Script Engine applies the move to the board and evaluates the resulting state:

- if the move wins the game, the evaluation returns `win`,
- if the move loses, it returns `loss`,
- otherwise, it returns `ongoing`.

This evaluation is used to determine whether the reasoning loop should continue.

Step 5: Repeat or Terminate

```
<CALL> repeat prompt into $PID
```

If the evaluation indicates that the game is ongoing, the Script Engine re-enters the reasoning loop with the updated board. If the evaluation indicates a terminal state, the Script Engine terminates early.

In this example, the evaluation is `ongoing`, so the Script Engine would normally repeat. However, because the user asked for a single move, the Script Engine returns the move immediately.

Final Output

The Script Engine returns the selected move in natural language:

```
The best next move is column 3.
```

This output is not generated by the PRE–CAS model. It is produced by the Script Engine after executing the reasoning program. The neural model's role is to construct the program; the Script Engine's role is to execute it.

Why This Trace Matters

This example illustrates several key properties of Toy Board:

- **Reasoning is explicit.** The system constructs a program rather than predicting text.
- **Execution is deterministic.** The Script Engine interprets the program line by line.
- **Search is delegated.** The neural model does not learn game strategies; it invokes search engines.
- **Domain logic is external.** The reasoning model never learns the rules of Connect Four.

- **Failure is explicit.** If search cannot find a move, the Script Engine returns an abort.
- **Generalization is structural.** The same program structure works for Connect Four, Nim, Sliding Puzzle, Sorting, Sudoku, and other domains.

This trace demonstrates Toy Board's core idea: a small neural model can solve complex problems by synthesizing programs that orchestrate modular cognitive tools.

In the next section, we analyze how this architecture scales and why it provides a practically feasible path toward AGI.

16.13 Scaling Analysis: Why Toy Board Suggests a Feasible Path to AGI

The Red Thread: Why Minimal Structure Scales Better Than Complexity

Throughout this book, one idea has quietly guided every chapter: *general intelligence does not emerge from complexity, but from the right decomposition.*
Triadic Cosmos argues that AGI is not a matter of building ever-larger monolithic models, but of identifying the smallest set of cognitive functions that can be composed, reused, and scaled across domains. Toy Board is the first operational proof of this principle.
By reducing reasoning to a minimal cognitive vocabulary, by externalizing domain logic into modular engines, and by expressing cognition as explicit programs executed by a deterministic interpreter, Toy Board shows that intelligence can grow through structure rather than size.
This is the red thread of the entire book: **a small reasoning core, connected to a growing ecosystem of tools through a stable cognitive interface, scales more efficiently than any monolithic architecture.**
The following scaling analysis makes this claim explicit. It shows why a domain-bound AGI like Toy Board can remain compact while expanding to hundreds or thousands of domains, and why monolithic transformer scaling cannot match this trajectory.

Toy Board is intentionally small. The reasoning model is roughly one megabyte, the Coherence model is even smaller, and the domain engines are compact deterministic modules. Yet the system solves heterogeneous tasks—games, puzzles, sorting, planning— through a unified cognitive architecture. This raises an important question: what does Toy Board imply about the scalability of AGI?

The central claim of this chapter is that Toy Board demonstrates a *structurally scalable* path to AGI. The system does not scale by increasing model size, but by increasing the number of domains and tools connected through a stable cognitive interface. This section analyzes why this approach is feasible, how it differs from transformer-based scaling, and what it suggests about the future of AGI architectures.

Scaling Through Architecture, Not Parameters

Transformer-based systems scale by increasing parameter count. To internalize more domains, they must learn more patterns, more rules, and more correlations. This leads to rapidly increasing model sizes—hundreds of billions of parameters for broad competence, and potentially far more for general intelligence.

Toy Board takes the opposite approach:

- the reasoning model remains small,
- domain logic is externalized into modular engines,
- search handles combinatorial complexity,
- the Script Engine provides deterministic execution,
- the board-game ontology provides a uniform interface.

Scaling occurs by adding new domains, not by enlarging the model. Each new domain implements the same interface and plugs into the same reasoning pipeline.

The Cost of Internalizing Domain Logic

If a transformer were to internalize the rules of Connect Four, Nim, MiniChess, Sliding Puzzle, Sudoku, and Sorting, it would require:

- embeddings for all domain states,
- learned representations of legal moves,
- implicit transition functions,
- implicit evaluation heuristics,
- implicit search-like behavior.

This is extremely parameter-intensive. Toy Board avoids this cost entirely by delegating domain logic to deterministic engines. The reasoning model only needs to know:

- how to classify a domain,
- how to invoke search,
- how to evaluate results,
- how to repeat or terminate.

This is a small cognitive vocabulary, not a large knowledge base.

Scaling Through Tool Integration

Toy Board demonstrates that a small model can orchestrate powerful tools:

- Monte Carlo search for adversarial domains,
- Beam Search for puzzles and planning,
- domain engines for state transitions,
- the Script Engine for execution.

As more tools are added, the system becomes more capable without increasing the size of the reasoning model. This is analogous to human cognition: intelligence grows by acquiring tools, not by enlarging the brain.

The Role of the Board-Game Ontology

The board-game ontology is the key to Toy Board's scalability. It provides:

- a stable interface for new domains,
- a uniform structure for search,
- a predictable format for reasoning programs,
- a shared language for the Script Engine.

Because the ontology is minimal and domain-agnostic, it can support an unbounded number of environments. Adding a new domain requires no retraining of the reasoning model.

A Feasible Path to Large-Scale AGI

Toy Board suggests that a full AGI system could be built using the same principles:

- a small reasoning core (tens of megabytes),
- a large library of domain engines (gigabytes),
- a large library of search and planning tools,
- a stable cognitive interface connecting them.

Such a system would scale linearly with the number of domains, not exponentially with the complexity of the world. It would be:

- interpretable,
- modular,
- debuggable,
- extensible,
- computationally feasible.

This stands in contrast to monolithic transformer scaling, which becomes increasingly expensive and opaque as models grow.

Why Toy Board Matters

Toy Board is not a toy. It is a proof of concept for a different kind of AGI architecture:

- one that separates reasoning from world knowledge,
- one that delegates complexity to modular engines,
- one that uses explicit programs instead of implicit activations,
- one that scales through structure rather than size.

It demonstrates that general intelligence may not require massive models, but the right architecture—one that reasons, plans, and decides through explicit cognitive mechanisms.

In the next section, we discuss the implications of this architecture for future research and how Toy Board can evolve into a larger, more capable AGI system.

Why the 10^6 Factor Is a Lower Bound

Toy Board suggests that a domain-bound AGI architecture can be orders of magnitude more parameter-efficient than a monolithic transformer. A conservative estimate places the efficiency gap at a factor of 10^6 (one million). This section explains where that factor comes from and why it is almost certainly an *underestimate*.

1. Internalizing Domain Logic vs. Externalizing It A transformer must internalize:

- the full state space of each domain,
- the legal move generator,
- the transition function,
- the evaluation heuristics,
- search-like behavior,
- and the natural-language mapping on top.

Toy Board externalizes all of this into deterministic engines.
For a domain like Connect Four:

$$|\text{state space}| \approx 4.5 \times 10^{12}$$

A transformer must compress this into embeddings and attention patterns. Toy Board stores none of it; it only stores the *ability to call* the domain engine.
This alone yields a parameter reduction of at least:

$$10^3 \text{ to } 10^4 \text{ per domain.}$$

With 100–1000 domains, this becomes:

$$10^5 \text{ to } 10^7.$$

2. Search as a Learned Behavior vs. a Tool Transformers must approximate search implicitly through:

- deep attention stacks,
- multi-step reasoning patterns,
- chain-of-thought expansions,
- and massive training corpora.

Toy Board performs search explicitly through Monte Carlo or Beam Search.
Empirically, Monte Carlo search with even 1000 playouts corresponds to:

$$10^3 \text{ to } 10^5 \text{ forward passes of a transformer.}$$

Toy Board replaces this with a single `CALL search` instruction.
This yields another factor of:

$$10^3 \text{ to } 10^5.$$

3. Program Synthesis vs. Token Prediction Transformers must encode:

- control flow,
- variable binding,
- memory,
- recursion,
- and termination

implicitly in their weights.
Toy Board encodes these explicitly in:

- the cognitive vocabulary,
- the Script Engine,
- the paging system,
- and the program grammar.

This removes the need for the model to learn these behaviors, reducing parameter requirements by another factor of:

$$10^2 \text{ to } 10^3.$$

4. Combined Lower Bound Multiplying the conservative factors:

$$10^3 \times 10^3 \times 10^2 = 10^8.$$

Even with pessimistic assumptions:

$$10^2 \times 10^2 \times 10^2 = 10^6.$$

Thus the millionfold factor is not an upper bound but a *lower bound*. It represents the minimum structural advantage of an AGI architecture that:

- externalizes domain logic,
- externalizes search,
- externalizes control flow,
- and uses explicit programs instead of implicit activations.

5. Why This Matters This analysis suggests that:

- A transformer-based AGI would require *exabyte-scale* models.
- A modular AGI like Toy Board could remain in the *gigabyte* range.
- The gap between the two approaches grows with each new domain.

Toy Board therefore provides not only a conceptual blueprint but also a quantitative argument for why AGI must be modular, tool-driven, and program-executing rather than monolithic and correlation-driven.

16.14 Implications and Future Directions

Toy Board is intentionally small, but its architectural implications are large. It demonstrates that a general reasoning system does not require a monolithic model that internalizes all domains, rules, heuristics, and search behaviors. Instead, it shows that intelligence can emerge from the interaction between a compact reasoning core, a deterministic interpreter, modular domain engines, and explicit cognitive tools. This section explores the broader implications of this architecture and outlines several directions for future research.

Implication 1: Reasoning as Program Synthesis

Toy Board reframes reasoning as program synthesis rather than token prediction. The PRE–CAS model constructs explicit cognitive programs that the Script Engine executes deterministically. This has several consequences:

- reasoning becomes interpretable and inspectable,
- errors become debuggable,
- behavior becomes reproducible,
- and the system can be extended without retraining.

This suggests that future AGI systems may rely on explicit program generation rather than implicit reasoning encoded in neural activations.

Implication 2: Externalized Knowledge and Tools

Toy Board externalizes:

- domain rules,
- search algorithms,
- evaluation heuristics,
- and control flow.

This reduces the burden on the neural model and allows the system to scale by adding new tools rather than increasing parameter count. It also mirrors human cognition: we use external tools—calculators, diagrams, simulations—to extend our reasoning capabilities.

Implication 3: A Stable Cognitive Interface

The board-game ontology provides a stable interface for world interaction. Because all domains implement the same `BoardState`/`BoardMove`/`BoardDomain` structure, the reasoning model can operate uniformly across heterogeneous environments. This suggests that AGI may require:

- a small number of stable cognitive interfaces,
- rather than a single massive model that internalizes everything.

The ontology acts as a cognitive API: a contract between reasoning and action.

Implication 4: Separation of Concerns Enables Modularity

Toy Board separates:

- reasoning (PRE–CAS),
- execution (Script Engine),
- world dynamics (domain engines),
- and combinatorial exploration (search engines).

This separation allows each component to evolve independently. New domains can be added without retraining the model. New search engines can be introduced without changing the ontology. This modularity is essential for long-term scalability.

Implication 5: A Quantitative Argument for Modular AGI

The scaling analysis in the previous section shows that a modular AGI architecture can be at least 10^6 times more parameter-efficient than a monolithic transformer. This is not a speculative claim; it follows directly from:

- the cost of internalizing domain logic,
- the cost of approximating search,
- the cost of implicit control flow,
- and the cost of embedding large state spaces.

Toy Board therefore provides a quantitative argument for why AGI must be modular, tool-driven, and program-executing.

Future Direction 1: Expanding the Domain Library

Toy Board currently supports a diverse set of domains, but the architecture can scale to hundreds or thousands. Future work may include:

- physics simulations,
- algebraic manipulation,
- planning in continuous spaces,
- multi-agent environments,
- real-world robotics abstractions.

Each new domain strengthens the case for the ontology's generality.

Future Direction 2: Richer Cognitive Vocabularies

The current cognitive vocabulary is intentionally minimal. Future versions may introduce:

- conditional branching,
- loops with explicit counters,
- subroutine definitions,
- memory stacks,
- or higher-level reasoning primitives.

These additions would allow Toy Board to express more complex reasoning strategies.

Future Direction 3: Multi-Step Reasoning and Meta-Search

Toy Board already supports repeated reasoning through the `repeat` instruction. Future extensions may include:

- meta-search over search parameters,
- adaptive playout strategies,
- self-reflective evaluation of reasoning traces,
- or hierarchical planning.

This would move the system closer to human-like strategic reasoning.

Future Direction 4: Integration with Natural Language Interfaces

Toy Board currently uses natural language only as input. Future systems may:

- generate natural-language explanations of reasoning,
- translate programs into human-readable steps,
- or integrate with LLMs as front-end interpreters.

This would make the system more accessible and easier to deploy.

Future Direction 5: Toward a Full AGI Architecture

Toy Board is a prototype, but it demonstrates the essential ingredients of a scalable AGI architecture:

- explicit reasoning,
- deterministic execution,
- modular domain engines,
- externalized search,

- and a stable cognitive interface.

A full AGI system may consist of:

- a reasoning core of tens of megabytes,
- a library of domain engines of gigabytes,
- a suite of search and planning tools,
- and a cognitive API connecting them.

Toy Board shows that such a system is not only possible but practical.

In the next section, we conclude the chapter by summarizing the architectural principles that make Toy Board a compelling blueprint for domain-bound AGI.

16.15 Conclusion — A Practical Example of Domain-Bound AGI

The previous chapter introduced a conceptual blueprint for domain-bound AGI: a system that separates reasoning from world knowledge, delegates combinatorial complexity to external tools, and interacts with domains through a stable cognitive interface. Toy Board is the first practical instantiation of that blueprint. It demonstrates that the architecture is not only theoretically sound but operationally viable, even at extremely small scales.

Toy Board shows that a one-megabyte reasoning model, combined with a deterministic interpreter and modular domain engines, can solve heterogeneous tasks across games, puzzles, and optimization problems. It does so not by memorizing domain rules or approximating search, but by synthesizing explicit cognitive programs that orchestrate external tools. This is the essence of domain-bound AGI: intelligence emerges from the coordination of modules, not from the size of a monolithic model.

A Working Instantiation of the Blueprint

Toy Board realizes every major component of the blueprint:

- **Explicit reasoning** through program synthesis (PRE–CAS).
- **Deterministic execution** through the Script Engine.
- **A stable cognitive interface** through the board-game ontology.
- **Externalized domain logic** through modular domain engines.
- **Externalized combinatorial reasoning** through Monte Carlo and Beam Search.
- **A minimal learned component** through the Coherence model.

Each component is small, interpretable, and replaceable. Together, they form a coherent cognitive system capable of solving tasks far beyond what its parameter count would suggest.

A Demonstration of Structural Generalization

Toy Board's most important contribution is its demonstration of *structural* generalization. The reasoning model does not learn the rules of Connect Four, Nim, MiniChess, Sliding Puzzle, Sudoku, or Sorting. Instead, it learns:

- how to classify a domain,
- how to invoke search,
- how to evaluate outcomes,
- how to repeat or terminate,
- and how to express these steps as a program.

Because the cognitive vocabulary and ontology are stable, the same reasoning structure applies across all domains. This is the hallmark of AGI: the ability to reuse cognitive processes across heterogeneous environments.

A Quantitative Argument for Modularity

Toy Board also provides a quantitative argument for why AGI must be modular. As shown in the scaling analysis, a monolithic transformer would require at least a 10^6-fold increase in parameters to internalize the same domain logic, search behavior, and control flow that Toy Board externalizes. This factor is a lower bound, derived from:

- the size of domain state spaces,
- the cost of approximating search,
- the cost of implicit control flow,
- and the cost of embedding heterogeneous domains.

Toy Board therefore demonstrates that modularity is not only conceptually elegant but computationally necessary.

A Path Forward

Toy Board is not a full AGI system, but it is a working prototype of the architecture that could support one. It shows that:

- reasoning can be explicit,
- execution can be deterministic,
- domains can be modular,
- search can be external,
- and intelligence can scale through structure rather than size.

Future systems may extend this architecture with richer cognitive vocabularies, more powerful search engines, larger domain libraries, and more sophisticated interpreters. But the core insight remains: AGI does not require a monolithic model. It requires the right architecture.

Toy Board is the first practical example of that architecture—a small system that points toward a scalable, interpretable, and computationally feasible path to general intelligence.

From Prototype to Ontology

Toy Board is more than a demonstration; it is a working specimen of a domain-bound AGI. It shows that intelligence can be constructed from explicit reasoning programs, deterministic interpreters, modular domain engines, and external cognitive tools. It proves that a small model can coordinate complex behaviors across heterogeneous environments when supported by the right architecture. And it provides a quantitative argument for why modularity, structure, and explicit interfaces are not engineering conveniences but cognitive necessities.

Yet Toy Board is only one instantiation of a deeper idea. The architecture it embodies—explicit memory, structured reasoning, domain-specific experts, search-as-a-tool, and a stable cognitive interface—reflects principles that extend far beyond games, puzzles, or sorting. These principles point toward a general theory of intelligence: a decomposition of cognition into functional roles that can be instantiated, combined, and scaled.

The next chapter develops this theory. It introduces an ontology for AGI: a model-agnostic framework that identifies the fundamental cognitive functions required for general intelligence and clarifies how they interact. Where Toy Board provided the *example*, the ontology provides the *explanation*. Where Toy Board showed *how* domain-bound AGI works, the ontology explains *why* it must work that way. Together, they form a bridge from concrete systems to general principles, from implementation to theory, and from domain-bound intelligence to a compositional architecture for AGI.